COLLÈGE DE PARIS SUPÉRIEUR

RÉPUBLIQUE TOGOLAISE

Information Technology

Tronc Commun

Travail – Liberté – Patrie

INF-ASM-B2

Architecture et Programmation en Assembleur

Microcontrôleurs AVR, assembleur x64, bas niveau et systèmes embarqués

Fondements matériels et programmation bas niveau

Architecture des ordinateurs, microcontrôleurs AVR, registres, mémoire, interruptions, timers, E/S, compilation croisée, AVR-GCC, AVRDude, NASM et architecture x64

AVR · Assembleur · Toolchain · TP

Bachelor 2 (IT) | Semestre 2 | 24h (CM + TD + TP) | 2 ECTS

Enseignant : Dr AFENYIVEH Dzo M. Serge

dzo.serge@gmail.com | d.afenyiveh@univkara.com

(+228) 91 75 98 33 | (+228) 98 35 23 31

Collège de Paris Supérieur | 2025–2026

Version 4.2 | Mise à jour : 19 avril 2026

Cours et Travaux Dirigés

Architecture des microcontrôleurs, langage assembleur, programmation bas niveau et mises en pratique

Sommaire

Syllabus du cours **1**

1 Fondements préliminaires **15**
1.1 Systèmes de numération 17
1.2 Représentation des entiers 27
1.3 Logique booléenne 31
1.4 Machine à états et cycle d'exécution 35
1.5 Travaux dirigés du chapitre 0 42
1.6 Quiz du chapitre 0 : testez vos connaissances 53

2 Introduction à l'architecture des microcontrôleurs AVR **59**
2.1 Qu'est-ce qu'un microcontrôleur ? 62
2.2 Histoire et évolution des microcontrôleurs 68
2.3 Microcontrôleur vs microprocesseur : quelle différence ? 69
2.4 Les microcontrôleurs dans le monde réel 73
2.5 La famille AVR : notre terrain de jeu 75
2.6 Architecture interne : visite guidée de l'ATmega328P 78
2.7 L'horloge, le reset et le cycle d'instruction 90
2.8 Compilation croisée et développement pour AVR 93
2.9 Mise en place de l'environnement et premier programme 99
2.10 Conclusion du chapitre 104
2.11 Travaux dirigés du chapitre 1 105
2.12 Quiz du chapitre 1 : testez vos connaissances 115

3 Fondements du langage assembleur **123**
3.1 Du langage machine à l'assembleur : traduction et mnémoniques 125
3.2 Structure d'un programme assembleur AVR 128

3.3 Registres de travail et registres spéciaux 132
3.4 Instructions de transfert : LDI, MOV, LDS, STS 135
3.5 Premier programme : analyse ligne par ligne 139
3.6 Comparaison avec l'assembleur x64 (ouverture) 141
3.7 Conclusion du chapitre 142
3.8 Travaux dirigés du chapitre 2 144
3.9 Quiz du chapitre 2 : testez vos connaissances 155

Syllabus du cours

Architecture et Programmation en Assembleur

Comprendre la machine, programmer au plus près du matériel

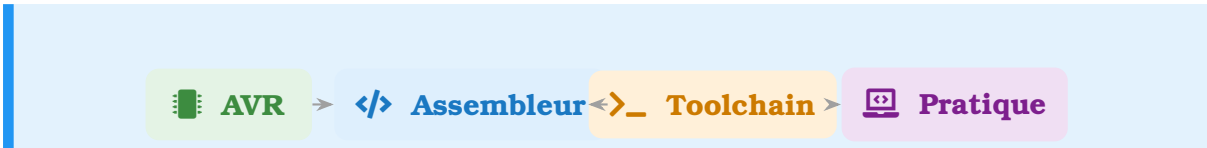
Bachelor 2 (IT) – Tronc commun – Semestre 2

Collège de Paris Supérieur

Description du cours

Ce cours introduit les étudiants à la **programmation bas niveau** et aux fondements de l'**architecture des systèmes numériques**. Il répond à une question centrale : *comment une machine exécute-t-elle réellement un programme ?*

À travers l'étude du langage assembleur et de l'architecture **AVR**, les étudiants découvrent la représentation des données, le fonctionnement du processeur, l'organisation mémoire et l'interaction directe avec le matériel.



Informations générales

Intitulé	Architecture et Programmation en Assembleur
Code	INF-ASM-B2
Niveau	Bachelor 2 (IT)
Semestre	Semestre 2
Volume horaire	**24 heures** (CM : 8h, TD : 8h, TP : 8h)
Crédits	2 ECTS
Institution	Collège de Paris Supérieur
Enseignant	Dr AFENYIVEH Dzo M. Serge
Contact	dzo.serge@gmail.com
Prérequis	• Bases en algorithmique et programmation • Notions de numération binaire et hexadécimale • Autonomie minimale sur ordinateur

Objectifs pédagogiques

À l'issue de ce module, l'étudiant sera capable de :

- **Identifier et expliquer** les composants essentiels d'un microcontrôleur (CPU, ALU, registres, mémoire).
- **Distinguer** langage machine, assembleur et langage de haut niveau.
- **Lire, analyser et interpréter** un programme assembleur AVR.
- **Écrire et déboguer** des programmes assembleur simples avec instructions de transfert, calcul et branchement.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- **Manipuler** la pile, les sous-programmes et les entrées/sorties numériques.
- **Utiliser** les outils de développement, de simulation et de programmation.

Compétences développées

Conceptuelles
Structure des systèmes numériques

Techniques
Lecture et écriture assembleur

Méthodologiques
Décomposition bas niveau

Pratiques
Simulation et débogage

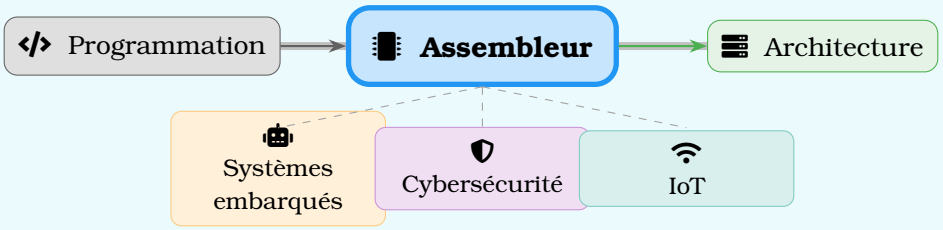
Transversales
Rigueur et analyse

Réf.	Compétence détaillée	Niveau
C1	Comprendre la structure d'un microcontrôleur et le rôle de chaque composant	Maîtrise
C2	Lire, analyser et écrire du code assembleur AVR	Maîtrise
C3	Manipuler registres, mémoire, pile et configurer des E/S	Application
C4	Utiliser les outils de développement et de simulation	Application
C5	Décomposer un problème et raisonner au niveau instruction	Maîtrise

Maîtrise = autonomie attendue **Application** = mise en œuvre guidée

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Positionnement dans la formation



Ce cours établit un lien fondamental entre la **programmation de haut niveau** et le **fonctionnement matériel**. Il prépare aux enseignements avancés en architecture, systèmes d'exploitation et applications embarquées.

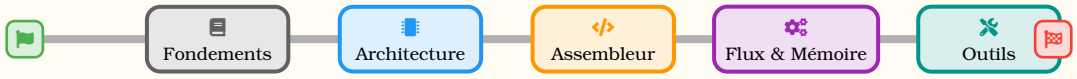
Prérequis

Les prérequis attendus pour ce cours sont :

- ✔ Bases élémentaires en **algorithmique et programmation**
- ✔ Compréhension des **variables, conditions et boucles**
- ✔ Familiarité avec les **nombres binaires et hexadécimaux**
- ✔ Autonomie de base dans l'usage d'un **ordinateur et d'un éditeur**

Une mise à niveau est prévue en début de cours pour harmoniser les acquis.

Programme du cours



 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Chapitre 0 — Fondements préalables 4h

- Numération : binaire, décimale, hexadécimale, conversions
- Représentation des données : entiers signés/non signés, complément à deux
- Logique booléenne : AND, OR, NOT, XOR, tables de vérité
- Notion d'état machine et cycle d'exécution

Chapitre 1 — Architecture des microcontrôleurs AVR 4h

- Introduction aux microcontrôleurs : définition, domaines d'application et présentation de la famille AVR.
- Architecture interne : processeur (CPU), unité arithmétique et logique (ALU) et registres généraux (`R0-R31`).
- Organisation mémoire : mémoire Flash (programme), SRAM (données) et EEPROM (stockage non volatile).
- Horloge système, mécanisme de réinitialisation (*reset*) et cycle d'instruction.
- Comparaison entre microprocesseur et microcontrôleur.

Chapitre 2 — Fondements du langage assembleur 4h

- Du langage machine à l'assembleur : traduction, mnémoniques
- Structure d'un programme : directives, sections, étiquettes, commentaires
- Registres de travail et registres spéciaux (SREG, SP, PC)
- Instructions de transfert : `LDI`, `MOV`, `LDS`, `STS`
- Premier programme : analyse ligne par ligne

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Chapitre 3 — Calculs et contrôle du flux — 4h

- Instructions arithmétiques : `ADD`, `SUB`, `INC`, `DEC`, `MUL`
- Instructions logiques : `AND`, `OR`, `EOR`, `COM`, `NEG`
- Registre d'état SREG : drapeaux Z, C, N, V, S, H
- Comparaisons : `CP`, `CPI`, `TST`
- Branchements : `BREQ`, `BRNE`, `BRLO`, `BRSH`, `RJMP`
- Structures de contrôle : boucles, conditions

Chapitre 4 — Mémoire, pile et sous-programmes — 4h

- Organisation détaillée de la mémoire SRAM
- Modes d'adressage : direct, indirect (X, Y, Z), avec déplacement
- La pile : principe LIFO, pointeur de pile SP, `PUSH`/`POP`
- Sous-programmes : `RCALL`, `RET`, passage de paramètres
- Sauvegarde et restauration du contexte

Chapitre 5 — Entrées/sorties numériques — 4h

- Ports d'E/S : registres DDRx, PORTx, PINx
- Configuration des broches : entrée, sortie, pull-up interne
- Applications pratiques : LED, bouton-poussoir, anti-rebond logiciel
- Temporisations par boucles logicielles
- Introduction aux interruptions et aux timers (principes)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Chapitre 6 — Outils de développement et ouvertures 2h

- Chaîne de développement : édition → assemblage → simulation → programmation
- Outils : Microchip Studio, AVRDude, Tinkercad Circuits, Proteus
- Débogage : points d'arrêt, pas à pas, inspection des registres
- Ouvertures : systèmes embarqués, IoT, robotique, cybersécurité

Planning des séances

Séance	Chap.	Contenu principal	Format
S1	Ch. 0	Mise à niveau : numération, logique booléenne, architecture de base	CM + TD
S2	Ch. 1	Architecture AVR : composants internes, mémoire, cycle d'instruction	CM + TD
S3	Ch. 2	Syntaxe assembleur, registres, instructions de transfert	CM + TD
S4	Ch. 3	Calculs arithmétiques/logiques, comparaisons, branchements	TD + TP
S5	Ch. 4–5	Mémoire, pile, sous-programmes, E/S numériques	TP
S6	Ch. 6	Outils, simulation, révision, synthèse et **évaluation finale**	TP + Éval

Volume total : 24 heu **Durée séance :** 4 heure **Période :** 6 semaines

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Approches pédagogiques

- **Cours magistraux** avec démonstrations en direct et illustrations visuelles.
- **Travaux dirigés** : analyse de code, exercices de conversion, traçage d'exécution.
- **Travaux pratiques** : simulation sur Tinkercad/Proteus, manipulation réelle sur Arduino.
- **Progression spiralaire** : les concepts sont revisités avec une complexité croissante.
- **Accompagnement** : exercices corrigés, retours individualisés, sessions de questions.

Principe directeur : Comprendre *pourquoi* avant de coder *comment*. Chaque instruction est reliée à son effet réel sur la machine.

Conseils pour réussir

1. **Assiduité** : assister à toutes les séances (CM, TD, TP).
2. **Révision active** : relire et résumer les notes après chaque cours.
3. **Pratique régulière** : rejouer les exemples sur simulateur.
4. **Traçage manuel** : exécuter les programmes à la main, registre par registre.
5. **Documentation** : commenter systématiquement son code.
6. **Curiosité** : poser des questions sans attendre.
7. **Collaboration** : échanger avec les pairs (sans copier).

© 2025–2026 Dr AFENYIVEH Dzo M. Serge

Modalités d'évaluation

15%
Quiz & Exercices
Vérification régulière

→

35%
TD + TP
Analyse & Pratique

→

50%
Examen final
Synthèse individuelle

Composante	Poids	Description
Quiz et exercices	15%	Vérifications courtes des notions récentes
Travaux dirigés	15%	Analyse, lecture et écriture de séquences assembleur
Travaux pratiques	20%	Simulation, manipulation, tests sur matériel ou simulateur
Examen final	50%	Épreuve individuelle couvrant l'ensemble du programme

Critères d'évaluation

L'évaluation porte sur :

- la **compréhension** des concepts architecturaux ;
- la **rigueur** syntaxique et logique du code ;
- la capacité à **lire et expliquer** du code assembleur ;
- l'aptitude à **produire** une solution correcte et justifiée.

Session de rattrapage : selon le calendrier académique, elle remplace uniquement l'exame

© 2025–2026 Dr AFENYIVEH Dzo M. Serge

Outils et ressources techniques

Environnements de développement

- **Microchip Studio** : IDE complet (édition, assemblage, simulation, débogage)
- **AVRDude** : programmation en ligne de commande
- **Arduino IDE** : support simplifié pour des tests rapides

Simulateurs en ligne

- **Tinkercad Circuits** : simulation gratuite sans installation
- **Wokwi** : simulateur AVR moderne

Simulateurs avancés

- **Proteus** : simulation de circuits complexes
- **SimulIDE** : alternative open source

Matériel pédagogique

- Carte Arduino UNO (ATmega328P)
- LED, résistances (220 Ω, 10 kΩ)
- Boutons-poussoirs
- Plaque d'essai (*breadboard*)
- Câbles de connexion

`Microchip Studio` `Tinkercad` `Proteus` `Arduino`

Supports pédagogiques fournis

- **Polycopié de cours** complet
- **Fiches de synthèse** par chapitre
- **Séries de TD** avec corrigés détaillés
- **Exemples de programmes** commentés
- **Guides de TP** pas à pas
- **Tableaux de référence** (instructions, registres)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Références bibliographiques

Ouvrages de référence :

[1] **Mazidi, Naimi & Naimi**
AVR Microcontroller and Embedded Systems : Using Assembly and C
Pearson Education. ★★★

[2] **Hyde, R.**
The Art of Assembly Language
No Starch Press. ★★★

[3] **Patterson, D. & Hennessy, J.**
Computer Organization and Design : The Hardware/Software Interface
Morgan Kaufmann. ★★

[4] **Tanenbaum, A. & Austin, T.**
Structured Computer Organization
Pearson. ★★

Documentation technique :

- **AVR Instruction Set Manual** : https://ww1.microchip.com/downloads/en/DeviceDoc/AVR-InstructionSet-Manual-DS40002198.pdf
- **ATmega328P Datasheet** : https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf

Ressources en ligne :

- **AVR Freaks** : https://www.avrfreaks.net/ – Communauté et forums
- **Arduino Documentation** : https://docs.arduino.cc/
- **Tinkercad Circuits** : https://www.tinkercad.com/circuits
- **Ben Eater (YouTube)** : https://www.youtube.com/c/BenEater –

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Tutoriels vidéo

Cadre académique et intégrité

Présence et participation :

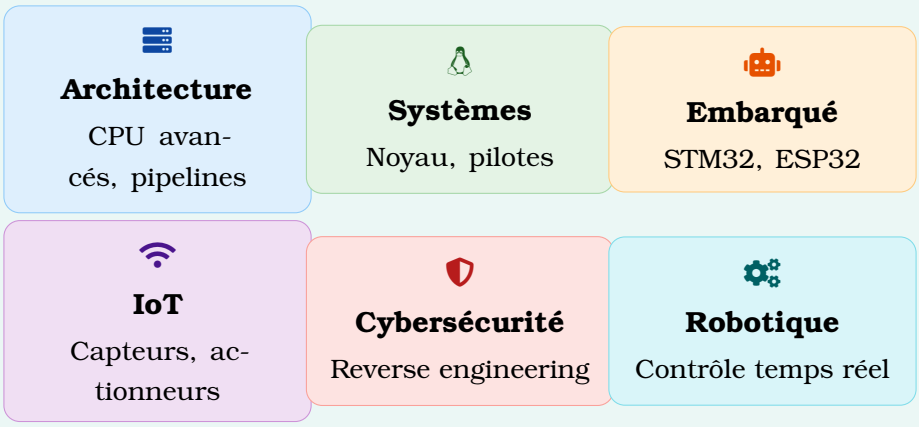
- La présence aux CM, TD et TP est **obligatoire**.
- Toute absence doit être justifiée dans les délais prévus par le règlement.
- L'implication active et la régularité du travail sont attendues.

Intégrité académique :
Le plagiat, la copie et toute forme de fraude sont **strictement interdits**. Cela inclut :

- la copie de code sans citation appropriée ;
- l'usage non autorisé d'outils externes pendant les évaluations ;
- toute communication non autorisée pendant une épreuve.

Les infractions sont passibles de sanctions disciplinaires.

Perspectives et prolongements



 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

*Ce cours constitue une **fondation essentielle** pour comprendre le fonctionnement interne des systèmes numériques et ouvre des perspectives vers de nombreux domaines de l'informatique et de l'électronique.*

Synthèse du cours

Ce cours repose sur **cinq principes fondamentaux** :

1. **Consolider les bases** avant d'aborder le bas niveau.
2. **Comprendre l'architecture** avant d'écrire du code.
3. **Pratiquer régulièrement** sur des cas simples et progressifs.
4. **Relier chaque instruction** à son effet réel sur la machine.
5. **Ouvrir progressivement** vers les systèmes embarqués et l'IoT.

L'objectif n'est pas seulement d'apprendre une syntaxe, mais d'acquérir une **vision durable** du fonctionnement interne des systèmes numériques.

> *« Comprendre l'assembleur, c'est voir ce que la machine fait vraiment quand un langage de haut niveau semble simple. »*

Collège de Paris Supérieur

Contact enseignant : dzo.serge@gmail.com

Année académique 2025–2026

Ce syllabus peut faire l'objet d'ajustements mineurs en cours de semestre, sans modification

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

de ses objectifs fondamentaux.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

CHAPITRE I

Fondements préliminaires

Chapitre 0

Avant de programmer la machine, apprenons à penser comme elle

4 heures | Théorie fondamentale | Exercices pratiques

Pourquoi ce chapitre est essentiel

Ce chapitre pose les **fondations scientifiques indispensables** à tout le cours. Avant d'aborder les registres, les instructions assembleur, l'architecture des processeurs ou les systèmes embarqués, il faut comprendre comment une machine numérique :

- **représente les nombres** (binaire, hexadécimal) ;
- **code les valeurs négatives** (complément à deux) ;
- **raisonne en termes logiques** (AND, OR, NOT, XOR) ;
- **change d'état** et **exécute des instructions**.

Ces notions sont omniprésentes dans votre quotidien numérique :

- Calculatrice de poche
- Thermomètre numérique
- Montre digitale
- Feu tricolore automatique
- Télécommande infrarouge
- Smartphone et WiFi

« Une machine numérique ne comprend ni les mots, ni l'intuition humaine : elle comprend des bits, des états et des règles précises. »

◎ Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

1. **Convertir** un nombre entre les bases décimale, binaire et hexadécimale.
2. **Expliquer** le principe d'un système de numération positionnel.
3. **Distinguer** une représentation signée d'une représentation non signée.
4. **Appliquer** le complément à deux pour représenter les entiers négatifs.
5. **Construire** et **interpréter** des tables de vérité.
6. **Utiliser** les opérations logiques AND, OR, NOT et XOR.
7. **Décrire** une machine à états finis et le cycle d'exécution d'un processeur.

🔑 Prérequis

Ce chapitre ne suppose pas de connaissances avancées, mais il est utile de :

- ✔ Comprendre les puissances entières ($2^3 = 8$, $10^2 = 100$, etc.)
- ✔ Savoir qu'un signal numérique peut prendre deux états (haut/bas, 0/1)
- ✔ Être capable de suivre un raisonnement logique simple
- ✔ Manipuler des tableaux et des schémas élémentaires

Bonne nouvelle : tout est repris progressivement, avec des exemples concrets !

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

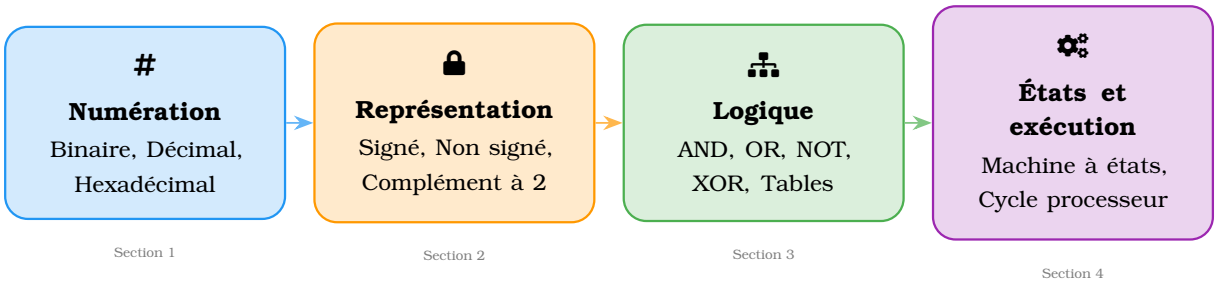


Figure 1.1 – Organisation logique du chapitre : des fondements de la numération jusqu'au cycle d'exécution du processeur.

1.1 Systèmes de numération

1.1.1 Des doigts aux transistors : pourquoi compter en base 2 ?

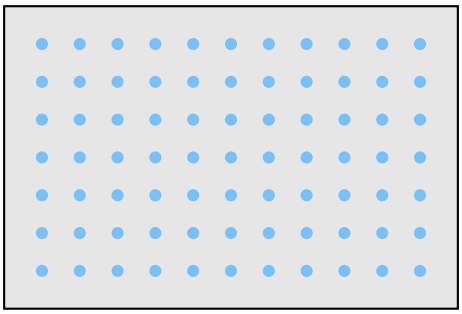
Dans la vie courante, nous comptons en base **10** parce que nous avons dix doigts. Mais un circuit électronique ne possède pas de doigts : il distingue uniquement deux niveaux de tension. C'est pourquoi les machines utilisent le système **binaire** (base 2).

Le système **hexadécimal** (base 16) est une notation compacte du binaire, très utilisée par les programmeurs pour représenter des adresses mémoire, des codes couleur ou des identifiants réseau.



Figure 1.2 – Les deux niveaux logiques d'un circuit numérique.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



Vue microscopique — millions de transistors

Figure 1.3 – Transistors sur un circuit intégré vus au microscope électronique. Chaque transistor fonctionne comme un interrupteur à deux états (passant/-bloqué), réalisant physiquement le 0 et le 1 du système binaire. Un processeur moderne contient plusieurs milliards de ces composants élémentaires.
Source : YouTube — https://www.youtube.com/watch?v=U885cIhOXBM

Exemple du quotidien : la télécommande infrarouge

Quand vous appuyez sur `Volume+` d'une télécommande, le téléviseur ne reçoit pas le texte « volume ». Il reçoit une **séquence codée de bits** sous forme d'impulsions infrarouges :

`Power` → `10100101 11011010` (protocole NEC, 32 bits)

Chaque touche correspond à un code binaire unique, décodé par le micro-contrôleur du téléviseur.

1.1.2 Principe d'un système positionnel

Système de numération positionnel

Dans un système de numération **positionnel** de base b, la valeur d'un nombre s'obtient en multipliant chaque chiffre par une puissance de b correspondant à sa position.

Pour un nombre à n chiffres $d_{n-1}\, d_{n-2}\, \ldots\, d_1\, d_0$ en base b :

$$N = \sum_{i=0}^{n-1} d_i \times b^i = d_{n-1} \times b^{n-1} + \cdots + d_1 \times b^1 + d_0 \times b^0$$

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

où $d_i \in \{0, 1, \ldots, b-1\}$ est le chiffre de rang i.

</> Application en base 10

Le nombre décimal 572 se décompose comme :

$$572_{10} = 5 \times 10^2 + 7 \times 10^1 + 2 \times 10^0 = 500 + 70 + 2$$

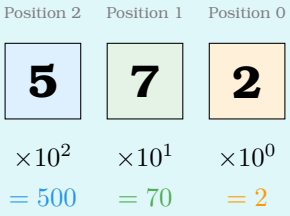


Table 1.1 – Les trois systèmes de numération en informatique.

Système	Base	Symboles	Usage principal
Décimal	10	0–9	Vie courante, affichage utilisateur
Binaire	2	0, 1	Langage natif des circuits
Hexadécimal	16	0–9, A–F	Notation compacte du binaire

1.1.3 Le système binaire (base 2)

Système binaire

Le système **binaire** utilise uniquement deux symboles : **0** et **1**. Chaque position représente une puissance de 2.

Un chiffre binaire est appelé **bit** (*binary digit*). Un groupe de 8 bits forme un **octet** (*byte*).

Table 1.2 – Puissances de 2 à connaître par cœur.

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
1	2	4	8	16	32	64	128	256	512	1 024

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Pourquoi ces valeurs reviennent-elles partout ?

- Un octet (8 bits) peut coder $2^8 = 256$ valeurs : composantes couleur RGB (0–255).
- Un mot de 16 bits code $2^{16} = 65\,536$ valeurs : ports réseau TCP/UDP (0–65 535).
- Un mot de 32 bits code environ 4 milliards de valeurs : adresses IPv4.
- $2^{10} = 1\,024 \approx 1\,000$: origine du préfixe « kilo » en informatique (1 Kio).

1.1.3.1 Conversion binaire vers décimal

Conversion de 1101_2 en décimal

$$1101_2 = \underbrace{1}_{\text{bit 3}} \times 2^3 + \underbrace{1}_{\text{bit 2}} \times 2^2 + \underbrace{0}_{\text{bit 1}} \times 2^1 + \underbrace{1}_{\text{bit 0}} \times 2^0 = 8 + 4 + 0 + 1 = \boxed{13_{10}}$$

1	**1**	**0**	**1**	$= 13$
$\times 8$	$\times 4$	$\times 2$	$\times 1$	
$= 8$	$= 4$	$= 0$	$= 1$	

1.1.3.2 Conversion décimal vers binaire

Conversion par divisions successives

Pour convertir un entier décimal N en binaire :

1. Diviser N par 2, noter le **reste** (0 ou 1).
2. Recommencer avec le **quotient** jusqu'à obtenir 0.
3. Lire les restes **de bas en haut**.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

</> Conversion de 25_{10} en binaire

$$
\begin{array}{ll}
25 \div 2 = 12 & \text{reste } \mathbf{1} \\
12 \div 2 = 6 & \text{reste } \mathbf{0} \\
6 \div 2 = 3 & \text{reste } \mathbf{0} \\
3 \div 2 = 1 & \text{reste } \mathbf{1} \\
1 \div 2 = 0 & \text{reste } \mathbf{1}
\end{array}
\quad \uparrow \text{Lec}
$$

$$25_{10} = 11001_2$$

Vérification : $1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 = 25$ ✓

Cas concret : adresse IP en binaire

L'adresse IPv4 `192.168.1.10` est composée de quatre octets. Chaque nombre décimal se convertit en un octet binaire :

192	168	1	10
11000000	10101000	00000001	00001010

Le routeur compare ces bits avec le **masque de sous-réseau** (opération AND bit à bit, cf. section 1.3) pour déterminer si deux machines sont sur le même réseau.

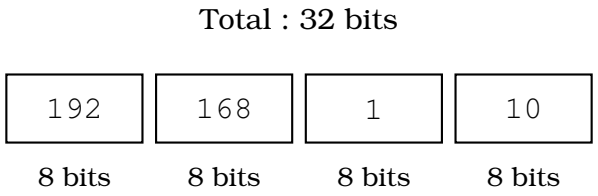


Figure 1.4 – Structure d'une adresse IPv4 : quatre octets (32 bits) écrits en notation décimale pointée. Chaque octet représente une valeur de 0 à 255, soit exactement la plage d'un entier non signé sur 8 bits. Le masque de sous-réseau utilise une opération AND bit à bit pour séparer l'adresse réseau de l'adresse hôte.

Source : Wikimedia Commons, CC BY-SA 4.0 — https://commons.wikimedia.org/wiki/File:IPv4_address_structure_and_writing_systems-en.svg

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.1.4 Le système hexadécimal (base 16)

Système hexadécimal

Le système **hexadécimal** utilise seize symboles :

$$0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9,\ A,\ B,\ C,\ D,\ E,\ F$$

où $A = 10$, $B = 11$, $C = 12$, $D = 13$, $E = 14$, $F = 15$.

Intérêt majeur : un chiffre hexadécimal représente exactement **4 bits** (un *nibble*).

Table 1.3 – Correspondance complète binaire–décimal–hexadécimal.

Bin.	Déc.	Hex.	Bin.	Déc.	Hex.
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	10	A
0011	3	3	1011	11	B
0100	4	4	1100	12	C
0101	5	5	1101	13	D
0110	6	6	1110	14	E
0111	7	7	1111	15	F

</> Pourquoi l'hexadécimal est indispensable

Comparez ces deux écritures du même nombre :

Binaire : `1111 0010 1011 0001` (16 caractères)

Hexadécimal : `F2B1` (4 caractères)

L'hexadécimal est **4 fois plus compact** tout en restant directement convertible en binaire.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.1.4.1 Conversion binaire vers hexadécimal et inversement

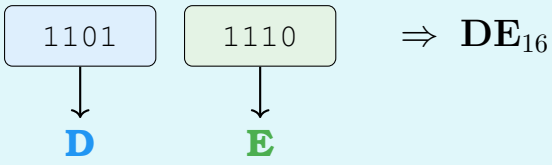
Conversion binaire–hexadécimal

Binaire vers hexadécimal :

1. Regrouper les bits par paquets de 4 **à partir de la droite**.
2. Compléter à gauche par des zéros si nécessaire.
3. Convertir chaque groupe de 4 bits en son chiffre hexadécimal.

Hexadécimal vers binaire : remplacer chaque chiffre hexadécimal par ses 4 bits.

Conversion de 11011110_2 en hexadécimal



Vérification : $D_{16} \times 16 + E_{16} = 13 \times 16 + 14 = 222_{10}$. Et $11011110_2 = 128 + 64 + 16 + 8 + 4 + 2 = 222_{10}$ ✓

Notations en programmation

Base	Notation	Exemple
Binaire	Préfixe `0b`	`0b11001`
Hexadécimal	Préfixe `0x`	`0xFF`, `0x1A3F`
Décimal	Aucun préfixe	`255`, `6719`

En assembleur AVR, les trois formes sont équivalentes :

`LDI R16, 0xFF` = `LDI R16, 0b11111111` = `LDI R16, 255`

1.1.5 Application : les codes couleur RGB

L'une des applications les plus visibles de l'hexadécimal est le codage des couleurs à l'écran.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Codage RGB sur 24 bits

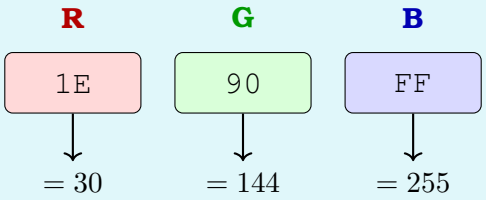
Chaque pixel d'un écran est défini par trois composantes : **Rouge** (R), **Vert** (G) et **Bleu** (B), chacune codée sur un octet (0–255), soit 24 bits par pixel. La notation hexadécimale `#RRGGBB` est universelle en développement web.

Table 1.4 – Exemples de couleurs RGB en hexadécimal.

Couleur	Code Hex	R	G	B
Rouge pur	`#FF0000`	255	0	0
Vert pur	`#00FF00`	0	255	0
Bleu pur	`#0000FF`	0	0	255
Blanc	`#FFFFFF`	255	255	255
Noir	`#000000`	0	0	0
Orange	`#FF9800`	255	152	0

</> Décomposition du code `#1E90FF`

Le code `#1E90FF` (bleu « Dodger Blue ») :



Conversion de $1E_{16}$: $1 \times 16 + 14 = 30_{10}$. Conversion de 90_{16} : $9 \times 16 + 0 = 144_{10}$.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

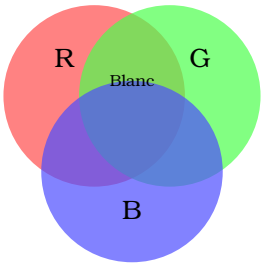


Figure 1.5 – Synthèse additive des couleurs primaires Rouge, Vert et Bleu (RGB). Le mélange des trois à intensité maximale donne le blanc (`#FFFFFF`). Chaque composante est codée sur un octet (0–255) en hexadécimal.
Source : Wikimedia Commons, domaine public — https://commons.wikimedia.org/wiki/File:AdditiveColor.svg

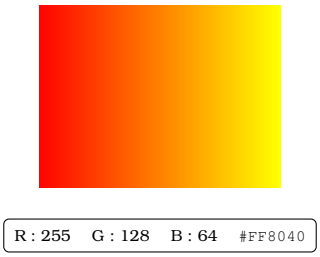
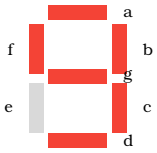


Figure 1.6 – Sélecteur de couleur (*color picker*) tel qu'on le trouve dans les navigateurs web et les logiciels de design (Figma, Photoshop, VS Code). Les valeurs R, G, B sont affichées simultanément en décimal et en hexadécimal, illustrant le lien direct entre ces deux systèmes de numération.

1.1.6 Application : l'afficheur 7 segments

L'afficheur 7 segments est un composant omniprésent : réveils, fours, compteurs, thermomètres numériques. Chaque chiffre affiché correspond à un **motif binaire de 7 bits** indiquant quels segments allumer.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



Chiffre **9** — Segments actifs : a, b, c, d, f, g

Figure 1.7 – Afficheur 7 segments affichant le chiffre 9. Le segment e (grisé) est éteint. Le microcontrôleur envoie un octet dont chaque bit contrôle un segment.

Table 1.5 – Codes 7 segments pour les chiffres 0–9 (convention `gfedcba`, actif haut).

Chiffre	Segments allumés	Binaire	Hex
0	a, b, c, d, e, f	`0111111`	`0x3F`
1	b, c	`0000110`	`0x06`
2	a, b, d, e, g	`1011011`	`0x5B`
3	a, b, c, d, g	`1001111`	`0x4F`
4	b, c, f, g	`1100110`	`0x66`
5	a, c, d, f, g	`1101101`	`0x6D`
6	a, c, d, e, f, g	`1111101`	`0x7D`
7	a, b, c	`0000111`	`0x07`
8	a, b, c, d, e, f, g	`1111111`	`0x7F`
9	a, b, c, d, f, g	`1101111`	`0x6F`

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

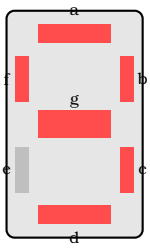


Figure 1.8 – Afficheur 7 segments avec identification des segments a–g. Ce composant est utilisé dans les réveils, fours, compteurs industriels et thermomètres numériques. Chaque segment est piloté par un bit distinct.
Source : Wikimedia Commons, CC BY-SA 3.0 — https://commons.wikimedia.org/wiki/File:7_segment_display_labeled.svg

Lien avec la programmation embarquée

Dans un microcontrôleur AVR, on stocke les motifs dans un tableau :

```
uint8_t digits[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F
```

Pour afficher le chiffre 5, il suffit d'écrire `PORTB = digits[5];` — le port envoie `0x6D` = 01101101_2, allumant les segments a, c, d, f et g.

1.2 Représentation des entiers

1.2.1 Un même octet, plusieurs significations

Un ordinateur ne stocke que des **bits**. La signification d'un motif binaire dépend entièrement du **contexte d'interprétation** choisi par le programmeur.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Table 1.6 – Un même octet, quatre interprétations différentes.

Interprétation	**Valeur**	**Contexte**
Motif binaire : `11001010`		
Entier non signé	202	Compteur, index
Entier signé (C2)	-54	Température, accélération
Composante couleur	R = 202	Pixel RGB
Code instruction	`CBI`	Opcode AVR

⚠ Point crucial pour le programmeur

La machine stocke des **bits**. C'est le **programmeur** qui décide de leur interprétation via le choix du type de données. En assembleur, il n'y a **pas de types** : c'est vous qui devez savoir si un registre contient un entier signé, non signé, un caractère ou une adresse.

1.2.2 Entiers non signés

Entier non signé sur n bits

Un entier **non signé** (*unsigned*) sur n bits représente uniquement des valeurs positives ou nulles :

$$\boxed{0 \leq x \leq 2^n - 1}$$

Table 1.7 – Plages des entiers non signés et usages concrets.

Bits	**Min**	**Max**	**Usage concret**
4	0	15	Chiffre hexadécimal, nibble
8	0	255	Couleur RGB, caractère ASCII, registre AVR
16	0	65 535	Port TCP/UDP, timer 16 bits
32	0	4 294 967 295	Adresse IPv4, compteur 32 bits

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.2.3 Entiers signés et complément à deux

Complément à deux sur n bits

Le **complément à deux** est la méthode standard de représentation des entiers signés dans les processeurs modernes :

- Le **bit de poids fort** (MSB) indique le signe : 0 = positif, 1 = négatif.
- Plage : $\boxed{-2^{n-1} \leq x \leq 2^{n-1} - 1}$

Pour obtenir la représentation de $-x$ (avec $x > 0$) :

1. Écrire x en binaire sur n bits.
2. **Inverser** tous les bits (opération NOT).
3. **Ajouter 1** au résultat.

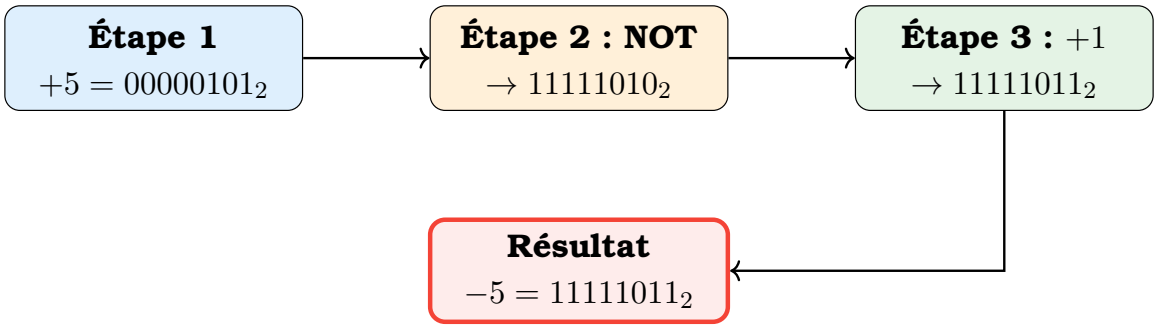


Figure 1.9 – Calcul du complément à deux de -5 sur 8 bits.

Table 1.8 – Comparaison des plages signée et non signée sur 8 bits.

Représentation	Min	Max	Type C
Non signé (8 bits)	0	255	`uint8_t`
Signé C2 (8 bits)	-128	$+127$	`int8_t`

Mêmes bits, valeurs différentes !

Le motif 11111111_2 vaut :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- 255 en non signé (`uint8_t`)
- -1 en signé (`int8_t`, complément à deux)

C'est une source classique de bugs en C et en assembleur !

Pourquoi le complément à deux est-il universel ?

L'**addition matérielle** fonctionne de manière identique pour les nombres signés et non signés :

```
    00000101   (+5)
+ 11111011   (−5)
1 00000000   (= 0)
```

Le 9e bit (retenue) est ignoré sur 8 bits

Le processeur utilise **un seul circuit additionneur** pour les deux cas.

1.2.4 Application : capteur de température DS18B20

Capteur DS18B20 : le complément à deux en action

Le capteur **DS18B20**, très courant en domotique et projets Arduino, renvoie la température sur 12 bits en **complément à deux**. Chaque LSB représente 0,0625 °C.

Registre (16 bits)	Valeur	Température
0000 0001 1001 0001	+401	25,0625 °C
0000 0000 0000 0000	0	0 °C
1111 1111 0101 1110	−162	−10,125 °C

Sans le complément à deux, il serait impossible de représenter les températures négatives avec un seul format binaire.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



Figure 1.10 – Capteur de température DS18B20 de Maxim Integrated (boîtier TO-92, 3 broches : GND, DQ, VDD). Très utilisé en domotique et projets Arduino, il communique via le protocole 1-Wire et transmet la température en complément à deux sur 12 bits.
Source : Wikimedia Commons, CC BY-SA 3.0 — https://commons.wikimedia.org/wiki/File:DS18B20.jpg

1.3 Logique booléenne

1.3.1 Le raisonnement binaire au cœur des machines

Tous les circuits numériques reposent sur des décisions **binaires** : vrai ou faux, 1 ou 0, tension haute ou basse. La **logique booléenne**, formalisée par George Boole (1815–1864), fournit le cadre mathématique de ces opérations.

0
FAUX / OFF / Bas

1
VRAI / ON / Haut

Figure 1.11 – Les deux valeurs de la logique booléenne.

La logique booléenne dans la vie quotidienne

- **Interrupteur :** allumé (1) ou éteint (0).
- **Capteur de porte :** ouverte (1) ou fermée (0).
- **Alarme maison :** déclenchée si capteur mouvement **ET** mode armé.
- **Climatisation :** active si température > seuil **OU** mode manuel forcé.
- **Recherche Google :** `"python" AND "tutorial"` filtre les résultats.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

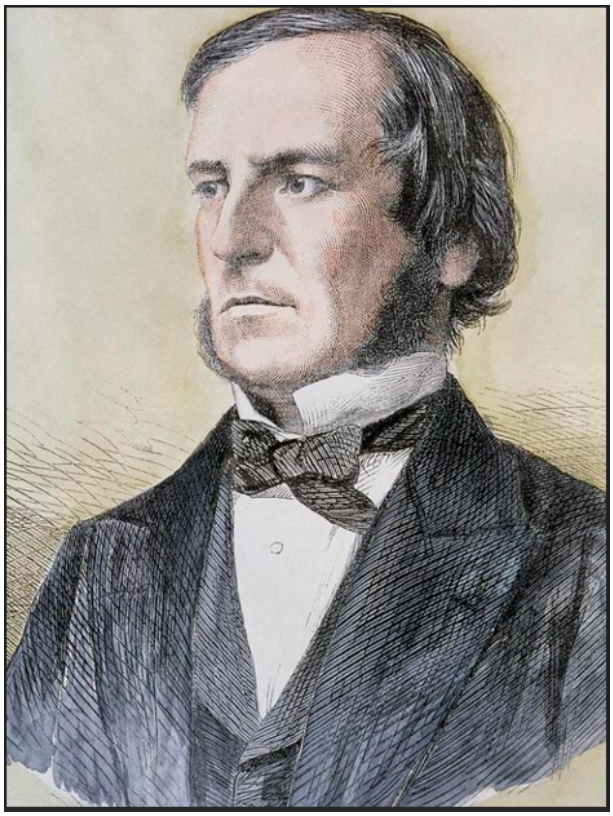


Figure 1.12 – George Boole (1815–1864), mathématicien britannique, fondateur de la logique booléenne. Son algèbre à deux valeurs (0 et 1) constitue le fondement théorique de tous les circuits numériques modernes.
Source : Wikimedia Commons, domaine public — https://commons.wikimedia.org/wiki/File:George_Boole_color.jpg

1.3.2 Les quatre opérations fondamentales

AND	**OR**	**NOT**	**XOR**
Conjonction	**Disjonction**	**Négation**	**Ou exclusif**
$A \wedge B$	$A \vee B$	$\neg A$	$A \oplus B$
« A et B »	« A ou B »	« non A »	« l'un ou l'autre »

Figure 1.13 – Les quatre opérations logiques fondamentales.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

AND ($\wedge$)

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Vrai seulement si A **et** B sont vrais

OR ($\vee$)

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

Vrai si **au moins** un est vrai

NOT ($\neg$)

A	$\neg A$
0	1
1	0

Inverse la valeur

XOR ($\oplus$)

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Vrai si A et B sont **différents**

Figure 1.14 – Tables de vérité des quatre opérations logiques fondamentales.

Mnémotechniques pour retenir les opérations

- **AND :** « Toutes les conditions doivent être vraies » — comme un coffre à deux clés.
- **OR :** « Au moins une suffit » — comme deux sonnettes pour une même porte.
- **NOT :** « L'inverse » — comme un interrupteur inverseur.
- **XOR :** « L'un ou l'autre, pas les deux » — comme un va-et-vient d'escalier.

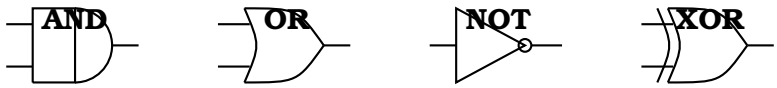


Figure 1.15 – Symboles normalisés (ANSI/IEEE) des portes logiques fondamentales : AND, OR, NOT, XOR. Ces composants élémentaires sont assemblés par milliards dans les processeurs modernes pour réaliser les opérations arithmétiques et logiques.
Source : Wikimedia Commons, domaine public — https://commons.wikimedia.org/wiki/File:Logic-gate-index.png

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.3.3 Applications concrètes de la logique

</> Système d'alarme domestique

$$\text{ALARME} = \text{MODE_ARMÉ} \wedge (\text{CAPTEUR_PORTE} \vee \text{CAPTEUR_MVT})$$

L'alarme sonne si le mode est armé **ET** (porte ouverte **OU** mouvement détecté). En assembleur AVR, cela se traduit par des instructions `AND` et `OR` sur des registres.

</> Interrupteur va-et-vient (escalier)

$$\text{LAMPE} = \text{INTER_1} \oplus \text{INTER_2}$$

La lampe s'allume si les deux interrupteurs sont dans des positions **différentes**. C'est la logique XOR : changez n'importe quel interrupteur et l'état bascule.



Figure 1.16 – Schéma électrique d'un interrupteur va-et-vient (deux points de commande). Le comportement correspond exactement à une porte XOR : la lampe s'allume lorsque les deux interrupteurs sont dans des positions différentes.

Source : Wikimedia Commons, domaine public — https://commons.wikimedia.org/wiki/File:Multiway_switching_-_two_locations_(schematic).svg

</> Masque de sous-réseau

Pour déterminer si deux machines sont sur le même réseau, on applique un AND bit à bit entre l'adresse IP et le masque :

```
      192.168.1.10  = 11000000.10101000.00000001.00001010
AND   255.255.255.0 = 11111111.11111111.11111111.00000000
-------------------------------------------------------
    = 192.168.1.0   = 11000000.10101000.00000001.00000000
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.3.4 Opérations bit à bit en programmation

En programmation et en assembleur, les opérations logiques s'appliquent **bit par bit** sur des mots binaires entiers.

Table 1.9 – Opérateurs bit à bit : symboles et exemples.

Op.	C	AVR	Math	Exemple
AND	`&`	`AND`	$\wedge$	`0b1100 & 0b1010 = 0b1000`
OR	`|`	`OR`	$\vee$	`0b1100 | 0b1010 = 0b1110`
XOR	`^`	`EOR`	$\oplus$	`0b1100 ^ 0b1010 = 0b0110`
NOT	`~`	`COM`	$\neg$	`~0x0F = 0xF0` (8 bits)

</> Masquage : extraire les 4 bits de poids faible

```
  10110110   (valeur = 0xB6 = 182)
AND 00001111   (masque = 0x0F)
-------------
  = 00000110   (résultat = 6)
```

En AVR : `ANDI R16, 0x0F` applique ce masque au registre R16.

</> Forçage de bit : allumer une LED sur le bit 3

Pour mettre le bit 3 à 1 sans modifier les autres bits :

```
  10100001   (PORTB actuel)
OR 00001000   (masque bit 3 = 0x08)
-------------
  = 10101001   (bit 3 allumé, reste inchangé)
```

En AVR : `ORI R16, 0x08` ou `SBI PORTB, 3`.

1.4 Machine à états et cycle d'exécution

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.4.1 Concept de machine à états finis

Machine à états finis (FSM)

Une **machine à états finis** (*Finite State Machine*, FSM) est un modèle décrivant un système qui :

- se trouve à tout instant dans l'un d'un nombre **fini d'états** ;
- change d'état selon des **transitions** déterminées par des entrées.

Formellement : FSM $= (S, \Sigma, \delta, s_0, F)$ où S est l'ensemble fini d'états, Σ l'alphabet d'entrée, δ la fonction de transition, s_0 l'état initial, et F les états finaux.

Machines à états dans le quotidien

- **Feu tricolore :** Rouge → Vert → Orange → Rouge.
- **Machine à laver :** Remplissage → Lavage → Rinçage → Essorage → Fin.
- **Distributeur :** Attente → Sélection → Paiement → Distribution.
- **Connexion WiFi :** Déconnecté → Recherche → Authentification → Connecté.
- **Protocole USB :** Idle → Setup → Data → Status → Idle.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.4.2 Exemple détaillé : le feu tricolore

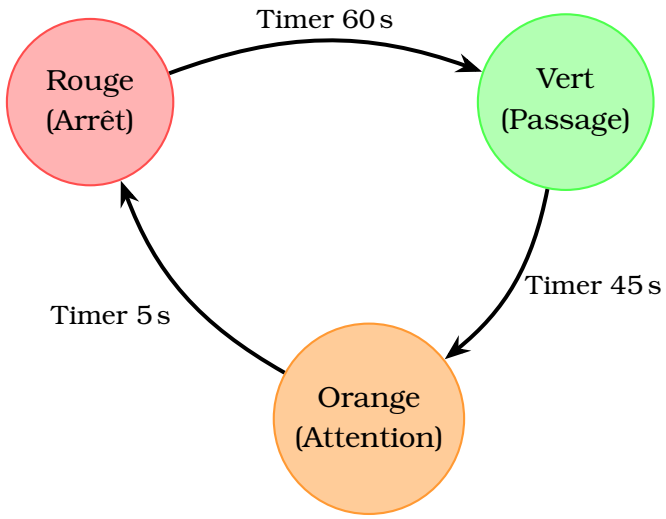


Figure 1.17 – Diagramme d'états d'un feu tricolore. Le système cycle entre trois états, chaque transition étant déclenchée par l'expiration d'un timer.

Table 1.10 – Table de transition du feu tricolore avec codage binaire.

État courant	**Condition**	**État suivant**	**Sortie (R,O,V)**	**Binaire**
Rouge	Timer $\geq$ 60 s	Vert	(1,0,0)	`0b100`
Vert	Timer $\geq$ 45 s	Orange	(0,0,1)	`0b001`
Orange	Timer $\geq$ 5 s	Rouge	(0,1,0)	`0b010`

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

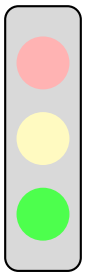


Figure 1.18 – Feu de signalisation tricolore en phase verte. Le séquencement Rouge → Vert → Orange est un exemple classique de machine à états finis, piloté par un microcontrôleur avec timer interne.
Source : Wikimedia Commons, CC BY-SA — https://commons.wikimedia.org/wiki/File:GreenTrafficLight.jpg

1.4.3 Le cycle d'exécution du processeur

Un processeur est lui-même une machine à états sophistiquée. Il traite chaque instruction selon un cycle régulier et déterministe.

Cycle Fetch-Decode-Execute

Le **cycle d'exécution** est la séquence que le processeur répète indéfiniment :

1. **Fetch :** lire l'instruction à l'adresse pointée par le PC.
2. **Decode :** interpréter l'opcode pour identifier l'opération.
3. **Execute :** réaliser l'opération (calcul ALU, transfert, branchement).
4. **Write-back :** enregistrer le résultat dans un registre ou en mémoire.
5. **Update PC :** avancer le compteur de programme.

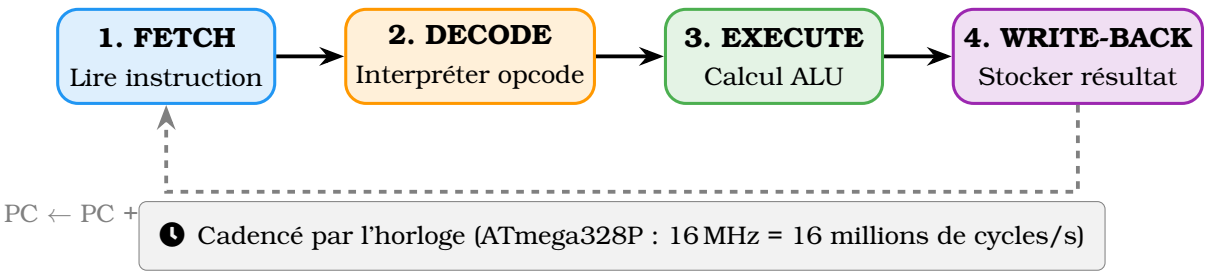


Figure 1.19 – Le cycle Fetch-Decode-Execute-Writeback. Sur un ATmega328P à 16 MHz, ce cycle se répète 16 millions de fois par seconde.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

</> Cycle concret : allumer une LED avec un ATmega328P

Considérons l'instruction `SBI PORTB, 5` (allumer la LED broche 13 d'Arduino) :

Phase	Action	Détail
Fetch	Lit `0x9A2D` en Flash	Opcode de `SBI PORTB, 5`
Decode	Reconnaît l'instruction SBI	Port = PORTB, bit = 5
Execute	Met le bit 5 de PORTB à 1	Tension PB5 passe à 5 V
Write-back	Résultat immédiat	LED allumée
Update PC	PC ← PC + 1	Instruction suivante

Ce processus prend **2 cycles d'horloge** = 125 ns à 16 MHz.

Registres clés du processeur AVR

- **PC** (*Program Counter*) : adresse de la prochaine instruction.
- **IR** (*Instruction Register*) : instruction en cours de décodage.
- **SREG** (*Status Register*) : drapeaux Z, C, N, V, S, H.
- **SP** (*Stack Pointer*) : sommet de la pile en SRAM.
- **R0–R31** : 32 registres généraux de 8 bits.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

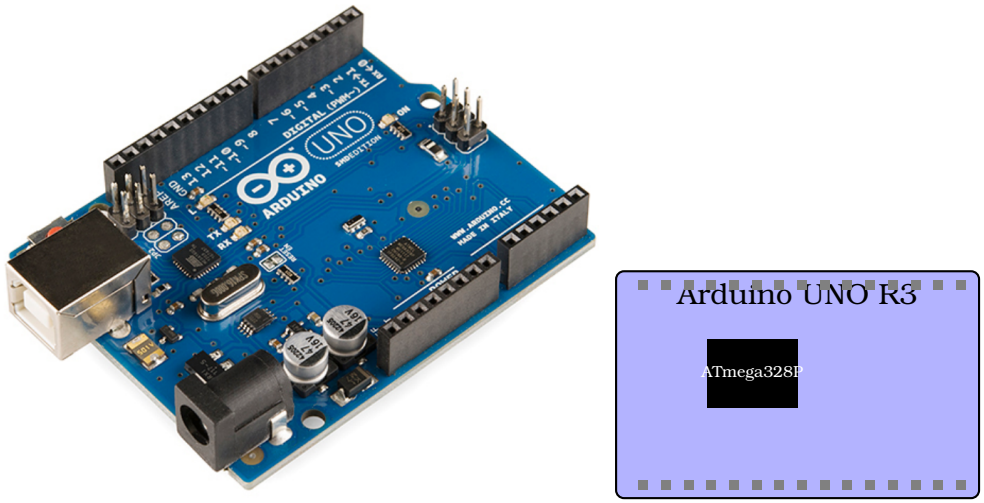


Figure 1.20 – Carte Arduino UNO R3 équipée du microcontrôleur ATmega328P (puce noire au centre). Ce processeur AVR 8 bits cadencé à 16 MHz exécute le cycle Fetch-Decode-Execute 16 millions de fois par seconde. C'est la plateforme de référence pour les travaux pratiques de ce cours.

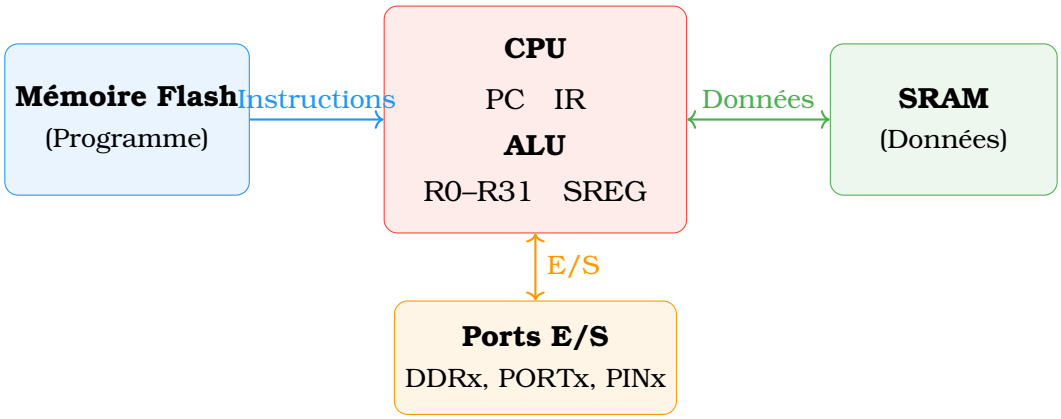


Figure 1.21 – Architecture simplifiée d'un microcontrôleur AVR : la mémoire Flash fournit les instructions, le CPU les exécute via l'ALU et les registres, la SRAM stocke les données, et les ports E/S communiquent avec l'extérieur.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

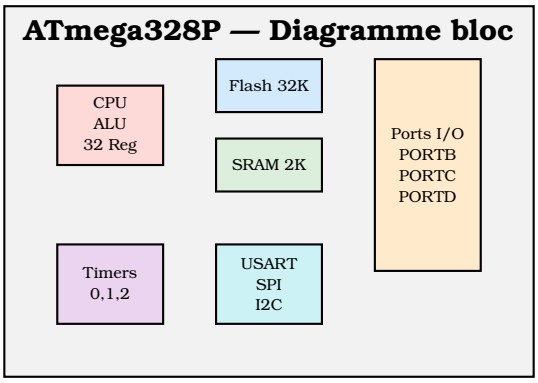


Figure 1.22 – Diagramme bloc de l'ATmega328P montrant l'architecture Harvard modifiée : mémoire Flash (programme) et SRAM (données) séparées, ALU 8 bits, 32 registres généraux, compteur de programme, registre d'état (SREG), ports d'E/S, timers et interfaces de communication (USART, SPI, I2C). Ce schéma sera détaillé au chapitre 1.
Source : Microchip Technology Datasheet — https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf

Synthèse du chapitre 0

Ce chapitre a posé les quatre piliers nécessaires à l'étude de l'architecture et de l'assembleur :

1. **Systèmes de numération** : le binaire est le langage natif des machines ; l'hexadécimal en est la notation compacte.
2. **Représentation des entiers** : un même motif binaire peut être interprété différemment. Le complément à deux code les négatifs avec un seul additionneur.
3. **Logique booléenne** : AND, OR, NOT et XOR sont les briques de tout circuit. Le masquage et le forçage de bits sont essentiels en bas niveau.
4. **Machine à états et cycle d'exécution** : un processeur répète indéfiniment le cycle Fetch-Decode-Execute, cadencé par une horloge.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

*Ces fondements seront mobilisés dans **chaque chapitre suivant** : les registres AVR stockent des octets (ch. 1), les instructions manipulent des bits (ch. 2–3), la pile utilise l'adressage binaire (ch. 4), et les E/S reposent sur le masquage logique (ch. 5).*

Formules et repères essentiels

Concept	**Formule ou repère**
Valeur positionnelle	$N = \sum_{i=0}^{n-1} d_i \times b^i$
Plage non signée (n bits)	$[0\,;\ 2^n - 1]$
Plage signée C2 (n bits)	$[-2^{n-1}\,;\ 2^{n-1} - 1]$
Complément à deux de x	$\text{NOT}(x) + 1$
AND (masquage)	Isole les bits à 1 dans le masque
OR (forçage)	Met à 1 les bits à 1 dans le masque
XOR (basculement)	Inverse les bits à 1 dans le masque
1 chiffre hex	$= 4$ bits (1 nibble)
1 octet	$= 8$ bits $= 2$ chiffres hex

1.5 Travaux dirigés du chapitre 0

Travaux dirigés : Du bit au système

5 exercices progressifs ancrés dans des situations réelles — réseau, développement web, systèmes embarqués, cybersécurité

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

TD 1 — Le monde des adresses : IP, MAC et couleurs RGB

Contexte : Vous êtes technicien réseau junior. Votre responsable vous demande d'analyser les configurations d'équipements et de comprendre le codage des données réseau et graphiques.

Situation 1 — Configuration réseau d'un serveur

Le serveur de l'entreprise a l'adresse IP `192.168.10.254` avec le masque `255.255.255.0`.

a) Convertissez chaque octet de l'adresse IP en binaire sur 8 bits.

b) Convertissez le masque de sous-réseau en binaire. Combien de bits sont à 1 ? Quelle est la notation CIDR correspondante (`/xx`) ?

c) Un collègue vous donne l'adresse `192.168.10.100`. En appliquant un AND bit à bit entre cette adresse et le masque, déterminez si elle appartient au même réseau que le serveur.

d) L'adresse de broadcast du réseau est obtenue en mettant tous les bits « hôte » à 1. Quelle est-elle ?

Situation 2 — Analyse d'une adresse MAC

La carte réseau d'un poste affiche l'adresse MAC `00:1A:2B:3C:4D:5E`.

a) Combien d'octets compose cette adresse ? Combien de bits au total ?

b) Convertissez le premier octet (`00`) et le dernier (`5E`) en binaire.

c) Le bit de poids faible du premier octet indique si l'adresse est unicast (0) ou multicast (1). Cette adresse est-elle unicast ou multicast ?

d) Le deuxième bit de poids faible indique si l'adresse est universelle (0) ou locale (1). Qu'en est-il pour cette adresse ?

Situation 3 — Design d'interface web

Vous développez une interface web et le designer vous fournit la palette suivante :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- Couleur principale : `#1E90FF` (Dodger Blue)
- Couleur d'accentuation : `#FF6B35` (Orange vif)
- Fond : `#F5F5F5` (Gris très clair)

a) Pour chaque couleur, décomposez le code hexadécimal en trois composantes R, G, B (en décimal).

b) La couleur `#1E90FF` a-t-elle plus de rouge, de vert ou de bleu ?

c) Calculez la luminosité approximative de chaque couleur avec la formule simplifiée : $L = 0.3 \times R + 0.6 \times G + 0.1 \times B$. Quelle couleur est la plus lumineuse ?

d) Vous devez créer une version « assombrie » de `#1E90FF` en divisant chaque composante par 2. Quel est le nouveau code hexadécimal ?

TD 2 — Capteurs IoT et données signées

Contexte : Vous développez une application IoT qui collecte des données de capteurs. Comprendre la représentation des données signées et non signées est crucial pour interpréter correctement les mesures.

Situation 1 — Capteur de température DS18B20

Le capteur DS18B20 renvoie la température sur 12 bits en complément à deux. Chaque LSB = 0,0625 °C. Le registre 16 bits lu contient la valeur avec les 4 bits de poids fort à 0 (ou 1 si négatif).

a) Le capteur renvoie `0x0191`. Convertissez en binaire, puis en décimal. Quelle est la température mesurée ?

b) Le capteur renvoie `0xFF5E`. Le MSB étant à 1, c'est une valeur négative. Appliquez la procédure inverse du C2 (NOT + 1) sur les 12 bits significatifs pour trouver la valeur absolue. Quelle est la température ?

c) Quelle est la plage de températures mesurables avec ce format 12 bits signé ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

d) Votre collègue lit la valeur brute `0x0000` et dit « le capteur est cassé ». A-t-il raison ?

Situation 2 — Accéléromètre 3 axes

Un accéléromètre MEMS (type ADXL345) renvoie l'accélération sur chaque axe en **complément à deux sur 16 bits**. La sensibilité est de 256 LSB/g (mode ±2g).

a) L'axe Z (vertical) renvoie `0x0100` = 256_{10}. Quelle accélération en g cela représente-t-il ? Est-ce cohérent pour un capteur posé à plat ?

b) L'axe X renvoie `0xFFE0`. Convertissez en signé (C2). Quelle accélération en g ?

c) Calculez la représentation C2 sur 16 bits de l'accélération $-0.5g$.

d) Quelle est la plage d'accélérations mesurables avec ce format ?

Situation 3 — Compteur d'énergie

Un compteur électrique intelligent utilise un registre 32 bits non signé pour stocker la consommation en Wh.

a) Quelle est la valeur maximale stockable ? Convertissez en kWh puis en MWh.

b) Une maison consomme en moyenne 10 kWh/jour. Au bout de combien de jours le compteur débordera-t-il ?

c) Si le compteur passait à 64 bits, combien de temps avant débordement avec la même consommation ?

d) Le compteur actuel affiche `0x000F4240`. Convertissez en décimal, puis en kWh.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

TD 3 — Logique booléenne pour la sécurité et le contrôle d'accès

Contexte : Vous concevez la logique de contrôle d'un système de sécurité pour un datacenter. La compréhension des opérations booléennes est essentielle pour définir les règles d'accès et d'alerte.

Situation 1 — Système d'alarme datacenter

Le système de sécurité possède les entrées suivantes :

- B : Badge valide présenté (1 = oui)
- C : Code PIN correct (1 = oui)
- H : Horaire autorisé (1 = oui, entre 7h et 20h)
- A : Mode admin activé (1 = oui, bypass horaire)

La porte s'ouvre si : Badge valide **ET** Code correct **ET** (Horaire autorisé **OU** Mode admin)

a) Écrivez l'expression booléenne : PORTE $= ?$

b) Construisez la table de vérité complète (16 lignes).

c) Un employé a un badge valide, tape le bon code à 22h sans mode admin. La porte s'ouvre-t-elle ?

d) L'administrateur système a le mode admin activé mais oublie son code. Peut-il entrer ?

Situation 2 — Règles de pare-feu simplifiées

Vous configurez un pare-feu avec les règles suivantes pour le trafic entrant :

- Autoriser si : (port 80 **OU** port 443) **ET** (source = réseau interne **OU** IP whitelist)
- Bloquer sinon

Variables : P_{80} (port 80), P_{443} (port 443), I (réseau interne), W (whitelist)

a) Écrivez l'expression : AUTORISER $= ?$

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

b) Un paquet arrive sur le port 443 depuis une IP externe non whitelistée. Est-il autorisé ?

c) Simplifiez l'expression si on sait que le réseau interne est toujours whitelisté ($I \Rightarrow W$).

d) Appliquez la loi de De Morgan pour exprimer la condition de blocage.

Situation 3 — Détection d'intrusion

Un système de détection d'intrusion génère une alerte si :

- Tentatives de connexion échouées ≥ 5 (F) **ET** même IP (S)
- **OU** Scan de ports détecté (P)
- **OU** Payload suspect (M) **ET** connexion depuis pays à risque (R)

a) Écrivez l'expression : ALERTE $= ?$

b) Construisez la table de vérité pour les 32 combinaisons possibles. (Indication : utilisez un tableau synthétique en regroupant les cas où ALERTE = 1)

c) Un attaquant utilise 10 IP différentes pour ses tentatives. L'alerte est-elle déclenchée par la première condition ?

d) Proposez une amélioration de la formule pour détecter aussi les attaques distribuées.

TD 4 — Manipulation de registres et configuration matérielle

Contexte : Vous programmez un microcontrôleur Arduino (ATmega328P) pour un projet de domotique. La manipulation des registres via des opérations bit à bit est fondamentale.

Situation 1 — Contrôle de LEDs sur PORTB

Le port B de l'ATmega328P contrôle 8 broches (PB0 à PB7). Chaque bit

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

du registre `PORTB` contrôle une LED (1 = allumée).
État actuel : `PORTB = 0b00110100` = `0x34`

a) Quelles LEDs sont actuellement allumées ? (numérotez de 0 à 7)

b) Écrivez l'opération pour **allumer** la LED 0 **sans modifier** les autres. Donnez le résultat.

c) Écrivez l'opération pour **éteindre** la LED 4 **sans modifier** les autres. Donnez le résultat.

d) Écrivez l'opération pour **inverser** l'état de la LED 2 (toggle). Donnez le résultat.

e) Vous voulez allumer les LEDs 0, 1, 6, 7 et éteindre toutes les autres. Quelle valeur écrire directement dans `PORTB` ?

Situation 2 — Lecture de capteurs sur PINC

Le port C est configuré en entrée et lit l'état de 8 capteurs. Le registre `PINC` contient `0b10110010` = `0xB2`.
Structure des capteurs :

- Bits 0-3 : 4 capteurs de contact (1 = contact)
- Bits 4-5 : Capteur de luminosité (valeur 0-3)
- Bits 6-7 : Capteur de température (valeur 0-3)

a) Quels capteurs de contact sont actifs ?

b) Extrayez la valeur du capteur de luminosité (bits 4-5). Utilisez un masque ET puis un décalage logique.

c) Extrayez la valeur du capteur de température (bits 6-7).

d) Écrivez une expression qui teste si **au moins un** capteur de contact est actif.

e) Le capteur de luminosité renvoie 0 = nuit, 1 = faible, 2 = moyen, 3 = jour. Quelle est la luminosité actuelle ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Situation 3 — Registre de configuration UART

Vous configurez le registre de contrôle UART `UCSRB` de l'ATmega328P. Voici la signification des bits :

Bit	Nom	Fonction
7	RXCIE	Interruption réception activée
6	TXCIE	Interruption transmission activée
5	UDRIE	Interruption buffer vide activée
4	RXEN	Récepteur activé
3	TXEN	Transmetteur activé
2	UCSZ2	Format données (bit 2)
1	RXB8	9e bit reçu (lecture seule)
0	TXB8	9e bit à transmettre

État actuel : `UCSRB = 0x00`

a) Écrivez le code pour activer le récepteur ET le transmetteur (bits 3 et 4) sans modifier les autres bits.

b) Ajoutez l'activation de l'interruption de réception (bit 7). Donnez la valeur finale de `UCSRB`.

c) Sans tout réécrire, comment désactiver uniquement le transmetteur (bit 3) ?

d) Écrivez une expression qui teste si le transmetteur est actif.

TD 5 — Conception de machines à états pour systèmes embarqués

Contexte : Vous concevez des systèmes embarqués et devez modéliser leur comportement par des machines à états finis. Cette approche est fondamentale en développement firmware et automates industriels.

Situation 1 — Connexion WiFi d'un objet connecté

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Modélisez la connexion WiFi d'un capteur IoT avec les états suivants :

- **IDLE** : En veille, WiFi désactivé
- **SCANNING** : Recherche des réseaux disponibles
- **CONNECTING** : Tentative de connexion au réseau configuré
- **AUTHENTICATING** : Échange des clés de sécurité (WPA2)
- **CONNECTED** : Connexion établie, IP obtenue
- **ERROR** : Échec de connexion

Transitions :

- IDLE $\xrightarrow{\text{bouton}}$ SCANNING
- SCANNING $\xrightarrow{\text{réseau trouvé}}$ CONNECTING
- SCANNING $\xrightarrow{\text{timeout 10s}}$ ERROR
- CONNECTING $\xrightarrow{\text{réponse AP}}$ AUTHENTICATING
- AUTHENTICATING $\xrightarrow{\text{succès}}$ CONNECTED
- AUTHENTICATING $\xrightarrow{\text{échec}}$ ERROR
- CONNECTED $\xrightarrow{\text{déconnexion}}$ IDLE
- ERROR $\xrightarrow{\text{retry}}$ SCANNING
- ERROR $\xrightarrow{\text{abandon}}$ IDLE

a) Dessinez le diagramme d'états complet avec cercles et flèches annotées.

b) Combien de bits faut-il pour encoder les 6 états en binaire ?

c) Proposez un codage binaire pour chaque état.

d) Ajoutez un état **RECONNECTING** pour les déconnexions temporaires (différent de ERROR).

Situation 2 — Protocole de communication série

Vous implémentez un protocole de communication simple avec un capteur. Les trames ont le format :

```
[START] [CMD] [DATA] [CHECKSUM] [END]
0xAA  [1 octet][N octets][1 octet] 0x55
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Modélisez la réception d'une trame avec une FSM.

a) Identifiez les états nécessaires pour parser cette trame.

b) Dessinez le diagramme d'états avec les transitions (octet reçu, valeur attendue).

c) Que se passe-t-il si on reçoit `0x55` alors qu'on attend des données ?

d) Ajoutez un mécanisme de timeout : retour à l'état initial si aucun octet pendant 100 ms.

Situation 3 — Distributeur de café connecté

Concevez la machine à états d'un distributeur de café intelligent avec :
Entrées :

- Boutons : Café court, Café long, Cappuccino, Annuler
- Capteurs : Niveau eau, Niveau café, Gobelet présent, Paiement validé (NFC)
- Timer : Durées de préparation

Sorties :

- Actionneurs : Pompe eau, Moulin, Vapeur
- Affichage : Messages utilisateur
- LEDs : État machine

Contraintes :

- Ne pas démarrer si niveau eau ou café insuffisant
- Attendre le gobelet avant de servir
- Permettre l'annulation à tout moment (sauf pendant le service)

a) Listez tous les états nécessaires (minimum 8).

b) Dessinez le diagramme d'états complet.

c) Créez la table de transition (état actuel, entrée, état suivant, sortie).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

d) Proposez un encodage des états sur 4 bits avec une logique (ex. : bit 3 = erreur).

e) Comment gérer les erreurs (réservoir vide pendant la préparation) ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.6 Quiz du chapitre 0 : testez vos connaissances

Vrai/Faux QCM Pratique

Répondez sans regarder le cours. Ces questions simulent des situations réelles rencontrées en entreprise.

1 QCM

Un développeur web définit une couleur avec le code `#FF8000`. Quelles sont les composantes RGB en décimal ?

A. R=255, G=128, B=0 (orange)

B. R=255, G=80, B=00 (rouge-orange)

C. R=15, G=8, B=0 (brun foncé)

D. R=128, G=0, B=255 (violet)

2 VRAI/FAUX

Une adresse IPv4 comme `192.168.1.1` occupe exactement 32 bits en mémoire (4 octets).

☐ **Vrai** ☐ **Faux**

Indice : chaque nombre (0–255) est un octet.

3 QCM

Un capteur de température renvoie la valeur `0xFF` (8 bits). Si on l'interprète comme signé (C2), quelle température représente-t-il si 1 LSB = 1 °C ?

A. +255 °C

B. +127 °C

C. -1 °C

D. 0 °C

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

4 VRAI/FAUX

Pour éteindre un bit spécifique dans un registre sans affecter les autres, on utilise l'opération OR avec un masque.

☐ **Vrai** ☐ **Faux**

Indice : OR met des bits à 1. Pour mettre à 0, quelle opération ?

5 QCM

Dans un système d'alarme, la sirène sonne si : `mode_armé AND (porte_ouverte OR mouvement)`. Si le mode n'est pas armé, la sirène peut-elle sonner ?

A. Oui, si la porte est ouverte

B. Oui, si un mouvement est détecté

C. Non, jamais

D. Cela dépend du type de capteur

6 VRAI/FAUX

L'opération XOR permet de basculer (toggle) un bit : si le bit est à 0 il passe à 1, et vice versa.

☐ **Vrai** ☐ **Faux**

Indice : calculez $0 \oplus 1$ *et* $1 \oplus 1$.

7 PRATIQUE

Vous devez configurer un registre de LEDs. État actuel : `0b01010101`. Écrivez les opérations pour :

a) Allumer la LED 1 (bit 1) : `PORTB = PORTB` _____

b) Éteindre la LED 6 (bit 6) : `PORTB = PORTB` _____

c) Inverser la LED 0 (bit 0) : `PORTB = PORTB` _____

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

8 QCM

Un processeur ARM cadencé à 1 GHz exécute 1 instruction par cycle. Combien d'instructions exécute-t-il en 1 microseconde ?

A. 1

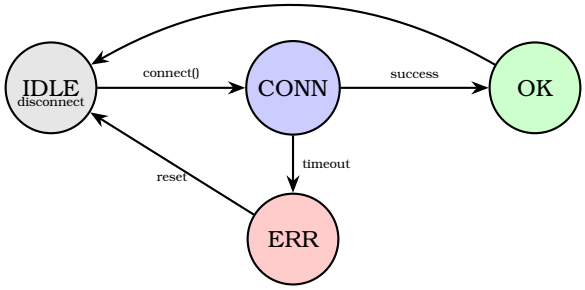
B. 1 000

C. 1 000 000

D. 1 000 000 000

9 PRATIQUE

Identifiez l'état suivant dans cette machine à états de connexion réseau :



État actuel : **CONN**. Événement : **timeout**. État suivant : _____

État actuel : **OK**. Événement : **disconnect**. État suivant : _____

État actuel : **ERR**. Événement : **connect()**. État suivant : _____

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

10 PRATIQUE

Défi réseau : Vous analysez un paquet avec Wireshark. L'adresse IP source est affichée comme `C0.A8.0A.FE` (notation hexadécimale). Convertissez-la en notation décimale pointée standard.

`C0` = _____ (décimal)

`A8` = _____ (décimal)

`0A` = _____ (décimal)

`FE` = _____ (décimal)

Adresse IP : _____

Bonus : Cette adresse appartient-elle à un réseau privé (RFC 1918) ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Récapitulatif : Ce que vous devez maîtriser

Numération — Applications

- Adresses IP : 4 octets, notation décimale
- Adresses MAC : 6 octets, notation hex
- Couleurs RGB : 3 octets, `#RRGGBB`
- Masques : opérations AND bit à bit
- 1 chiffre hex = 4 bits = 1 nibble

Représentation — Capteurs

- Non signé : compteurs, index, tailles
- Signé (C2) : températures, accélérations
- Même bits $\neq$ même valeur !
- Overflow : comportement circulaire

Logique — Sécurité & config

- AND : conditions multiples obligatoires
- OR : conditions alternatives
- XOR : toggle, chiffrement simple
- Masquage : extraction de champs
- Forçage : configuration de bits

FSM — Protocoles & systèmes

- États finis + transitions déterministes
- Connexions réseau : IDLE $\rightarrow$ CONN $\rightarrow$ OK
- Protocoles : parsing de trames
- Cycle CPU : Fetch-Decode-Execute

Compétences clés pour le métier IT :

Lire un dump hexadécimal • Configurer des registres matériels • Analyser des trames réseau Interpréter des données de capteurs • Concevoir des automates • Déboguer au niveau bit

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Fin du Chapitre 0 — Fondements préliminaires

Vous maîtrisez maintenant les bases indispensables :

Numération • Représentation • Logique • Machine à états

Chapitre suivant : Architecture des microcontrôleurs AVR

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

CHAPITRE II

Introduction à l'architecture des microcontrôleurs AVR

Chapitre 2
Introduction aux microcontrôleurs AVR

Comprendre la machine avant de la programmer

4 heures | Fondements théoriques | Systèmes embarqués | Architecture matérielle

Mise en situation : une journée ordinaire

Imaginez votre journée typique :

1. **6h30** — Votre **réveil** sonne. *Comment sait-il qu'il est 6h30 ?* Un petit composant électronique compte les secondes depuis minuit et compare avec l'heure programmée. Quand les deux correspondent, il active le buzzer.
2. **6h35** — Votre **cafetière programmable** commence à chauffer l'eau. *Comment sait-elle qu'il faut démarrer maintenant ?* Le même type de composant surveille l'heure, vérifie qu'il y a de l'eau dans le réservoir, chauffe à la bonne température et s'arrête automatiquement.

3. **7h00** — Vous montez dans votre **voiture**. Le moteur ne démarre pas immédiatement : le véhicule vérifie d'abord la pression d'huile, la température du moteur, l'état de la batterie... *Qui fait toutes ces vérifications ?*
4. **7h15** — En roulant, votre voiture **freine brusquement** sur une route mouillée. Les roues ne se bloquent pas grâce à l'**ABS**. *Qui surveille la vitesse de chaque roue 100 fois par seconde et ajuste la pression de freinage ?*
5. **8h00** — Vous badgez à l'entrée du bâtiment. La **porte s'ouvre** automatiquement. *Qui a lu le badge, vérifié l'autorisation et commandé l'ouverture en moins d'une seconde ?*

La réponse est toujours la même :
un microcontrôleur.

Un composant minuscule, souvent invisible, qui prend des milliers de décisions par seconde dans chacun de ces objets.

Pourquoi ce chapitre est fondamental

Ce cours porte sur la **programmation assembleur**. Mais avant d'écrire la moindre ligne de code, une question s'impose : *pour qui écrit-on ce code ?*

Quand vous écrivez un programme en Python ou en Java, vous ne vous souciez pas vraiment de la machine : le langage s'occupe des détails.
En assembleur, c'est l'inverse. **Chaque instruction que vous écrirez correspondra à une action physique précise** du microcontrôleur :

- déplacer un nombre d'un tiroir à un autre (registre) ;
- lire une valeur venant d'un capteur (port d'entrée) ;
- allumer une LED en envoyant du courant (port de sortie) ;
- compter le temps qui passe (timer) ;
- additionner deux nombres (ALU).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Pour comprendre **pourquoi** une instruction fait ce qu'elle fait, il faut connaître **la machine**. C'est exactement l'objet de ce chapitre.

Structure interne | **Mémoires (3 types)** | **Registres (32 tiroirs)** | **Interfaces extérieures** | **Exécution du code** | **Compilation et outils**

◎ Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

1. **Expliquer** avec vos propres mots ce qu'est un microcontrôleur et à quoi il sert.
2. **Donner** au moins 5 exemples d'objets du quotidien qui en contiennent.
3. **Distinguer** un microcontrôleur d'un microprocesseur en expliquant pourquoi on utilise l'un plutôt que l'autre.
4. **Dessiner** de mémoire un schéma simplifié de l'architecture AVR avec ses composants principaux.
5. **Expliquer** le rôle de chaque composant (CPU, ALU, registres, Flash, SRAM, EEPROM).
6. **Décrire** comment le processeur exécute une instruction, étape par étape.
7. **Comprendre** le principe de la compilation croisée et la chaîne d'outils AVR (avr-gcc, avr-objcopy, avrdude).
8. **Installer** un environnement de développement et faire clignoter une LED.

🔑 Prérequis

Ce chapitre suppose acquis les fondements du chapitre 0 :

- ✔ Systèmes de numération (binaire, hexadécimal) — *vous savez convertir* `0xFF` *en 255*
- ✔ Représentation des entiers (complément à deux) — *vous savez que* `11111111` *peut valoir 255 ou* -1

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- ✔ Opérations logiques (AND, OR, NOT, XOR) — *vous savez faire un masquage de bits*
- ✔ Notion de machine à états — *vous connaissez le feu tricolore comme FSM*

Si ce n'est pas le cas, relisez le chapitre 0 avant de continuer. Chaque notion ci-dessus sera utilisée concrètement dans ce chapitre.

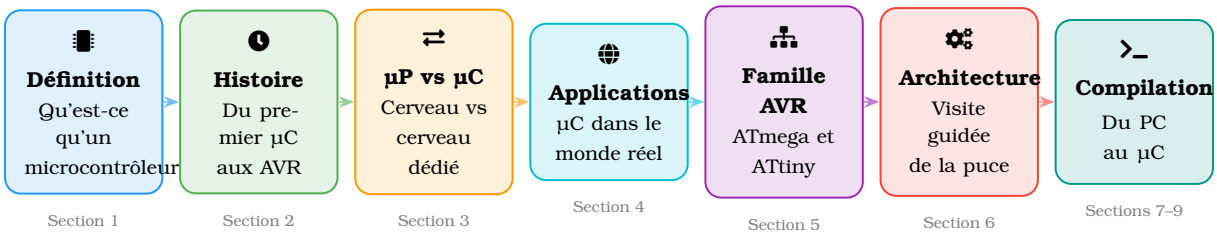


Figure 2.1 – Organisation du chapitre : de la découverte du microcontrôleur à la compilation croisée et au premier programme.

2.1 Qu'est-ce qu'un microcontrôleur ?

2.1.1 Commençons par une analogie

Le microcontrôleur, c'est comme un employé polyvalent

Imaginez une petite entreprise qui n'a qu'un seul employé. Cet employé doit tout faire :

- **lire** le courrier (= lire les capteurs) ;
- **réfléchir** et prendre des décisions (= calculer avec le processeur) ;
- **se souvenir** des consignes (= stocker le programme en mémoire) ;
- **noter** des résultats temporaires sur un bloc-notes (= utiliser la RAM) ;
- **agir** : allumer la lumière, ouvrir la porte, envoyer un message (= commander les sorties).

Un microcontrôleur, c'est exactement cela : **un employé miniature intégré dans une seule puce**, qui fait tout tout seul.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Par contraste, un microprocesseur, c'est un employé très intelligent mais qui a besoin de collègues (mémoire externe, circuits d'entrée/sortie) pour fonctionner.

2.1.2 Définition formelle

Microcontrôleur (*Microcontroller Unit* — MCU)

Un **microcontrôleur** est un **circuit intégré programmable** qui regroupe sur une **seule puce de silicium** un processeur (CPU), de la mémoire (RAM, ROM, EEPROM) et divers périphériques de gestion d'entrées/sorties.
Il constitue l'élément central des **systèmes embarqués**, conçu pour exécuter des tâches spécifiques avec une efficacité optimale en termes de consommation d'énergie, de réactivité et de compacité.
Contrairement aux microprocesseurs, qui requièrent des composants externes pour fonctionner (mémoire, interfaces, etc.), les microcontrôleurs intègrent ces éléments sur une même puce, réduisant ainsi les coûts, la complexité et la consommation électrique.

Microcontrôleur = un ordinateur miniature sur une seule puce

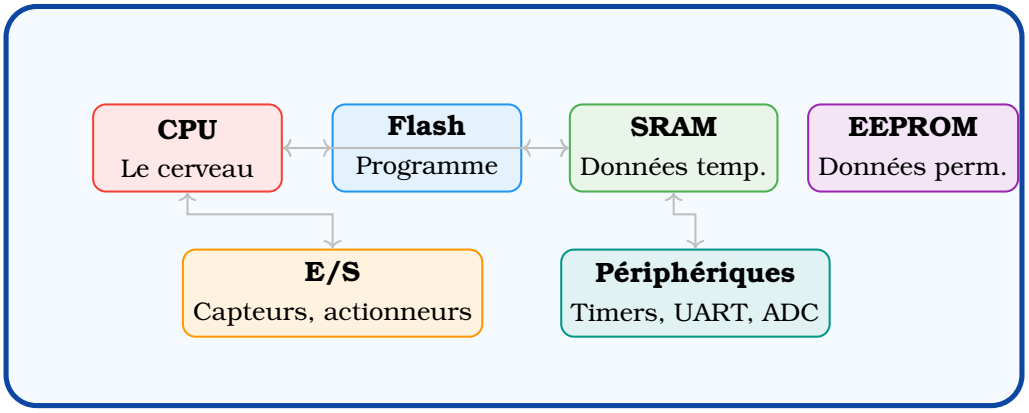


Figure 2.2 – Synoptique d'un microcontrôleur : tous les composants sur une seule puce.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Taille réelle d'un microcontrôleur

Un microcontrôleur comme l'ATmega328P mesure environ **3 cm de long** dans son boîtier DIP-28. Mais la puce de silicium à l'intérieur ne fait que **quelques millimètres carrés** — à peine la taille d'une tête d'épingle !
Et pourtant, cette minuscule surface contient :

- un processeur complet ;
- 32 768 octets de mémoire programme ;
- 2 048 octets de RAM ;
- 23 broches d'entrée/sortie ;
- 3 timers, 1 convertisseur analogique-numérique, des interfaces de communication. . .

Le tout consommant moins de 15 mW en fonctionnement normal.

2.1.3 Le microcontrôleur dans les objets du quotidien

Les systèmes embarqués utilisant des microcontrôleurs sont omniprésents : ils pilotent des appareils électroniques de consommation, des véhicules autonomes, des dispositifs médicaux et des systèmes industriels automatisés. Le rôle du microcontrôleur est de traiter les données issues des capteurs, d'exécuter des algorithmes et de contrôler les actionneurs de manière optimisée et fiable.

Quand vous réglez votre réveil sur 6h30 :

1. Vous appuyez sur des boutons pour entrer l'heure. Le µC **lit l'état des boutons** (entrées numériques).
2. Le µC **stocke** l'heure de réveil en mémoire (EEPROM — pour ne pas la perdre si on débranche le réveil).
3. Un **quartz** vibre exactement 32 768 fois par seconde. Le µC **compte ces vibrations** grâce à un timer. Après 32 768 vibrations = 1 seconde écoulée.
4. Chaque seconde, le µC met à jour l'heure et **l'affiche** sur l'écran à 7

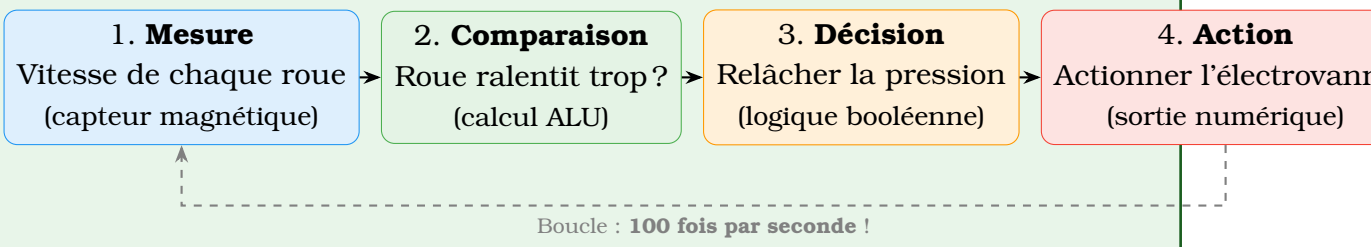
 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

segments (sorties).

5. Quand l'heure actuelle = l'heure programmée, le µC **active le buzzer** (sortie).

Bilan : Un seul composant, qui coûte moins de 0,50 €, gère toute la logique d'un réveil.

L'ABS (*Anti-lock Braking System*) empêche les roues de se bloquer lors d'un freinage brusque :



Le µC exécute cette boucle **100 fois par seconde** pour chaque roue. C'est impossible pour un humain, trivial pour un microcontrôleur à 16 MHz.

Quand vous présentez votre badge :

1. Le lecteur RFID **lit le numéro** du badge (communication série SPI ou I^2C).
2. Le µC **compare** ce numéro avec la liste des numéros autorisés (stockés en EEPROM ou Flash).
3. Si le numéro est dans la liste **ET** l'heure est dans la plage autorisée → le µC active un relais qui ouvre la porte.
4. Sinon → LED rouge + buzzer d'erreur.
5. Le µC **enregistre** l'événement (heure, badge, résultat) dans un journal en mémoire.

Vous reconnaissez ici la logique booléenne du chapitre 0 : l'accès est autorisé si `badge_valide AND heure_ok`.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Table 2.1 – Microcontrôleurs dans le quotidien : que fait-il exactement ?

Objet	**Entrées**	**Sorties**	**Traitement**
Réveil	Boutons, quartz	Afficheur, buzzer	Compter le temps, comparer avec l'alarme
Machine à laver	Bouton programme, capteur niveau d'eau, capteur température	Moteur, électrovanne, résistance	Enchaîner les phases selon le programme choisi
Télécommande TV	Boutons (20+)	LED infrarouge	Encoder chaque touche en séquence binaire (protocole NEC)
Thermostat	Capteur température, boutons réglage	Relais chauffage/clim, écran LCD	Comparer température mesurée au seuil, activer/désactiver
ABS voiture	4 capteurs de vitesse de roue	4 électrovannes de frein	100 comparaisons/seconde par roue
Badge d'accès	Lecteur RFID, horloge	Relais porte, LEDs, buzzer	Vérifier identité ET horaire autorisé

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.1.4 Pourquoi les microcontrôleurs sont-ils partout ?

Les 5 qualités qui expliquent leur succès

1. Intégration
Tout sur une seule puce : pas besoin de 10 composants séparés

2. Compacité
Tient dans un boîtier de 3 cm. Parfait pour montre, capteur, drone

3. Sobriété
15 mW en marche, 0,1 µW en veille. Des mois sur pile

4. Coût
Un ATmega328P coûte environ **2 €**.
Un ATtiny85 : **0,50 €**

5. Vitesse
16 millions d'opérations par seconde. Largement suffisant pour la plupart des applications

Chiffre clé : combien de µC dans le monde ?

On estime qu'il se vend environ **30 milliards** de microcontrôleurs par an dans le monde (source : IC Insights). C'est environ **4 µC par être humain par an**.
En comparaison, il se vend environ 400 millions de PC et 1,4 milliard de smartphones. Les microcontrôleurs sont donc **20 fois plus nombreux** que les smartphones.

Ce qu'il faut retenir

- Un microcontrôleur est un **ordinateur miniature complet sur une seule puce**.
- Il est **invisible** dans les objets mais **indispensable** à leur fonctionnement.
- Il suit toujours le même schéma : **lire** → **calculer** → **décider** → **agir**.
- Il est petit, pas cher, rapide et économe.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.2 Histoire et évolution des microcontrôleurs

L'histoire des microcontrôleurs remonte aux années 1970 avec l'apparition des premiers circuits intégrés programmables. Les premiers microprocesseurs, comme l'*Intel 4004* (1971), ont posé les bases en permettant une programmation flexible, mais n'étaient pas encore optimisés pour les applications embarquées.

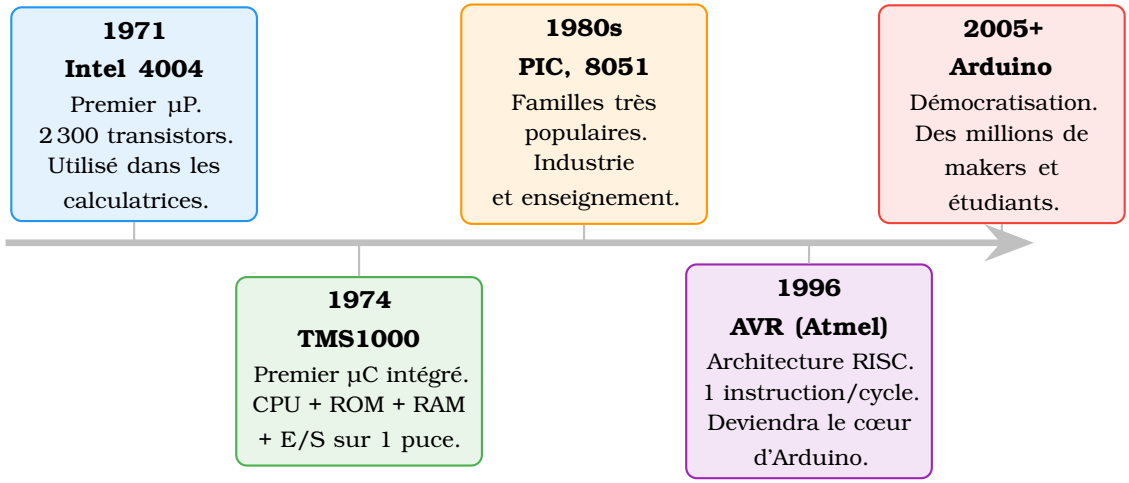
L'évolution des µC, comme celle des téléphones

Imaginez l'évolution des téléphones :

- **1970s** : téléphone fixe à fil (= premier microprocesseur Intel 4004, énorme et limité)
- **1990s** : premier portable Nokia (= premiers µC compacts comme le PIC et l'AVR)
- **2020s** : smartphone (= µC modernes ARM Cortex-M avec WiFi, Bluetooth, capteurs intégrés)

Comme les téléphones, les µC sont devenus plus **petits**, plus **puissants** et beaucoup moins **chers**, tout en faisant **plus de choses**.

Les grandes dates



En 1974, Texas Instruments introduit le *TMS 1000*, le premier microcontrôleur intégrant un processeur, une mémoire ROM et des entrées/sorties

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

programmables. Cette innovation réduit la complexité des systèmes embarqués en éliminant le besoin de composants externes multiples.
Les années 1980 et 1990 voient l'apparition de familles plus performantes comme les *PIC* de Microchip Technology et les AVR d'Atmel.
En résumé : en 50 ans, on est passé de puces à 2 300 transistors utilisées dans les calculatrices à des systèmes de plusieurs milliards de transistors capables de piloter une voiture autonome.

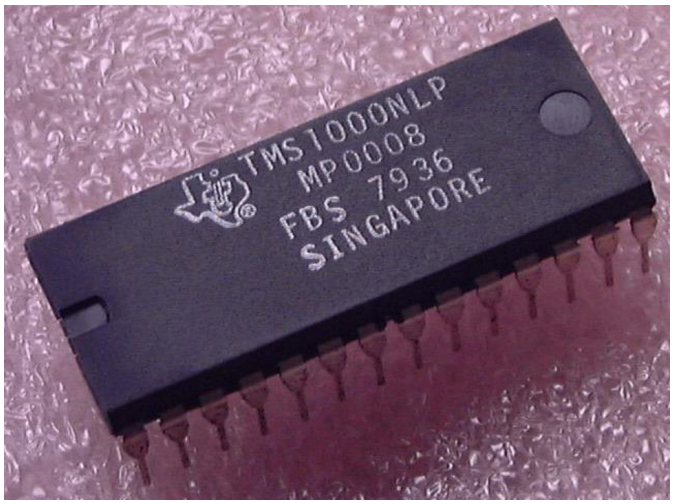


Figure 2.3 – Le microcontrôleur TMS 1000 de Texas Instruments (1974), premier microcontrôleur intégré : CPU + ROM + RAM + E/S sur une seule puce.

Aujourd'hui, les microcontrôleurs sont omniprésents, allant des objets connectés (IoT) aux applications industrielles de haute précision, avec des performances accrues et une intégration toujours plus poussée.

2.3 Microcontrôleur vs microprocesseur : quelle différence ?

C'est l'une des questions les plus fréquentes des débutants. Prenons le temps de bien l'expliquer.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Le restaurant vs le food truck

Microprocesseur = Restaurant complet

- Le chef (CPU) est excellent...
- ...mais il lui faut une cuisine (mémoire externe),
- des serveurs (circuits d'E/S),
- un bâtiment (carte mère),
- une équipe complète.
- **Puissant mais complexe et coûteux.**

Microcontrôleur = Food truck

- Le cuisinier (CPU) fait tout lui-même...
- ...dans un petit espace (une seule puce),
- avec sa propre plaque (E/S intégrées),
- son propre frigo (mémoire intégrée),
- tout est autonome.
- **Compact, autonome et économique.**

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.3.1 Comparaison détaillée

Table 2.2 – Microprocesseur vs microcontrôleur : comparaison point par point.

Critère	Microprocesseur (µP)	Microcontrôleur (µC)
Analogie	Restaurant avec équipe complète	Food truck autonome
Fonction	Exécution d'instructions générales, calcul généraliste puissant	Contrôle de systèmes embarqués spécifiques
Composants	CPU seul, nécessite mémoire et périphériques externes	CPU, mémoire et périphériques intégrés sur la puce
Mémoire	Externe (Go de RAM, To de SSD)	Intégrée (Ko à Mo)
E/S	Via chipset et cartes additionnelles	GPIO directement sur la puce
Consommation	5 à 150 W (nécessite ventilateur)	1 µW à 500 mW (fonctionne sur pile)
Prix	10 € à 1 000+ €	0,20 € à 10 €
Exemples	Intel Core i7, AMD Ryzen 9, Apple M3	ATmega328P, STM32, ESP32, PIC
Applications	Ordinateurs, serveurs, stations de travail, smartphones, IA, Big Data	Réveil, thermostat, voiture, drone, Arduino, domotique, IoT

2.3.2 Comparaison historique

Les microprocesseurs sont apparus dans les années 1970, avec le célèbre *Intel 4004*. Cependant, leur dépendance aux composants externes limitait leur utilisation dans des applications compactes et à faible consommation.

Dans les années 1980, l'essor des microcontrôleurs a permis de concevoir

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

des systèmes plus intégrés. Depuis, les microcontrôleurs dominent les systèmes embarqués, tandis que les microprocesseurs restent essentiels pour les applications à haute performance.

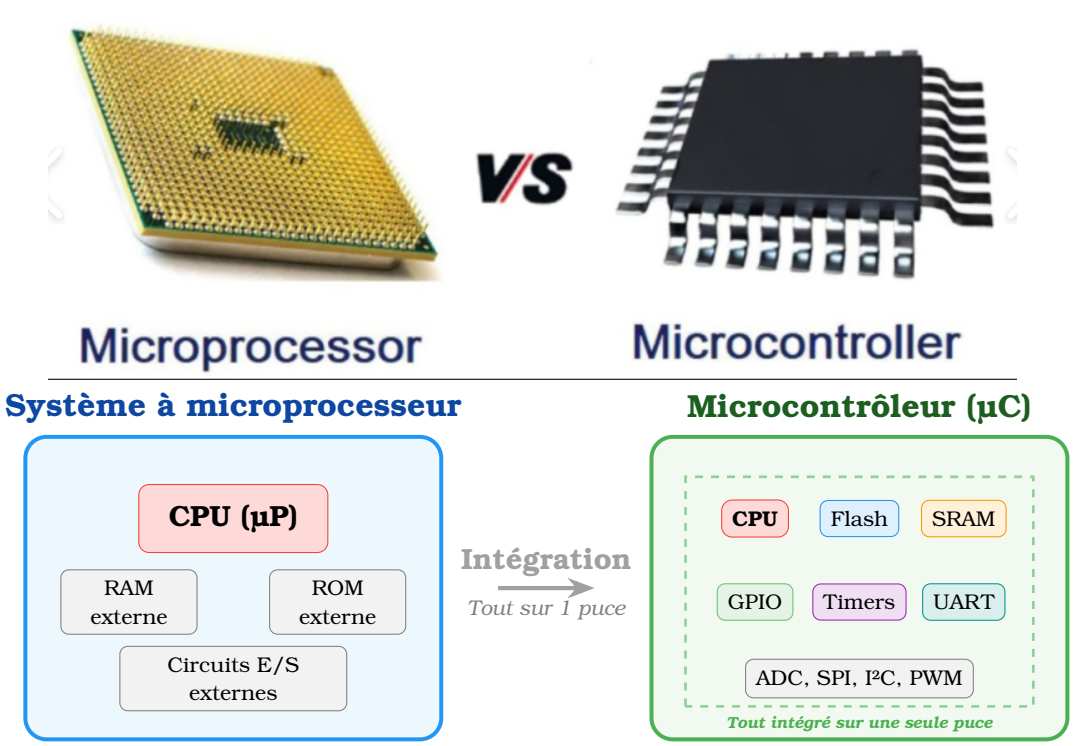


Figure 2.4 – Comparaison visuelle : le µP a besoin de composants externes, le µC intègre tout sur une puce.

Erreur courante

« **Un microcontrôleur, c'est un petit microprocesseur** » — FAUX !
Ce n'est pas une question de taille ou de puissance. La différence fondamentale est **l'intégration** :

- Le µP **ne contient que le CPU**. Il a besoin de composants externes.
- Le µC **contient tout** : CPU + mémoire + E/S + périphériques, sur une seule puce.

Un µP très petit (comme un ARM Cortex-A7) reste un µP. Un µC très puissant (comme un ESP32 avec WiFi et Bluetooth) reste un µC. **Ce qui les différencie, c'est le niveau d'intégration, pas la puissance.**

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

</> Pourquoi un thermostat n'utilise pas un Intel Core i7 ?

Un thermostat doit :

- lire une température (1 valeur) ;
- la comparer à un seuil (1 soustraction) ;
- allumer ou éteindre le chauffage (1 sortie).

Total des besoins : quelques centaines d'instructions par seconde.
Un Intel Core i7 peut exécuter **100 milliards** d'instructions par seconde.
C'est comme utiliser un **Boeing 747 pour aller chercher le pain**.
Le µC ATmega328P fait le travail parfaitement à 16 MHz, coûte **2 €** au lieu de **400 €**, consomme **15 mW** au lieu de **125 W**.

2.4 Les microcontrôleurs dans le monde réel

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Où trouve-t-on des microcontrôleurs ? Partout !

	Domaine	Exemples concrets	Nb µC
	Maison	Four, thermostat, lave-linge, cafetière, alarme, éclairage LED, volets roulants	10–30
	Automobile	Gestion moteur, ABS, airbags, climatisation, tableau de bord, capteurs de recul	50–100
	Santé	Thermomètre frontal, tensiomètre, pompe à insuline, oxymètre, pacemaker	1–5
	Industrie	Robot soudeur, capteur de vibration, automate de production	10–50
	IoT	Station météo connectée, serrure connectée, tracker GPS, arrosage automatique	1–3
	Loisirs	Drone, robot aspirateur, console de jeu rétro, guirlande LED programmable	3–20
	Aéronautique	Contrôle des systèmes de vol, satellites, systèmes de navigation	20–200

La figure 2.5 illustre la diversité des domaines d'application des microcontrôleurs, de la domotique à l'aéronautique.

La voiture : un réseau de microcontrôleurs

Une voiture moderne contient typiquement **50 à 100 microcontrôleurs** qui communiquent entre eux via un réseau appelé **CAN bus** (*Controller Area Network*).
Chaque µC est dédié à une fonction :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

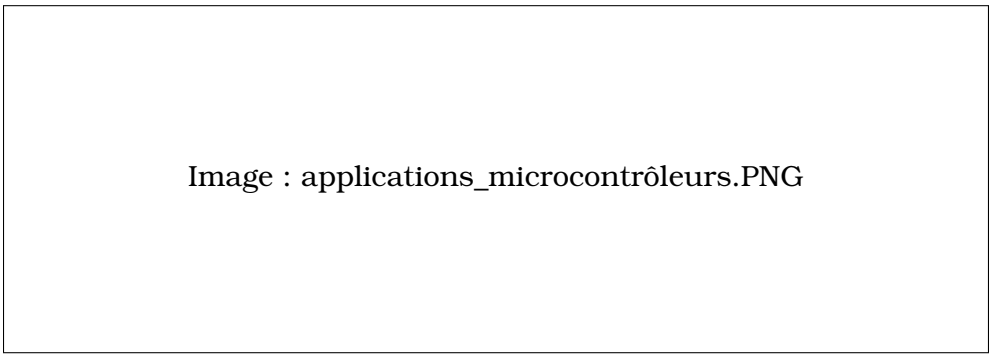


Figure 2.5 – Exemples d'applications des microcontrôleurs : objets connectés, dispositifs médicaux, automobile, IoT, aéronautique.

- Gestion moteur
- Boîte de vitesses
- ABS / ESP
- Airbags
- Direction assistée
- Climatisation
- Tableau de bord
- Éclairage
- Fermeture centralisée
- Lève-vitres
- Rétroviseurs
- Essuie-glaces auto

Ils fonctionnent tous en parallèle, chacun exécutant son propre programme.

2.5 La famille AVR : notre terrain de jeu

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.5.1 Pourquoi apprendre avec les AVR ?

La famille AVR : idéale pour débuter

Les microcontrôleurs **AVR**, développés par Atmel (désormais Microchip Technology), sont largement utilisés dans les systèmes embarqués en raison de leur faible coût, de leur efficacité énergétique et de leur facilité de programmation.
Créés en 1996 par deux étudiants norvégiens, Alf-Egil Bogen et Vegard Wollan, à l'Université de Trondheim, ils sont devenus le cœur des cartes Arduino.

Pourquoi sont-ils parfaits pour apprendre ?

Lisibilité
Le jeu d'instructions est petit (131 instructions), régulier et bien documenté. Chaque instruction fait **une seule chose**.

Écosystème
Arduino = des millions de tutoriels, forums, bibliothèques. Vous ne serez **jamais seul**.

Accessibilité
Une carte Arduino Uno coûte **10 €**. Simulation gratuite sur Tinkercad.

En pratique : dans ce cours, nous utiliserons l'**ATmega328P**, le µC qui équipe la carte Arduino Uno.

Pourquoi pas un ARM Cortex-M directement ?

C'est comme apprendre à conduire :

- On commence avec une **petite voiture** (= AVR) : peu de puissance, mais on comprend tout.
- Ensuite, on peut passer à une **voiture de sport** (= ARM) : plus puissante, mais les bases sont les mêmes.

Les concepts appris sur AVR (registres, mémoire, interruptions, E/S) s'appliquent à **toutes les architectures**.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.5.2 Famille AVR : ATmega et ATtiny

Microcontrôleurs ATmega Les ATmega sont des microcontrôleurs 8 bits performants, intégrant davantage de mémoire et de périphériques (UART, SPI, I2C, PWM). Ils sont privilégiés pour :

- **Systèmes embarqués avancés** : automatisation industrielle, robotique, objets connectés.
- **Cartes de prototypage** : l'ATmega328P est utilisé dans les cartes Arduino Uno.
- **Interfaces de communication** : intégration dans des modules Wi-Fi, Bluetooth et LoRa.

Microcontrôleurs ATtiny Conçus pour des applications compactes et à faible consommation :

- **Systèmes embarqués miniatures** : commande d'affichage LED, petits automatismes.
- **Capteurs et dispositifs autonomes** : surveillance environnementale, domotique, wearables.
- **Électronique à faible consommation** : dispositifs alimentés par batterie, capteurs IoT.

Table 2.3 – Les deux sous-familles AVR que vous rencontrerez.

Famille	Exemple	Flash	RAM	Broches	Usage typique
ATtiny	ATtiny85	8 Ko	512 o	8	Gadget, LED, capteur simple, wearable
ATmega	**ATmega328**	**32 Ko**	**2 Ko**	**28**	**Arduino Uno, ce cours**

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

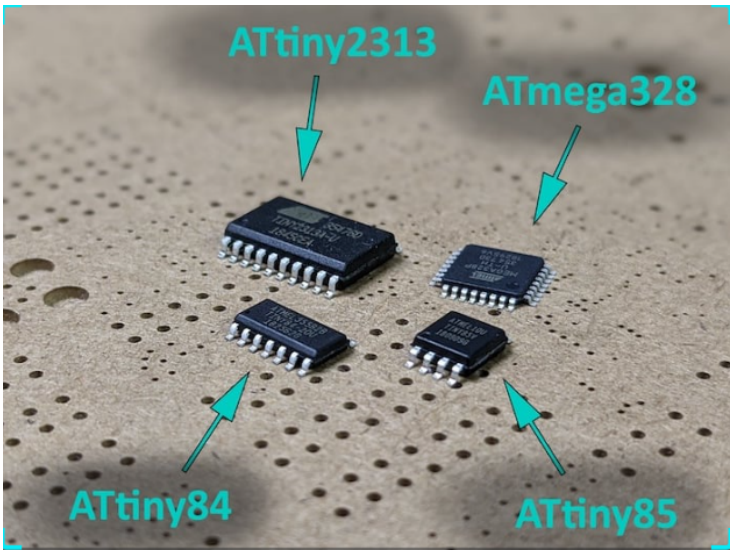


Figure 2.6 – Comparaison ATtiny85 vs ATmega328P.

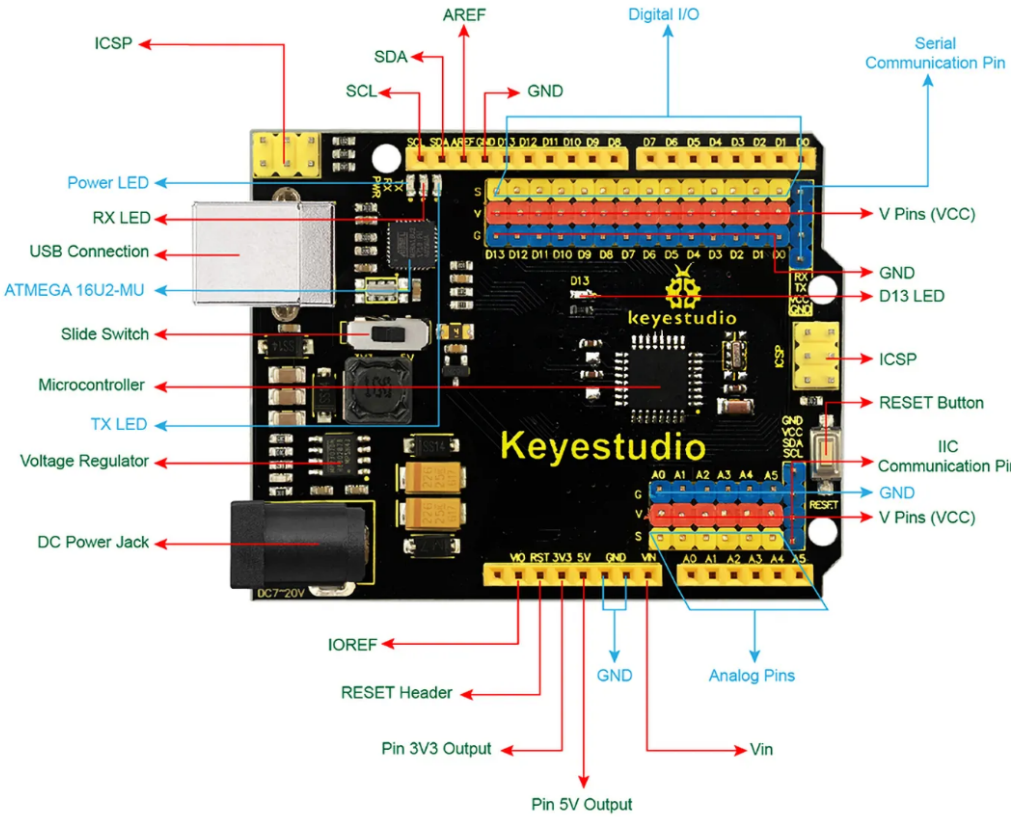
2.6 Architecture interne : visite guidée de l'ATmega328P

Nous allons maintenant « ouvrir le capot » du µC et examiner chaque composant, un par un.

Comment lire cette figure

Ce schéma montre que **tout est sur la même puce**. Chaque bloc coloré correspond à un sous-système que nous étudierons en détail :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



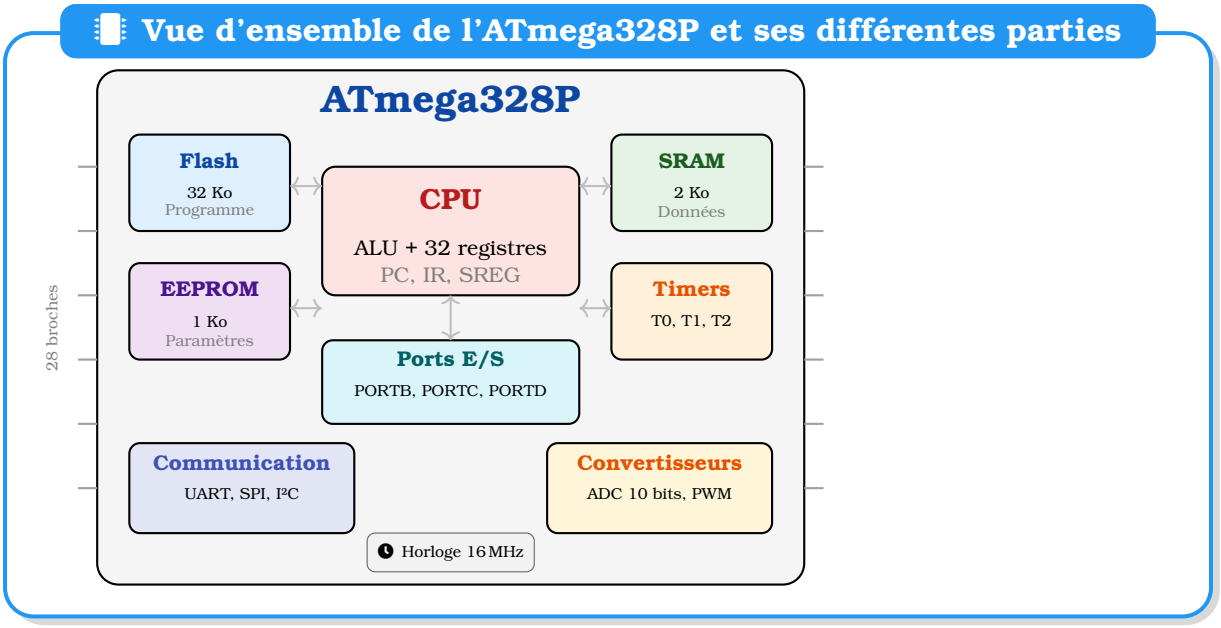


Figure 2.7 – Présentation des différentes parties internes de l'ATmega328P : CPU avec ALU et 32 registres au centre, entouré des trois mémoires (Flash, SRAM, EEPROM), des ports d'entrée/sortie (PORTB, PORTC, PORTD), des timers (T0, T1, T2), des interfaces de communication (UART, SPI, I²C) et des convertisseurs (ADC, PWM). L'ensemble est cadencé par un oscillateur à 16 MHz.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

	Bloc	Rôle — sera étudié en
🔴	**CPU + ALU**	Exécute les instructions — *ce chapitre et ch. 2*
🔵	**Flash**	Stocke votre programme — *ce chapitre*
🟢	**SRAM**	Variables et pile — *ch. 4*
🟣	**EEPROM**	Paramètres persistants — *ch. 5*
🩵	**Ports E/S**	Capteurs et actionneurs — *ch. 5*
🟠	**Timers**	Mesure du temps — *ch. 5*
🔷	**Communication**	UART, SPI, I²C — *ch. 6*

Quand vous écrirez `LDI R16, 0xFF` au chapitre 2, vous saurez exactement **où** dans cette puce l'opération a lieu (le bloc rouge, registre R16).

Le µC comme une petite usine

Imaginez une usine miniature :

- Le **directeur** (CPU) décide quoi faire.
- Le **calculateur** (ALU) effectue les opérations.
- Les **tiroirs sur le bureau** (registres) permettent de manipuler rapidement des valeurs.
- Le **manuel d'instructions** (Flash) contient la procédure à suivre.
- Le **bloc-notes** (SRAM) sert à noter des résultats temporaires.
- Le **coffre-fort** (EEPROM) conserve les données importantes même en cas de coupure.
- Les **portes et fenêtres** (E/S) permettent de communiquer avec l'extérieur.
- L'**horloge murale** (oscillateur) rythme le travail.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

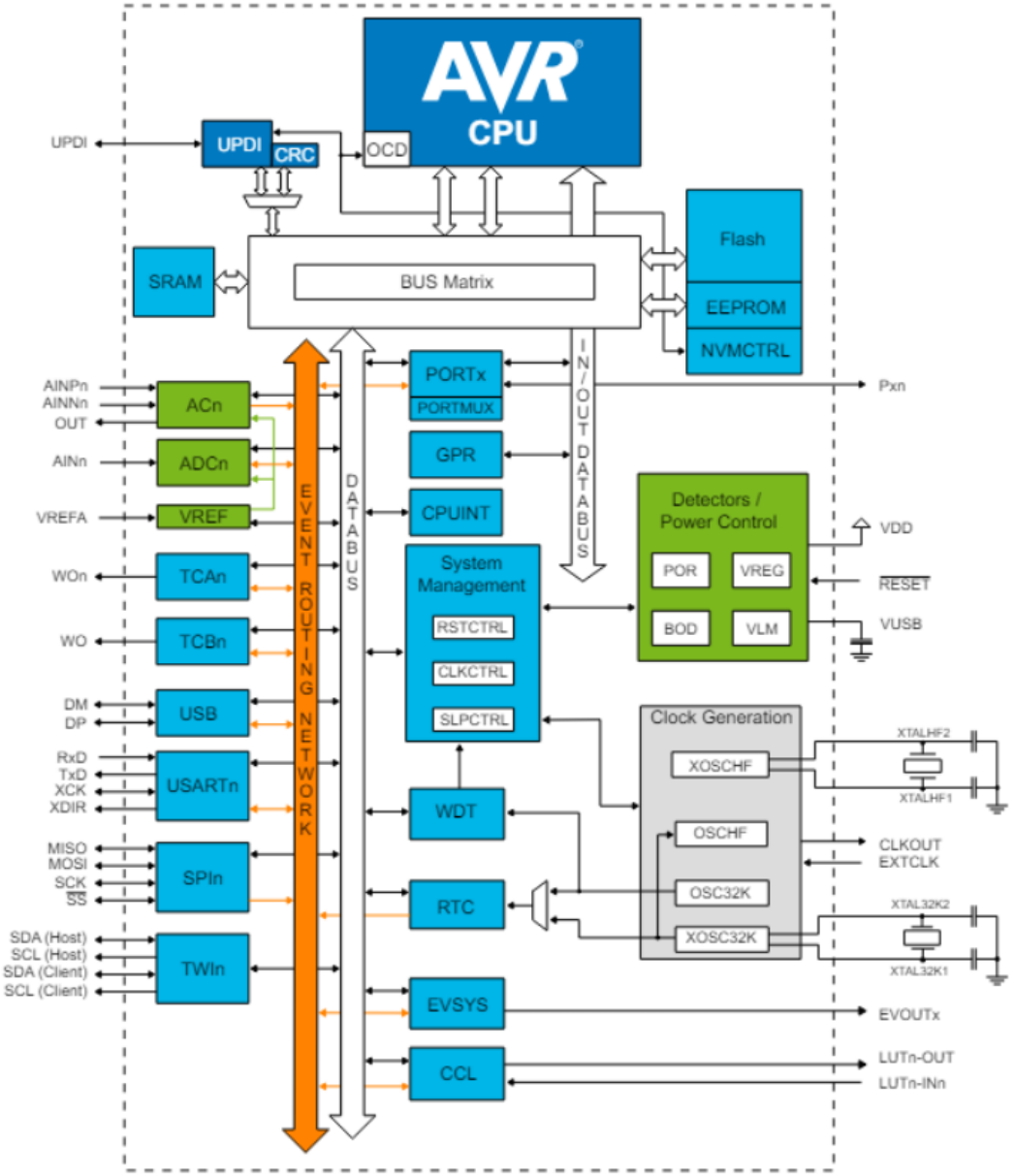


Figure 2.8 – Schéma de l'architecture interne d'un microcontrôleur AVR.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.6.1 Architecture RISC et Harvard : deux choix de conception

Architecture RISC (*Reduced Instruction Set Computing*)

L'architecture **RISC** repose sur un principe simple : **moins d'instructions, mais chacune est très rapide**.

Analogie : Imaginez deux cuisiniers.

- Le premier (CISC) connaît 500 recettes, mais chaque recette prend un temps variable.
- Le second (RISC) ne connaît que 130 recettes simples, mais chacune prend **exactement 1 minute**.

L'AVR est RISC : il possède **131 instructions**, et la grande majorité s'exécute en **un seul cycle d'horloge**. Cela permet un pipeline à un seul cycle pour l'exécution de la plupart des instructions.

Architecture Harvard

L'architecture **Harvard** utilise **deux mémoires séparées** avec **deux bus distincts** :

- un bus pour les **instructions** (programme en Flash) ;
- un bus pour les **données** (variables en SRAM).

Avantage : le processeur peut lire l'instruction suivante **en même temps** qu'il manipule les données de l'instruction en cours. C'est comme lire la recette de cuisine tout en mélangeant les ingrédients.
Par contraste : l'architecture *Von Neumann* (utilisée dans les PC) partage un seul bus. C'est plus simple mais plus lent.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

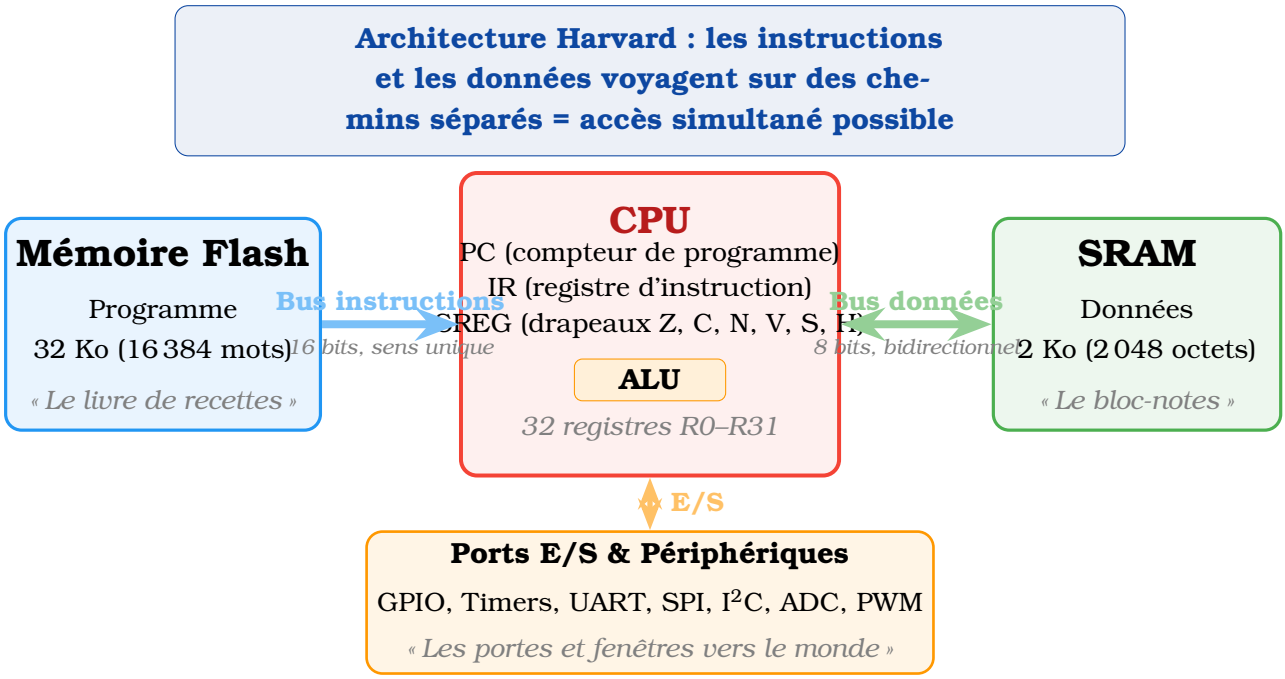


Figure 2.9 – Architecture Harvard de l'ATmega328P. Les deux bus séparés permettent de lire la prochaine instruction pendant que l'instruction en cours manipule des données.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.6.2 Le CPU et l'ALU : le cerveau et le calculateur

Le CPU (*Central Processing Unit*)

Le CPU est le **chef d'orchestre**. Il :

1. **va chercher** l'instruction suivante en Flash (grâce au PC) ;
2. **la décode** : « c'est une addition ? une comparaison ? » ;
3. **la fait exécuter** par l'ALU ou par un périphérique ;
4. **range le résultat** dans un registre ou en mémoire ;
5. **passe à l'instruction suivante** (PC += 1).

L'ALU (*Arithmetic and Logic Unit*)

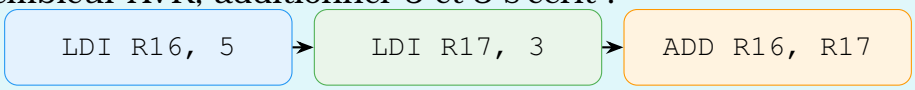
L'ALU est le **calculateur**. Elle sait faire :

1. **Opérations arithmétiques** : addition, soustraction, multiplication, incrémentation, décrémentation
2. **Opérations logiques** : ET, OU, XOR, inversion
3. **Opérations de décalage et rotation** pour manipuler les données binaires
4. **Comparaisons** : $7 > 3$?
5. **Mise à jour des drapeaux** : zéro ? négatif ? retenue ?

C'est tout ! Et pourtant, avec ces opérations, on peut faire tout ce qu'un programme fait.

L'ALU en action : 5 + 3

En assembleur AVR, additionner 5 et 3 s'écrit :



Après l'exécution : R16 contient 8.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.6.3 Les 32 registres : les tiroirs ultra-rapides

Les registres, c'est comme les poches de votre veste

- Vos **poches** = les registres. Accès instantané, mais capacité limitée.
- Votre **sac à dos** = la SRAM. Plus de place, mais il faut l'ouvrir (plus lent).
- Votre **armoire à la maison** = la Flash. Beaucoup de place, mais il faut rentrer chez vous.

Le processeur essaie toujours de travailler avec ses « poches » d'abord, car c'est **instantané**.

L'AVR dispose de **32 registres de travail généraux (R0–R31)** de 8 bits chacun, qui permettent d'exécuter des opérations directement sans accès à la mémoire. Parmi ces registres :

- Les registres R0 à R31 pour le stockage temporaire des données.
- Les registres d'entrée/sortie pour le contrôle des périphériques internes.
- Les registres dédiés à la gestion des interruptions et du statut du processeur (SREG).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

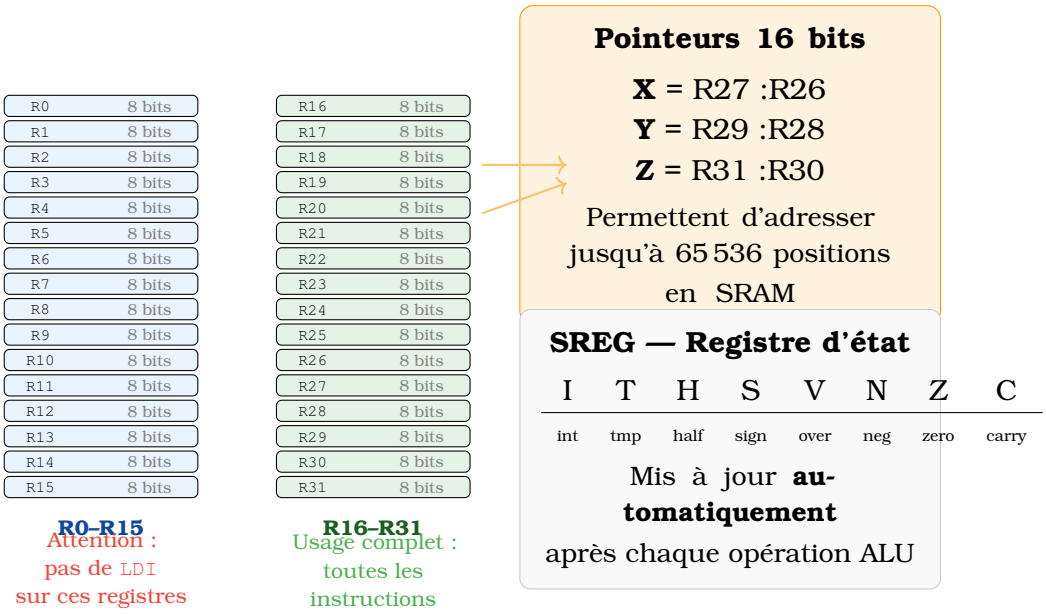


Figure 2.10 – Les 32 registres de l'AVR. R16–R31 acceptent toutes les instructions. R26–R31 peuvent former des pointeurs 16 bits (X, Y, Z).

Pourquoi R16–R31 et pas R0–R15 ?

En assembleur AVR, l'instruction `LDI` (*Load Immediate*) ne fonctionne **que sur R16 à R31**.
`LDI R16, 42` fonctionne. Mais `LDI R5, 42` provoquera une **erreur d'assemblage**.
C'est une limitation de l'encodage des instructions sur 16 bits : il n'y a que 4 bits pour le numéro de registre dans le format de `LDI`, ce qui permet uniquement les registres 16 à 31.

2.6.4 Les trois mémoires, expliquées simplement

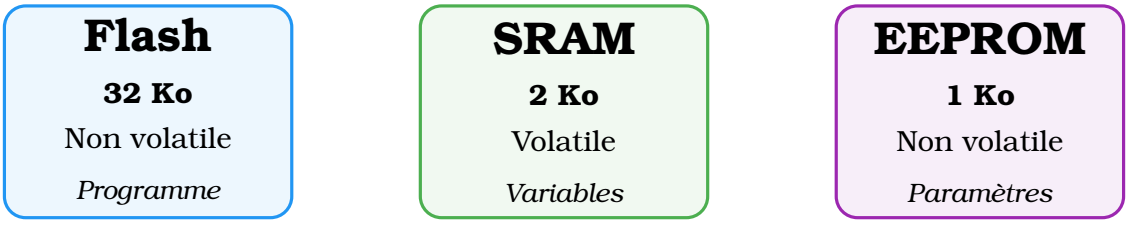


Figure 2.11 – Les trois mémoires de l'ATmega328P.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Table 2.4 – Comparaison détaillée des trois mémoires.

Mémoire	Taille	Persistante	Analogie	Contient
Flash	32 Ko	**Oui**	Livre de recettes	Le programme (vos instructions assembleur)
SRAM	2 Ko	**Non**	Bloc-notes	Variables, pile, résultats temporaires
EEPROM	1 Ko	**Oui**	Coffre-fort	Paramètres persistants (calibration, config)

Erreur courante

« **32 Ko, c'est minuscule !** » — Pas pour un microcontrôleur !
32 Ko de Flash, c'est **16 384 instructions** assembleur (chaque instruction fait 2 octets). Un programme de thermostat complet tient en **environ 2 000 instructions**. La Flash de l'ATmega328P peut donc contenir **8 programmes de ce type** simultanément.
Un PC a besoin de Go de RAM parce qu'il exécute un OS, un navigateur, un éditeur, un antivirus... en même temps. Le µC n'exécute qu'un seul programme, beaucoup plus simple.

2.6.5 Les périphériques et interfaces

En plus du CPU et de la mémoire, le µC contient des **périphériques intégrés** qui étendent ses capacités sans ajouter de composants externes.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

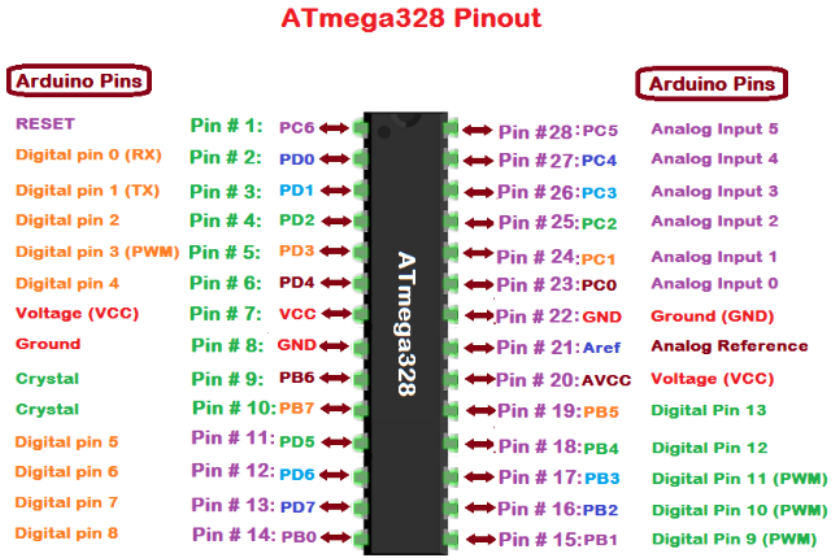


Figure 2.12 – Configuration des broches de l'ATmega328P.

Les périphériques de l'ATmega328P

	Périphérique	Analogie	Exemple d'utilisation
	Timers (0, 1, 2)	Un chronomètre intégré	Faire clignoter une LED toutes les 500 ms, mesurer la durée d'un appui bouton
	UART/USART	Un téléphone : envoyer/recevoir des messages	Afficher des données sur un terminal PC, communiquer avec un module Bluetooth
	SPI	Un talkie-walkie rapide	Piloter un écran LCD, lire une carte SD, connecter des mémoires
	I²C	Un bus scolaire : un seul chemin, plusieurs passagers	Connecter 10 capteurs différents avec seulement 2 fils
	ADC (10 bits)	Un thermomètre numérique intégré	Lire un potentiomètre, mesurer la tension d'une batterie, lire un capteur de lumière
	PWM	Un variateur de lumière	Faire varier la luminosité d'une LED, contrôler la vitesse d'un moteur

© 2025-2026 Dr AFENYIVEH Dzo M. Serge

2.6.6 Description des broches et fonctions associées

Les broches des microcontrôleurs AVR remplissent plusieurs fonctions selon leur configuration :

- **Entrées/sorties numériques** : configuration en mode entrée ou sortie pour interagir avec d'autres composants.
- **Broches d'alimentation** : VCC et GND pour fournir l'énergie nécessaire.
- **Broches de communication** : spécialisées pour UART, SPI, I^2C, etc.
- **Broches d'interruptions** : permettent une réponse rapide aux événements extérieurs.
- **Broches de timers/PWM** : utilisées pour la génération de signaux de contrôle de précision.

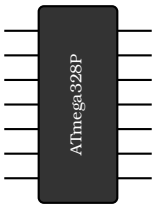
</> L'ADC en action : mesurer une température

Supposons que vous connectez un capteur de température (type LM35) à la broche A0 :

1. Le capteur génère une **tension analogique** proportionnelle à la température : $V = \text{température} \times 10\,\text{mV/°C}$
2. L'ADC 10 bits du µC convertit cette tension en un **nombre** entre 0 et 1023 :
$$\text{valeur} = \frac{V \times 1023}{5\,\text{V}}$$
3. Le programme calcule la température :
$$T = \frac{\text{valeur} \times 5}{1023} \times 100 \quad \text{(en °C)}$$
4. Le résultat est affiché sur un écran ou envoyé par UART au PC.

Bilan : le µC a lu une grandeur physique, l'a convertie en nombre, a fait un calcul, et a communiqué le résultat. Tout cela en quelques microsecondes.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



DIP-28

Figure 2.13 – Présentation de l'ATmega328P en boîtier DIP-28, la puce au cœur de l'Arduino Uno.

Le boîtier physique DIP-28 de l'ATmega328P est visible sur la figure 2.7 ci-dessus.

2.7 L'horloge, le reset et le cycle d'instruction

2.7.1 L'horloge : le métronome du µC

L'horloge, c'est comme le métronome d'un musicien

Un musicien joue les notes au rythme du métronome. Plus le métronome bat vite, plus la musique avance rapidement.
Le µC fonctionne pareil :

- Chaque « tic » de l'horloge = un cycle.
- À chaque cycle, le µC avance d'un pas dans le programme.
- Plus l'horloge est rapide, plus le µC travaille vite.

Horloge système

Le µC est cadencé par un **oscillateur à quartz** qui produit un signal régulier.
L'ATmega328P sur Arduino Uno fonctionne à :

$$f = 16\,\text{MHz} = 16\,000\,000 \text{ cycles par seconde}$$

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Chaque cycle dure :

$$T = \frac{1}{f} = \frac{1}{16\,000\,000} = 62{,}5\,\text{ns}$$

En d'autres termes : le µC peut exécuter une instruction en 62,5 ns, soit **16 millions d'instructions par seconde**.

Pour donner une idée de la vitesse. . .

En **1 seconde**, l'ATmega328P exécute autant d'instructions qu'un humain qui ferait une opération toutes les secondes pendant. . .

$$\frac{16\,000\,000}{3\,600 \times 24 \times 365} \approx \textbf{507 années}$$

Le µC fait en 1 seconde ce qu'un humain mettrait **5 siècles** à faire !

2.7.2 Le reset : redémarrer proprement

Reset

Le **reset** est l'équivalent d'un « redémarrage » du µC. Il :

- remet tous les registres à zéro ;
- replace le compteur de programme (PC) à l'adresse `0x0000` ;
- redémarre le programme depuis le début.

1. **Mise sous tension** (*Power-on Reset*) : quand vous branchez l'alimentation. *Comme allumer votre PC.*
2. **Bouton reset** (*External Reset*) : le bouton RESET de la carte Arduino. *Comme le bouton reset d'une console de jeu.*
3. **Chien de garde** (*Watchdog Reset*) : un timer spécial vérifie que le programme ne s'est pas « planté ». Si le programme ne « caresse le chien » régulièrement, le µC se redémarre. *Comme un vigile qui vérifie que l'employé est toujours éveillé.*

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

4. **Tension trop basse** (*Brown-out Reset*) : si la batterie est presque vide, le µC se réinitialise. *Comme un PC qui s'éteint quand la batterie est à 1%.*

2.7.3 Le cycle d'instruction : comment le µC exécute votre programme

Le cycle d'instruction, c'est comme suivre une recette de cuisine

1. **FETCH** (Chercher) : vous lisez l'instruction suivante dans le livre.
2. **DECODE** (Comprendre) : vous comprenez ce qu'il faut faire.
3. **EXECUTE** (Exécuter) : vous faites l'action.
4. **WRITE-BACK** (Ranger) : vous posez le résultat.
5. **NEXT** : vous passez à la ligne suivante et recommencez.

Le µC fait exactement pareil, mais **16 millions de fois par seconde**.

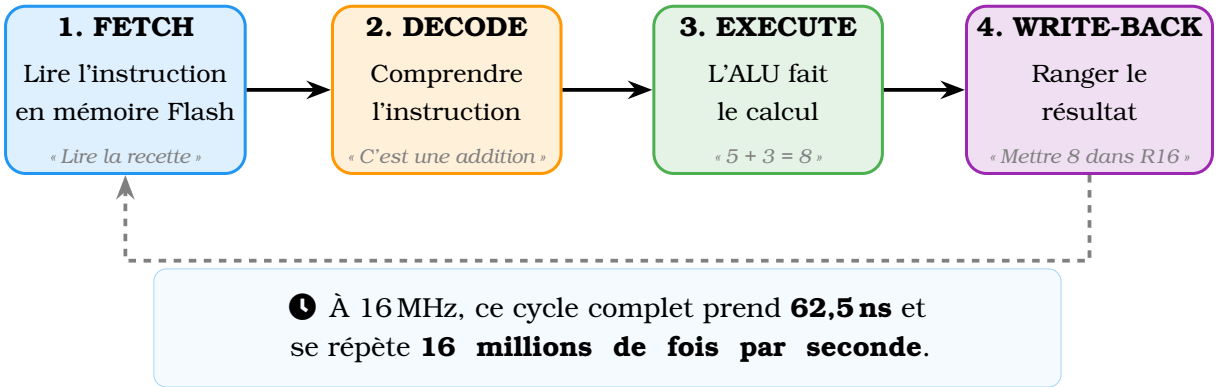


Figure 2.14 – Le cycle Fetch-Decode-Execute-Writeback, avec analogies culinaires pour chaque étape.

Exemple pas-à-pas : allumer une LED

Supposons que votre programme contient l'instruction `SBI PORTB, 5` (allumer la LED sur la broche 13 de l'Arduino = bit 5 du port B) :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Étape	Ce que fait le µC	Explication pour un débutant
1. Fetch	Lit `0x9A2D` en Flash	Le µC va chercher la prochaine instruction dans son « livre de recettes » (la Flash).
2. Decode	Reconnaît `SBI PORTB, 5`	Le décodeur traduit : « mettre le bit 5 du registre PORTB à 1 ».
3. Execute	Met le bit 5 de PORTB à 1	Le bit passe de 0 à 1. La broche PB5 passe de 0 V à 5 V.
4. Write-back	LED allumée !	Le courant traverse la LED (via une résistance), et elle s'allume.
5. PC += 1	Passe à l'instruction suivante	Le compteur de programme avance d'un cran.

Durée totale : 125 ns (2 cycles à 16 MHz). Votre œil ne verra rien : la LED s'allume « instantanément ».

2.8 Compilation croisée et développement pour AVR

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.8.1 Pourquoi ne peut-on pas programmer directement le µC ?

Écrire un livre dans une langue étrangère

- Vous, vous parlez **français** (= langage C).
- L'ouvrier ne comprend que le **chinois** (= code machine AVR, des suites de 0 et de 1).
- Vous avez besoin d'un **traducteur** sur votre bureau (= le compilateur `avr-gcc` sur votre PC).
- Une fois traduit, vous envoyez le texte à l'ouvrier (= téléversement avec `avrdude`).

C'est exactement ce qu'est la **compilation croisée** : traduire sur votre PC un programme en C vers le langage que comprend le µC AVR.

Compilation croisée (*Cross-compilation*)

La **compilation croisée** est un processus permettant de générer un programme exécutable pour une architecture cible (comme un µC AVR) en le compilant sur une architecture hôte différente (votre PC sous Linux ou Windows).

Ce concept est fondamental dans le développement embarqué, car il permet d'utiliser un environnement de développement plus puissant pour produire du code optimisé pour des dispositifs aux ressources limitées.

C'est nécessaire car :

- le µC n'a pas de clavier, pas d'écran, pas de système d'exploitation ;
- ses 2 Ko de RAM ne suffiraient pas pour un compilateur ;
- le format des instructions AVR (16 bits) est incompatible avec celui du PC (64 bits) ;
- un gain de performance est recherché (compiler sur un PC est bien plus rapide) ;
- il est important de tester et de déboguer avant de téléverser sur la cible.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.8.2 Pourquoi et comment utiliser la compilation croisée ?

La mise en œuvre de la compilation croisée implique :

- L'installation d'un compilateur croisé adapté à la cible (ex. `AVR-GCC` pour les µC AVR).
- L'écriture d'un code compatible avec l'architecture cible.
- La conversion de l'exécutable généré en un format téléchargeable sur la cible (`.hex`).

2.8.3 La chaîne de compilation croisée

Une chaîne de compilation croisée comprend plusieurs étapes essentielles :

- **Préprocesseur** : remplace les macros, inclut les fichiers d'en-tête et génère un code source « épuré ».
- **Compilateur** : convertit le code source en assembleur adapté à l'architecture cible.
- **Assembleur** : transforme le code assembleur en code machine (fichier objet `.o`).
- **Éditeur de liens (Linker)** : combine les fichiers objets en un exécutable (`.elf`).
- **Conversion du fichier binaire** : transformation en un format compatible avec la programmation du µC (`.hex`).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

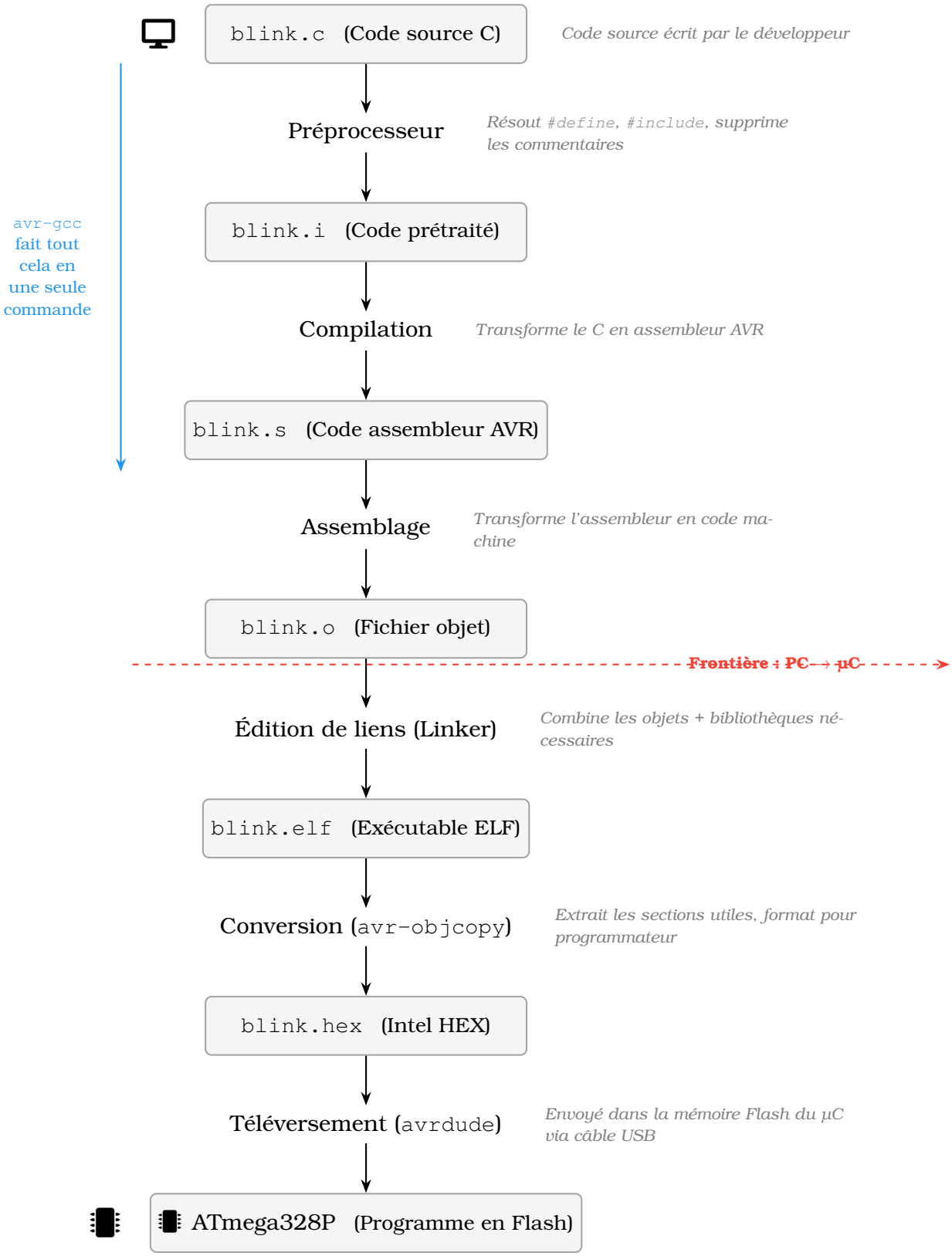


Figure 2.15 – Chaîne complète de compilation croisée pour AVR : du code source C au programme dans la Flash du µC. La commande `avr-gcc` effectue les étapes 1 à 4 automatiquement.

© 2025-2026 Dr AFENTIVEH Dzo-M. Serge

Légende des formats de fichiers :

- `.c` : code source écrit par le développeur
- `.i` : résultat du préprocesseur C (macros et `#include` résolus)
- `.s` : code assembleur généré par le compilateur
- `.o` : fichier objet, compilation partielle en binaire non exécutable
- `.elf` : fichier exécutable complet (format utilisé dans l'embarqué)
- `.hex` : format Intel HEX, utilisé pour programmer les microcontrôleurs

2.8.4 Exemple de compilation croisée pour ATmega328P

Prenons un exemple concret de compilation avec l'outil `AVR-GCC` :

1. **Préparation**

   Avant toute compilation, vérifions que l'environnement est installé :

```
avr-gcc --version
```

   Cela permet de s'assurer que l'environnement AVR est correctement installé (incluant avr-gcc, avr-binutils, avr-libc).

2. **Prétraitement, compilation, assemblage et édition de lien**

   La commande suivante effectue toutes les étapes de compilation croisée en une seule commande :

```
avr-gcc blink.c -o blink.elf -mmcu=atmega328p \
    -DF_CPU=16000000UL -Os
```

   Détail de chaque option :

   - `blink.c` : fichier source en langage C.
   - `-o blink.elf` : fichier binaire de sortie au format ELF (*Executable and Linkable Format*).
   - `-mmcu=atmega328p` : cible la puce ATmega328P pour définir les registres, instructions, taille mémoire, etc.
   - `-DF_CPU=16000000UL` : définit la fréquence d'horloge (16 MHz) utilisée par le code (ex. `_delay_ms()`).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- `-Os` : optimisation pour la taille (important sur microcontrôleurs à mémoire limitée).

Étapes internes implicites de cette commande :

a) **Prétraitement** : remplacement des `#define`, inclusion des fichiers `.h`, suppression des commentaires.

b) **Compilation** : transformation du C en assembleur AVR (`.s`).

c) **Assemblage** : transformation de l'assembleur en code machine (`.o`).

d) **Linking** : combinaison des objets, ajout des bibliothèques nécessaires → génération de `blink.elf`.

3. Conversion au format Intel HEX

```
avr-objcopy -O ihex -R .eeprom blink.elf blink.hex
```

Détail :

- `avr-objcopy` : extrait les sections utiles de `blink.elf` et les convertit dans un format compatible avec les programmateurs.
- `-O ihex` : spécifie la sortie au format **Intel HEX**, compris par la plupart des outils comme `avrdude`.
- `-R .eeprom` : exclut la section EEPROM (on ne programme que la Flash ici).
- `blink.hex` : fichier final à téléverser dans la mémoire Flash du µC.

4. Téléversement avec avrdude

```
avrdude -c arduino -p atmega328p \
    -U flash:w:blink.hex:i -P /dev/ttyUSB0
```

Explication des options :

- `-c arduino` : spécifie le programmeur (ici via bootloader Arduino).
- `-p atmega328p` : définit la puce cible.
- `-P /dev/ttyUSB0` : port série utilisé (sous Windows : `-P COM6`).
- `-U flash:w:blink.hex:i` :
  - `flash` : indique qu'on écrit dans la mémoire Flash,

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- `w:blink.hex` : spécifie le fichier à écrire,
- `:i` : format Intel HEX.

Remarque

Pour des projets plus complexes, on utilise souvent `avr-gcc -c` pour générer des objets (`.o`) séparés avant de les lier. L'IDE Arduino fait tout cela automatiquement quand vous cliquez sur « Téléverser ». Mais il est important de comprendre ce qui se passe en coulisses.

2.9 Mise en place de l'environnement et premier programme

La mise en place d'un environnement de développement adapté est essentielle pour programmer et tester des applications pour les microcontrôleurs AVR.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.9.1 Outils de développement

Table 2.5 – Les outils pour développer sur AVR — du plus simple au plus avancé.

	Outil	Pour qui / Description	Coût	Matériel ?
☁	**Tinkercad**	**Débutant complet.** Simulation en ligne, rien à installer.	Gratuit	Non
💻	**Arduino IDE**	Débutant avec une carte Arduino. Environnement simple et intuitif.	Gratuit	Oui
⚙	**Microchip Studio**	Intermédiaire/Avancé. IDE complet avec débogage pas-à-pas, simulation et vue des registres en temps réel.	Gratuit	Optionnel
🧪	**Proteus**	Simulation avancée de circuits complets avec visualisation oscilloscope.	Payant	Non
>_	**AVRDude**	Outil open-source de téléversement en ligne de commande pour programmer les µC AVR.	Gratuit	Oui

2.9.2 Installation de l'IDE Arduino

L'IDE Arduino est l'un des environnements les plus populaires pour développer sur µC AVR.

- **Téléchargement** : rendez-vous sur le site officiel : https://www.arduino.cc/en/software.
- **Installation** : suivez les instructions spécifiques à votre système d'exploitation (Windows, Linux ou macOS).
- **Configuration** :
  - Ajoutez votre carte Arduino dans l'interface de l'IDE.
  - Sélectionnez le bon modèle de carte et le port de communication.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Capture d'écran

Interface de l'IDE Arduino
avec le programme Blink chargé

Bouton « Téléverser » = compile + envoie le programme dans le µC en un clic

Figure 2.16 – L'IDE Arduino avec le programme Blink chargé. Le bouton « Téléverser » (flèche droite) compile et envoie le programme dans le µC en un seul clic.

2.9.3 Installation et utilisation d'AVRDude

AVRDude est un logiciel open-source qui permet de programmer les µC AVR en téléversant le code compilé.

- **Téléchargement** : AVRDude est disponible sur https://www.nongnu.org/avrdude/.
- **Installation** : suivez les instructions adaptées à votre OS.
- **Configuration** : assurez-vous que le µC est correctement connecté via un programmateur compatible (USBasp, Arduino, etc.).

Commande de test : vérifiez la connexion avec le µC :

```
avrdude -c usbasp -p m328p
```

Sortie attendue : si la connexion est correcte, vous devriez voir :

```
AVR device initialized and ready to accept instructions
```

2.9.4 Le « Hello World » de l'embarqué : faire clignoter une LED

Dans le monde de la programmation PC, le premier programme est traditionnellement « Hello World ». Dans le monde de l'embarqué, il n'y a pas d'écran. Le premier programme est donc **faire clignoter une LED** — le test le plus simple pour vérifier que toute la chaîne fonctionne.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Listing 2.1 – Le « Blink » : votre premier programme Arduino. Chaque ligne est commentée pour un débutant.

```
// setup() s'execute UNE SEULE FOIS au demarrage
void setup() {
    // Configurer la broche 13 comme SORTIE
    // (broche 13 = LED integree sur Arduino Uno)
    pinMode(13, OUTPUT);
}

// loop() s'execute EN BOUCLE, indefiniment
void loop() {
    // Mettre la broche 13 a l'etat HAUT (5V)
    // = la LED s'allume
    digitalWrite(13, HIGH);

    // Attendre 1000 millisecondes = 1 seconde
    delay(1000);

    // Mettre la broche 13 a l'etat BAS (0V)
    // = la LED s'eteint
    digitalWrite(13, LOW);

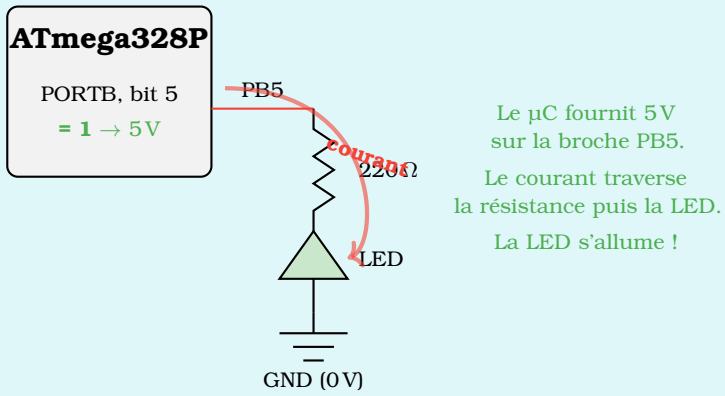
    // Attendre encore 1 seconde
    delay(1000);

    // Le programme revient automatiquement au debut
    // de loop() : la LED reclignote.
}
```

</> Ce qui se passe physiquement

Quand `digitalWrite(13, HIGH)` s'exécute :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge



La résistance de 220 Ω limite le courant à environ 20 mA pour ne pas griller la LED.

2.9.5 Programmation et déploiement

Une fois l'IDE Arduino et AVRDude installés et configurés, vous êtes prêt à programmer et déployer un code sur votre µC AVR :

1. Écrivez votre code dans l'IDE Arduino.
2. Compilez le programme (bouton « Vérifier »).
3. Téléversez-le sur la carte (bouton « Téléverser »).
4. Si vous utilisez AVRDude directement :

```
avrdude -c usbasp -p m328p -U flash:w:main.hex:i
```

Avec cette méthode, votre programme sera exécuté sur le µC et vous pourrez tester son fonctionnement.

Conseil pour démarrer sans matériel

1. Commencez avec **Tinkercad Circuits** (https://www.tinkercad.com/circuits) — simulation gratuite dans le navigateur.
2. Créez un circuit avec un Arduino Uno et une LED.
3. Copiez le programme Blink (listing 2.1).
4. Cliquez sur « Démarrer la simulation ».
5. Observez la LED clignoter !

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

C'est exactement ce que vous ferez en TP.

2.10 Conclusion du chapitre

Ce que vous avez appris

Ce chapitre vous a fait découvrir le microcontrôleur « de l'extérieur vers l'intérieur » :

1. **Ce que c'est :** un ordinateur miniature complet sur une seule puce (CPU + mémoire + E/S + périphériques).
2. **Où on le trouve :** partout ! Réveil, cafetière, voiture, badge, thermostat, drone, dispositifs médicaux, satellites. . .
3. **La différence avec un µP :** le µC intègre tout, le µP a besoin de composants externes. *Food truck vs restaurant.*
4. **La famille AVR :** architecture RISC, 131 instructions, idéale pour apprendre. L'ATmega328P est notre µC de référence.
5. **L'architecture interne :**
   - CPU (le chef) + ALU (le calculateur)
   - 32 registres (les tiroirs ultra-rapides)
   - Flash (le programme), SRAM (les données), EEPROM (les paramètres)
   - Périphériques (timers, UART, SPI, I²C, ADC, PWM. . .)
6. **Le cycle d'exécution :** Fetch → Decode → Execute → Write-back, 16 millions de fois par seconde.
7. **La compilation croisée :** on écrit sur PC, on compile pour AVR, on téléverse dans le µC. La chaîne complète : avr-gcc → avr-objcopy → avrdude.
8. **L'environnement :** Tinkercad pour simuler, Arduino IDE pour développer, AVRDude pour téléverser.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.11 Travaux dirigés du chapitre 1

Travaux dirigés : De la puce au système

5 exercices progressifs ancrés dans des situations réelles — architecture, mémoire, registres, périphériques, compilation croisée

TD 1 — Identification et choix d'un microcontrôleur

Contexte : Vous êtes ingénieur junior dans une entreprise de domotique. Votre responsable vous demande de choisir le microcontrôleur adapté pour trois projets différents et de justifier vos choix.

Situation 1 — Thermostat connecté

Votre entreprise développe un thermostat intelligent qui doit :

- Lire la température ambiante via un capteur analogique (LM35)
- Afficher la température sur un écran LCD 16×2 (interface I^2C)
- Communiquer avec un serveur via un module WiFi ESP-01 (interface UART)
- Piloter un relais pour activer/désactiver le chauffage
- Stocker la température de consigne même après coupure de courant
- Fonctionner 24h/24 avec une consommation minimale

a) Listez les périphériques internes du µC nécessaires pour chaque fonctionnalité ci-dessus. Présentez votre réponse sous forme de tableau (fonctionnalité → périphérique → justification).

b) Le projet nécessite-t-il un ATtiny85 ou un ATmega328P ? Justifiez en comparant le nombre de broches, la taille mémoire et les périphériques disponibles.

c) Quel type de mémoire (Flash, SRAM ou EEPROM) utiliseriez-vous pour stocker la température de consigne ? Pourquoi ?

d) Estimez le nombre minimum de broches d'E/S nécessaires. Le boîtier

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

DIP-28 de l'ATmega328P est-il suffisant ?

Situation 2 — Détecteur de mouvement miniature

Un client souhaite un petit capteur de mouvement autonome sur pile qui :

- Détecte un mouvement via un capteur PIR (signal numérique : 0 ou 1)
- Allume une LED pendant 30 secondes quand un mouvement est détecté
- Fonctionne pendant 6 mois sur une pile CR2032 (225 mA h)

a) Ce projet nécessite-t-il un ATmega328P ou un ATtiny85 suffit-il ? Justifiez.

b) Combien de broches d'E/S sont nécessaires au minimum ?

c) Si le µC consomme 1 mA en mode actif et 0,1 µA en mode veille, et que le capteur détecte en moyenne 10 mouvements par jour (LED allumée 30 s à chaque fois), calculez la durée de vie de la pile.

d) Quel périphérique interne utiliserez-vous pour mesurer les 30 secondes ?

Situation 3 — Microprocesseur ou microcontrôleur ?

Votre responsable vous soumet trois projets. Pour chacun, indiquez s'il faut un microprocesseur ou un microcontrôleur, et justifiez votre choix.

a) **Projet A :** Un serveur de base de données qui traite 10 000 requêtes SQL par seconde et stocke 500 Go de données.

b) **Projet B :** Un système d'arrosage automatique qui lit un capteur d'humidité, vérifie l'heure et ouvre une électrovanne si le sol est sec.

c) **Projet C :** Un système de reconnaissance faciale en temps réel pour le contrôle d'accès d'un immeuble (caméra HD, réseau de neurones).

d) Pour le projet B, dessinez un schéma bloc simplifié montrant le µC au centre avec ses entrées (capteurs) et sorties (actionneurs).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

TD 2 — Exploration de l'architecture interne de l'ATmega328P

Contexte : Vous préparez un TP sur l'ATmega328P. Avant de programmer, vous devez comprendre précisément comment les composants internes interagissent.

Situation 1 — Carte d'identité de l'ATmega328P

a) Complétez le tableau suivant de mémoire (sans regarder le cours) :

Caractéristique	Flash	SRAM	EEPROM
Taille	_____	_____	_____
Volatile ?	_____	_____	_____
Contient	_____	_____	_____
Analogie	_____	_____	_____

b) L'ATmega328P possède 32 Ko de Flash. Sachant que chaque instruction assembleur AVR occupe 2 octets (1 mot de 16 bits), combien d'instructions peut contenir la Flash au maximum ?

c) Un programme de contrôle de feux tricolores occupe 1 200 instructions. Quel pourcentage de la Flash utilise-t-il ?

d) La SRAM fait 2 Ko. Si votre programme utilise 50 variables de type `uint8_t` (1 octet chacune) et 20 variables de type `uint16_t` (2 octets chacune), combien d'octets de SRAM sont occupés par ces variables ? Quel pourcentage de la SRAM totale ?

Situation 2 — Les registres en pratique

a) Pourquoi l'instruction `LDI R5, 42` provoque-t-elle une erreur d'assemblage, alors que `LDI R16, 42` fonctionne ? Expliquez la raison technique (limitation d'encodage).

b) Les registres R26-R27 forment le pointeur X. Si R27 contient `0x01` et R26 contient `0xF0`, quelle adresse en SRAM le pointeur X désigne-t-il ? Montrez le calcul.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

c) Après l'exécution de `ADD R16, R17` avec R16 = 200 et R17 = 100 (en non signé sur 8 bits), quelle valeur contient R16 ? Quels drapeaux du registre SREG sont affectés ? (Calculez en binaire.)

d) Le registre SREG contient la valeur `0b00000010` après une opération. Quels drapeaux sont actifs ? Qu'est-ce que cela signifie sur le résultat de l'opération ?

Situation 3 — Architecture Harvard vs Von Neumann

a) Expliquez avec vos propres mots la différence entre l'architecture Harvard et l'architecture Von Neumann. Utilisez un schéma pour chacune.

b) Pourquoi l'architecture Harvard permet-elle une exécution plus rapide des instructions ? Illustrez avec l'exemple du pipeline.

c) Un PC utilise l'architecture Von Neumann. Citez un avantage de cette architecture par rapport à Harvard.

d) L'AVR utilise une architecture Harvard « modifiée ». Qu'est-ce qui la rend « modifiée » par rapport à une Harvard pure ? *(Indice : pensez à l'instruction `LPM` qui lit des données depuis la Flash.)*

TD 3 — Périphériques et interfaces de l'ATmega328P

Contexte : Vous concevez un système embarqué et devez comprendre comment les périphériques internes du µC interagissent avec le monde extérieur.

Situation 1 — Conversion analogique-numérique (ADC)

Un capteur de luminosité (LDR + résistance en pont diviseur) est connecté à la broche A0 de l'Arduino. La tension varie entre 0 V (obscurité totale) et 5 V (plein soleil).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

a) L'ADC de l'ATmega328P a une résolution de 10 bits. Combien de valeurs différentes peut-il produire ?

b) Quelle est la résolution en tension ? (c'est-à-dire : quelle variation minimale de tension l'ADC peut-il détecter ?)

c) Le capteur donne une tension de 3,2 V. Quelle valeur numérique l'ADC renvoie-t-il ?

d) Vous voulez déclencher une alarme quand la luminosité descend en dessous de 20% de la valeur maximale. Quel seuil numérique devez-vous programmer ?

e) L'ADC effectue une conversion en 13 cycles d'horloge ADC. Si le prescaler est configuré pour diviser l'horloge système (16 MHz) par 128, combien de conversions par seconde peut-on réaliser ?

Situation 2 — Timers et génération de signaux

Vous devez faire clignoter une LED à une fréquence précise de 2 Hz (allumée 250 ms, éteinte 250 ms) en utilisant un timer.

a) L'horloge système est à 16 MHz. Si le Timer0 (8 bits) compte de 0 à 255 sans prescaler, combien de temps met-il pour déborder (*overflow*) ?

b) Avec un prescaler de 1024, combien de temps met le Timer0 pour déborder ?

c) Pour obtenir un délai de 250 ms avec le Timer0 et un prescaler de 1024, combien de débordements faut-il compter ?

d) Le Timer1 est un timer 16 bits. Quelle est sa valeur maximale ? Avec un prescaler de 256, combien de temps met-il pour déborder ?

e) Expliquez la différence entre le mode « Normal » et le mode « CTC » (*Clear Timer on Compare Match*) d'un timer.

Situation 3 — Communication série (UART)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Vous connectez l'Arduino à un PC via le câble USB pour afficher des mesures sur un terminal série.

a) Le protocole UART transmet les données bit par bit. Si le débit est configuré à 9600 bauds, combien de temps dure la transmission d'un bit ?

b) Une trame UART standard contient : 1 bit de start + 8 bits de données + 1 bit de stop = 10 bits. Combien de temps faut-il pour transmettre un octet complet ?

c) Combien d'octets par seconde peut-on transmettre à 9600 bauds ?

d) Vous voulez envoyer le texte `"Temp=25.3C"` (10 caractères). Combien de temps prend cette transmission à 9600 bauds ?

e) Pourquoi utilise-t-on souvent 115200 bauds au lieu de 9600 dans les projets modernes ? Quel est le facteur d'accélération ?

TD 4 — Cycle d'exécution et calculs de performance

Contexte : Pour écrire du code assembleur efficace, vous devez comprendre combien de temps prend chaque instruction et comment optimiser vos programmes.

Situation 1 — Timing des instructions

L'ATmega328P fonctionne à 16 MHz. La plupart des instructions AVR s'exécutent en 1 cycle, mais certaines en prennent 2 ou 3.

a) Calculez la durée d'un cycle d'horloge en nanosecondes.

b) L'instruction `LDI R16, 0xFF` prend 1 cycle. Combien de temps dure-t-elle ?

c) L'instruction `RJMP` (saut relatif) prend 2 cycles. Combien de temps dure-t-elle ?

d) Un programme contient la séquence suivante. Calculez le temps total

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

d'exécution :

Instruction	Cycles	Description
`LDI R16, 5`	1	Charger 5 dans R16
`LDI R17, 3`	1	Charger 3 dans R17
`ADD R16, R17`	1	R16 = R16 + R17
`STS 0x0100, R16`	2	Stocker R16 en SRAM
`RJMP debut`	2	Retourner au début

e) Combien de fois par seconde cette séquence peut-elle s'exécuter ?

Situation 2 — Le cycle Fetch-Decode-Execute en détail

a) Décrivez les 4 phases du cycle d'exécution pour l'instruction `ADD R16, R17`. Pour chaque phase, précisez ce qui se passe dans le CPU et quels composants sont impliqués (PC, IR, ALU, registres, SREG).

b) L'AVR utilise un pipeline à 2 étages : pendant qu'une instruction s'exécute, la suivante est déjà en cours de lecture (*fetch*). Dessinez un chronogramme montrant l'exécution de 4 instructions successives avec et sans pipeline. Combien de cycles gagne-t-on ?

c) Pourquoi les instructions de branchement (`RJMP`, `BREQ`) prennent-elles 2 cycles au lieu de 1 ? *(Indice : pensez au pipeline.)*

Situation 3 — Comparaison avec d'autres architectures

a) Un processeur ARM Cortex-M4 fonctionne à 168 MHz. Combien de fois est-il plus rapide que l'ATmega328P en termes de cycles par seconde ?

b) Malgré cette différence, expliquez pourquoi un ATmega328P est préférable pour un thermostat.

c) Un Arduino Mega utilise un ATmega2560 à 16 MHz avec 256 Ko de Flash et 8 Ko de SRAM. Comparez avec l'ATmega328P en termes de capacité mémoire (facteur multiplicatif).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

d) Calculez combien d'instructions assembleur peut contenir la Flash de l'ATmega2560.

>_ TD 5 — Compilation croisée et mise en pratique

Contexte : Vous devez comprendre la chaîne de compilation croisée pour AVR et être capable de compiler, convertir et téléverser un programme manuellement.

Situation 1 — Comprendre la chaîne de compilation

a) Remettez dans l'ordre les étapes de la compilation croisée (elles sont mélangées) :

Lettre	Étape
E	Téléversement avec `avrdude`
B	Compilation C → assembleur
D	Conversion ELF → HEX avec `objcopy`
A	Prétraitement (`#include`, `#define`)
F	Édition de liens (linker)
C	Assemblage → fichier objet

b) Pour chaque étape, indiquez le format du fichier d'entrée et de sortie (ex. : `.c` → `.i`).

c) Expliquez pourquoi on ne peut pas directement téléverser un fichier `.elf` dans le µC. Quel est le rôle de la conversion en `.hex` ?

d) Quelle commande unique permet d'effectuer les étapes A à F en une seule ligne ? Écrivez-la avec toutes les options nécessaires pour un ATmega328P à 16 MHz.

Situation 2 — Analyse des options de compilation

Considérez la commande suivante :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

```
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os \
    -o thermostat.elf thermostat.c
```

a) Expliquez le rôle de chaque option :

- `-mmcu=atmega328p`
- `-DF_CPU=16000000UL`
- `-Os`
- `-o thermostat.elf`

b) Que se passerait-il si vous oubliiez l'option `-mmcu` ?

c) Quelle est la différence entre `-Os` (optimisation taille) et `-O2` (optimisation vitesse) ? Pourquoi `-Os` est-il préféré sur µC ?

d) Écrivez la commande `avr-objcopy` pour convertir `thermostat.elf` en `thermostat.hex`. Expliquez chaque option.

e) Écrivez la commande `avrdude` pour téléverser `thermostat.hex` sur un ATmega328P via un Arduino connecté sur `COM3` (Windows) ou `/dev/ttyUSB0` (Linux).

Situation 3 — Premier programme : analyse du Blink

Considérez le programme Blink suivant :

```
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    DDRB |= (1 << PB5);     // ligne A
    while (1) {
        PORTB |= (1 << PB5);    // ligne B
        _delay_ms(500);
        PORTB &= ~(1 << PB5);   // ligne C
        _delay_ms(500);
    }
    return 0;
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

```
13 }
```

a) Que fait la ligne A ? Expliquez l'opération bit à bit `|= (1 « PB5)` en détail. Quel est le rôle du registre `DDRB` ?

b) Que fait la ligne B ? Expliquez la différence avec la ligne A.

c) Que fait la ligne C ? Expliquez l'opération `&= ~(1 « PB5)`. Pourquoi utilise-t-on un AND avec un masque inversé ?

d) Ce programme utilise-t-il la fonction Arduino `digitalWrite()` ? Pourquoi cette version est-elle plus « bas niveau » ?

e) Si on change `_delay_ms(500)` par `_delay_ms(100)`, à quelle fréquence la LED clignote-t-elle ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

2.12 Quiz du chapitre 1 : testez vos connaissances

Vrai/Faux QCM Pratique

Répondez sans regarder le cours. Temps estimé : 15 minutes.

1 QCM

Quelle est la différence fondamentale entre un microprocesseur et un microcontrôleur ?

A. Le µP est plus rapide que le µC

B. Le µC intègre CPU + mémoire + E/S sur une seule puce, le µP ne contient que le CPU

C. Le µC est plus cher que le µP

D. Le µP ne peut pas exécuter de programmes

2 VRAI/FAUX

La mémoire SRAM de l'ATmega328P conserve son contenu même après une coupure de courant.

☐ **Vrai** ☐ **Faux**

Indice : pensez à l'analogie du bloc-notes vs du coffre-fort.

3 QCM

L'ATmega328P fonctionne à 16 MHz. Combien de temps dure un cycle d'horloge ?

A. 62,5 ms

B. 62,5 µs

C. 62,5 ns

D. 16 ns

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

4 VRAI/FAUX

L'instruction `LDI R5, 100` est valide en assembleur AVR.

☐ **Vrai** ☐ **Faux**

Indice : quels registres acceptent l'instruction LDI ?

5 QCM

Quel outil convertit le fichier `.elf` en fichier `.hex` dans la chaîne de compilation AVR ?

A. `avr-gcc`

B. `avrdude`

C. `avr-objcopy`

D. `avr-as`

6 VRAI/FAUX

L'architecture Harvard utilise un seul bus partagé pour les instructions et les données.

☐ **Vrai** ☐ **Faux**

Indice : c'est l'architecture Von Neumann qui fait cela.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

7 PRATIQUE

Associez chaque composant à sa fonction. Reliez chaque élément de la colonne gauche à la colonne droite :

A. CPU	**1.** Stocke le programme (non volatile)
B. ALU	**2.** Variables temporaires (volatile)
C. Flash	**3.** Effectue les calculs arithmétiques et logiques
D. SRAM	**4.** Chef d'orchestre, coordonne l'exécution
E. EEPROM	**5.** Paramètres persistants (non volatile)

Réponses : A=_____ B=_____ C=_____ D=_____ E=_____

8 QCM

Combien de registres de travail généraux possède l'AVR ?

A. 8 registres de 16 bits

B. 16 registres de 8 bits

C. 32 registres de 8 bits

D. 64 registres de 8 bits

9 PRATIQUE

Complétez le cycle Fetch-Decode-Execute pour l'instruction `LDI R16, 42` :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

9

1. FETCH : Le CPU lit l'instruction à l'adresse _____ (pointée par le _____)

2. DECODE : Le CPU reconnaît l'instruction _____ et identifie le registre _____ et la valeur _____

3. EXECUTE : La valeur _____ est chargée dans le registre _____

4. UPDATE PC : Le compteur de programme passe à _____

10 PRATIQUE

Défi compilation : Écrivez les 3 commandes nécessaires pour compiler un fichier `alarm.c` pour un ATmega328P et le téléverser via Arduino sur le port `COM5` :

Commande 1 — Compiler :

`avr-gcc` _____

Commande 2 — Convertir :

`avr-objcopy` _____

Commande 3 — Téléverser :

`avrdude` _____

Bonus : quelle commande unique de l'IDE Arduino remplace ces 3 commandes ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Récapitulatif : Ce que vous devez maîtriser

Architecture — Concepts

- µC = CPU + mémoire + E/S sur 1 puce
- µP ≠ µC : intégration, pas puissance
- RISC : 131 instructions, 1 cycle chacune
- Harvard : 2 bus séparés (instructions/données)
- Flash = programme, SRAM = variables, EEPROM = config

Composants internes

- CPU : chef d'orchestre (Fetch-Decode-Execute)
- ALU : calculs arithmétiques et logiques
- 32 registres R0–R31 (R16+ = usage complet)
- SREG : drapeaux Z, C, N, V, S, H
- Pointeurs X, Y, Z pour adressage indirect

Périphériques & interfaces

- ADC 10 bits : convertir analogique → numérique
- Timers (0, 1, 2) : mesure du temps, PWM
- UART : communication série avec PC
- SPI, I^2C : connexion de capteurs
- GPIO : entrées/sorties numériques

Compilation croisée

- `avr-gcc` : C → ELF (compile)
- `avr-objcopy` : ELF → HEX (convertit)
- `avrdude` : HEX → Flash du µC (téléverse)
- IDE Arduino = fait tout en 1 clic
- Blink = le « Hello World » de l'embarqué

Compétences clés pour la suite du cours :

Dessiner l'architecture AVR de mémoire • Expliquer le rôle de chaque composant • Distinguer Flash/SRAM/EEPROM Calculer des temps d'exécution • Compiler et téléverser un programme • Lire du code C bas niveau

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Fin des exercices — Chapitre 1

Vous avez pratiqué les concepts fondamentaux :

Architecture • Mémoires • Registres • Périphériques • Compilation croisée

Chapitre suivant : Fondements du langage assembleur AVR

Vous allez écrire vos premières instructions : `LDI`, `MOV`, `ADD`, `SUB`...

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Fiche de révision express

Concept	**Ce qu'il faut retenir**
Microcontrôleur	Ordinateur complet sur 1 puce : CPU + mémoire + E/S
µP vs µC	µP = cerveau seul (restaurant). µC = tout intégré (food truck).
Architecture	Harvard (2 bus séparés) + RISC (instructions simples et rapides)
Registres	32 tiroirs R0–R31, 8 bits chacun. R16–R31 = usage complet.
Flash	32 Ko, non volatile, contient le programme.
SRAM	2 Ko, volatile, contient les variables.
EEPROM	1 Ko, non volatile, contient les paramètres.
Horloge	16 MHz = 16 M instructions/s = 1 instruction en 62,5 ns
Cycle d'instruction	Fetch → Decode → Execute → Write-back
Compilation croisée	avr-gcc → avr-objcopy → avrdude
Premier programme	Blink = faire clignoter une LED = le « Hello World » de l'embarqué

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Fin du Chapitre 2 — Introduction aux microcontrôleurs AVR

Vous connaissez maintenant la **machine** qui exécutera vos programmes assembleur.

Chapitre suivant : ***Fondements du langage assembleur AVR***

Vous allez écrire vos premières instructions : `LDI`, `MOV`, `ADD`...
et voir concrètement comment le µC les exécute, registre par registre.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

CHAPITRE III

Fondements du langage assembleur

</> Chapitre 3
Fondements du langage assembleur

Du langage machine aux premières instructions AVR

4 heures | Théorie et pratique | Assembleur AVR & x64 | Premiers programmes

Mise en situation : parler la langue de la machine

Au chapitre précédent, vous avez découvert le **microcontrôleur** — la machine qui exécutera vos programmes. Vous connaissez ses composants internes : CPU, ALU, registres, mémoires, périphériques.
Maintenant, une question se pose : *comment lui donner des ordres ?*

Imaginez que vous êtes devant une personne qui ne comprend qu'une seule langue — une langue très simple, avec seulement 131 mots. Chaque mot correspond à **une action précise** :

- `LDI` = « charge cette valeur dans ce tiroir »
- `MOV` = « copie le contenu d'un tiroir dans un autre »
- `ADD` = « additionne le contenu de deux tiroirs »

- `LDS` = « va chercher une valeur en mémoire »
- `STS` = « range une valeur en mémoire »
- `BRNE` = « si le résultat n'est pas zéro, recommence »

Cette langue, c'est le langage assembleur.

C'est le niveau le plus bas auquel un humain peut programmer une machine, juste au-dessus des 0 et des 1 du langage machine.

◎ Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

1. **Expliquer** comment le langage machine (binaire) est traduit en mnémoniques assembleur.
2. **Décrire** la structure complète d'un programme assembleur AVR : directives, sections, étiquettes, commentaires.
3. **Identifier et utiliser** les registres de travail (R0–R31) et les registres spéciaux (SREG, SP, PC).
4. **Expliquer** le rôle de chaque drapeau du registre SREG (Z, C, N, V, S, H, T, I).
5. **Maîtriser** les instructions de transfert : `LDI`, `MOV`, `LDS`, `STS`.
6. **Analyser** un programme assembleur ligne par ligne, en traçant l'état des registres.
7. **Comparer** la syntaxe et les registres des architectures AVR et x64.
8. **Écrire et exécuter** un programme simple effectuant une addition.

🔑 Prérequis

Ce chapitre suppose acquis les notions des chapitres 0 et 1 :

✔ Numération binaire et hexadécimale — *vous savez que* `0xFF` = 255 = 11111111_2

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- ✔ Opérations logiques (AND, OR, XOR) et masquage de bits
- ✔ Architecture d'un µC : CPU, ALU, 32 registres, Flash, SRAM, EEPROM
- ✔ Cycle Fetch-Decode-Execute
- ✔ Compilation croisée (avr-gcc → avr-objcopy → avrdude)

Si ce n'est pas le cas, relisez les chapitres 0 et 1.

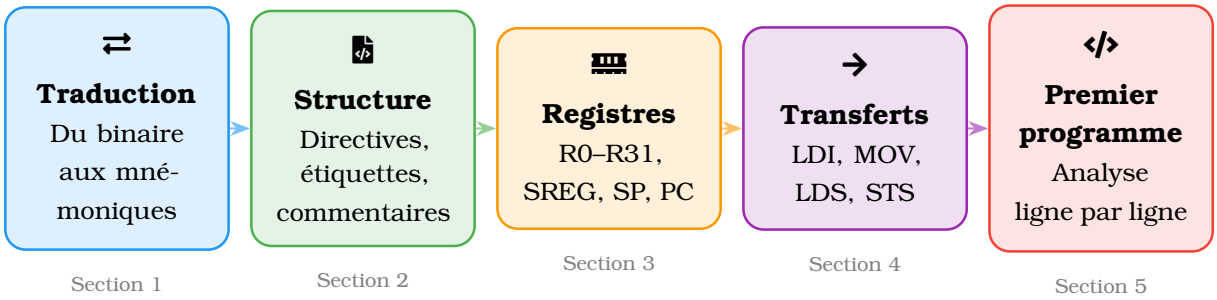


Figure 3.1 – Organisation du chapitre : du langage machine aux premiers programmes assembleur AVR.

3.1 Du langage machine à l'assembleur : traduction et mnémoniques

3.1.1 Le problème du langage machine

Programmer en binaire, c'est comme écrire avec des codes postaux

Imaginez que pour écrire une lettre, au lieu d'utiliser des mots, vous devez écrire uniquement des codes postaux. Pour dire « Bonjour », vous écrivez `75001 69002 13001`. C'est techniquement possible, mais :

- Personne ne peut lire votre lettre sans un dictionnaire de codes.
- La moindre erreur d'un chiffre change complètement le sens.
- Écrire une page prend des heures.

C'est exactement ce que vivaient les premiers programmeurs dans les années 1940 : ils devaient écrire chaque instruction en **suites de 0 et de**

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1.

</> Le même programme en binaire et en assembleur

Voici l'instruction « charger la valeur 42 dans le registre R16 » dans les deux formes :

Langage machine	Assembleur
`1110 1010 0000 1010`	`LDI R16, 42`
16 bits = 1 mot d'instruction	3 éléments lisibles
Incompréhensible pour un humain	Immédiatement compréhensible

Correspondance 1 à 1

L'assembleur traduit

Point essentiel : les deux formes représentent **exactement la même instruction**. L'assembleur ne fait que remplacer les suites de bits par des noms lisibles (**mnémoniques**).

3.1.2 Qu'est-ce qu'une mnémonique ?

Mnémonique

Une **mnémonique** (du grec *mnêmonikos* = « qui aide à se souvenir ») est un **nom court et lisible** qui représente une instruction machine.

Exemples :

- `LDI` = *Load Immediate* = charger une valeur directe
- `MOV` = *Move* = copier un registre
- `ADD` = *Add* = additionner
- `SUB` = *Subtract* = soustraire
- `LDS` = *Load from Data Space* = charger depuis la SRAM
- `STS` = *Store to Data Space* = stocker en SRAM

Chaque mnémonique correspond à **exactement un opcode** (code binaire d'opération).

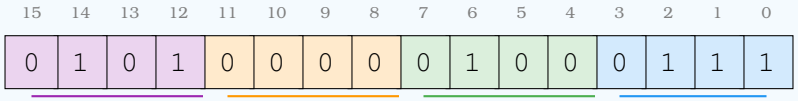
 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.1.3 Comment le processeur comprend-il une instruction ?

Anatomie d'une instruction AVR (16 bits)

Chaque instruction AVR est encodée sur **16 bits** (2 octets = 1 mot). Ces 16 bits sont découpés en champs qui indiquent au processeur **quoi faire** et **avec quoi**.

Exemple : `LDI R16, 42`



1110 = opcode de LDI **0010** = bits hauts de 42
0000 = R16 (encodé comme 0) **1010** = bits bas de 42

Décodage :

- **Opcode** `1110` → le CPU reconnaît : « c'est une instruction LDI »
- **Registre** `0000` → registre 0 + 16 = **R16** (seuls R16–R31 sont possibles, encodés sur 4 bits comme 0–15)
- **Valeur K** = `0010 1010` = $2 \times 16 + 10 = 42$

Pourquoi comprendre l'encodage ?

Vous n'aurez **jamais** à écrire du binaire directement. Mais comprendre l'encodage vous aide à :

- Comprendre **pourquoi** `LDI R5, 42` est impossible (le champ registre n'a que 4 bits)
- Comprendre **pourquoi** la valeur K est limitée à 0–255 (8 bits disponibles)
- Lire un **dump hexadécimal** de la Flash lors du débogage

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.1.4 Le processus de traduction

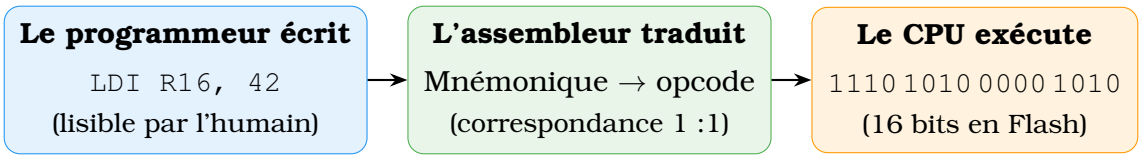


Figure 3.2 – Le processus de traduction : du texte lisible au binaire exécutable.

Table 3.1 – Exemples de correspondance mnémonique ↔ binaire pour les instructions AVR.

Mnémonique	Opcode (4 bits)	Binaire complet	Signification
`LDI R16, 5`	`1110`	`1110 0000 0000 0101`	Charger 5 dans R16
`MOV R17, R16`	`0010 11`	`0010 1100 0001 0000`	Copier R16 dans R17
`ADD R16, R17`	`0000 11`	`0000 1100 0000 0001`	R16 = R16 + R17
`SUB R16, R17`	`0001 10`	`0001 1000 0000 0001`	R16 = R16 − R17
`RJMP -1`	`1100`	`1100 1111 1111 1111`	Sauter en boucle

3.2 Structure d'un programme assembleur AVR

Un programme assembleur, c'est comme une recette de cuisine bien organisée

Une recette a toujours la même structure :

1. **En-tête** : nom du plat, temps de préparation, nombre de personnes
   (= **directives** : quel µC, quels alias)
2. **Liste des ingrédients** : 200g de farine, 3 œufs...
   (= **constantes et données** : `.equ`, `.db`)
3. **Étapes numérotées** : 1. Mélanger... 2. Cuire...
   (= **instructions** avec **étiquettes** : `main:`, `loop:`)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

4. **Notes** : « servir chaud »
   (= **commentaires** : `; ceci est un commentaire`)

3.2.1 Les 6 éléments d'un programme AVR

Anatomie complète d'un programme assembleur AVR

1. Directives
`.include`
Fichier de définition du µC
+ alias `.def`

2. Constantes
`.equ F_CPU = 16000000`
Valeurs nommées

3. Vecteur reset
`.org 0x0000`
Première instruction au démarrage

4. Étiquettes
`main:, loop:`
Noms d'adresses pour les sauts

</> 5. Instructions
`LDI, MOV, ADD`
Le code exécutable
= les ordres au CPU

6. Commentaires
`; texte explicatif`
Ignorés par l'assembleur
Essentiels pour l'humain

3.2.2 Directives de préprocesseur

Directive

Une **directive** est une instruction destinée à l'**assembleur** (l'outil), **pas au processeur**. Elle n'est pas traduite en code machine — elle configure l'environnement d'assemblage.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Table 3.2 – Principales directives assembleur AVR.

Directive	Syntaxe	Rôle et exemple
`.include`	`.include "m328Pdef.inc"`	Inclut le fichier de définition du µC (noms des registres, adresses des ports)
`.def`	`.def temp = r16`	Crée un alias lisible pour un registre (`temp` au lieu de `r16`)
`.equ`	`.equ F_CPU = 16000000`	Définit une constante numérique nommée
`.org`	`.org 0x0000`	Place le code à une adresse précise en Flash
`.db`	`.db 0x3F, 0x06, 0x5B`	Définit des octets de données en Flash
`.dw`	`.dw 1234`	Définit des mots de 16 bits en Flash
`.byte`	`.byte 1`	Réserve des octets en SRAM (variable non initialisée)

3.2.3 Étiquettes (labels)

Étiquette

Une **étiquette** (*label*) est un **nom symbolique** qui représente une adresse en mémoire Flash. Elle permet de faire référence à un endroit du programme sans connaître son adresse numérique.

Syntaxe : le nom suivi de deux-points :

```
main:   loop:   fin:   ISR_TIMER0:
```

L'assembleur remplace automatiquement chaque étiquette par son adresse réelle lors de la traduction.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.2.4 Commentaires

Commentaire

Un **commentaire** est un texte ignoré par l'assembleur, destiné uniquement à l'humain qui lit le code. En assembleur AVR, il commence par un point-virgule `;`.

Le code assembleur sans commentaires est illisible !

En assembleur, chaque instruction fait **une seule chose**. Un programme de 50 lignes peut être totalement incompréhensible sans commentaires. La règle d'or :

Commentez CHAQUE ligne en assembleur.

Mauvais :

```
ldi r16, 0x05
ldi r17, 0x03
add r16, r17
```

Bon :

```
ldi r16, 0x05     ; Charger la premiere valeur (5)
ldi r17, 0x03     ; Charger la deuxieme valeur (3)
add r16, r17      ; Additionner : R16 = 5 + 3 = 8
```

3.2.5 Programme complet annoté

Listing 3.1 – Programme assembleur AVR complet et annoté : addition de 5 + 3.

```
; ============================================
; Programme : Addition de deux nombres
; Cible     : ATmega328P (Arduino Uno)
; Auteur    : Cours B2 IT
; Date      : 2025
; ============================================

; --- 1. DIRECTIVES D'EN-TETE ---
.include "m328Pdef.inc"    ; Definitions registres ATmega328P
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

```
.def temp    = r16          ; Alias : R16 = "temp"
.def result  = r17          ; Alias : R17 = "result"

; --- 2. CONSTANTES ---
.equ VALEUR_A = 5           ; Premiere valeur a additionner
.equ VALEUR_B = 3           ; Deuxieme valeur

; --- 3. VECTEUR DE RESET ---
.org 0x0000                 ; Adresse 0 = premiere instruction
    rjmp main               ; Sauter au programme principal

; --- 4. PROGRAMME PRINCIPAL ---
main:                       ; Etiquette du debut
    ldi temp, VALEUR_A      ; Charger 5 dans R16 (temp)
    ldi result, VALEUR_B    ; Charger 3 dans R17 (result)
    add result, temp        ; R17 = R17 + R16 = 3 + 5 = 8
    ; A ce stade, result (R17) contient 8

; --- 5. BOUCLE INFINIE ---
endless:                    ; Etiquette de fin
    rjmp endless            ; Boucler indefiniment
    ; Le programme ne doit JAMAIS se terminer
    ; (pas d'OS pour reprendre la main)
```

Pourquoi `.org 0x0000` + `rjmp main` ?

Quand le µC démarre (ou après un reset), le compteur de programme (PC) est mis à **0x0000**. C'est donc la première adresse exécutée.
Mais à l'adresse 0x0000, on place habituellement les **vecteurs d'interruption** (pas le code principal). Le `rjmp main` saute par-dessus ces vecteurs pour atteindre le vrai début du programme.
Pour un programme simple sans interruptions, vous pouvez aussi écrire le code directement à partir de 0x0000, mais la convention `rjmp main` est une bonne pratique.

3.3 Registres de travail et registres spéciaux

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.3.1 Les 32 registres de travail (R0–R31)

Vous connaissez déjà les 32 registres depuis le chapitre 1. Approfondissons maintenant leur utilisation **en pratique dans un programme assembleur**.

Classification pratique des registres AVR

R0–R15
Usage restreint
Pas de `LDI`
Pas de `ANDI`, `ORI`
Pas de `CPI`
Utilisables avec `MOV`, `ADD`, `SUB`

R16–R25
Usage complet
Toutes les instructions acceptées
Les registres les plus utilisés en pratique

R26–R31
Usage complet + pointeurs
X = R27 :R26
Y = R29 :R28
Z = R31 :R30
Adressage indirect en SRAM

3.3.2 Le registre SREG : les 8 drapeaux expliqués

Registre de statut SREG

Le **SREG** (*Status Register*) est un registre de 8 bits mis à jour **automatiquement** après chaque opération arithmétique ou logique de l'ALU. Chaque bit est un **drapeau** (*flag*) qui donne une information sur le résultat.

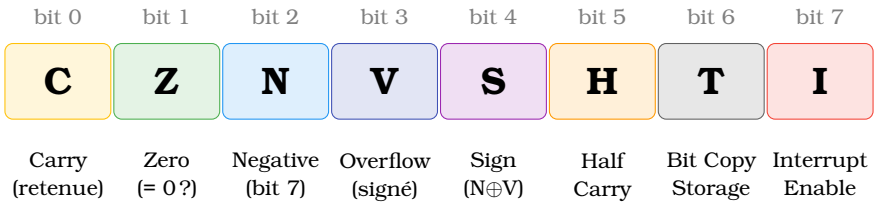


Figure 3.3 – Les 8 drapeaux du registre SREG de l'ATmega328P.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Les 4 drapeaux les plus utilisés

Flag	Nom	Activé si...	Exemple concret
Z	Zero	Le résultat = 0	`SUB R16, R16` → R16 = 0, Z = **1**
C	Carry	Retenue hors du bit 7 (non signé)	`ADD` avec 200 + 100 = 300 > 255 → C = **1**, résultat = 44
N	Negative	Le bit 7 du résultat = 1	Résultat = `10000001` → N = **1** (négatif en signé)
V	oVerflow	Dépassement en arithmétique signée	100 + 50 = 150 > 127 → V = **1** (erreur en signé)

</> SREG en action : 250 + 10

```
  11111010   (250 = R16)
+ 00001010   (10 = R17)
  1 00000100   (= 260)
```

← 9e bit = **Carry**

Résultat dans R16 : $260 - 256 = 4$ (seuls 8 bits conservés)
Drapeaux SREG :

- Z = 0 (résultat $\neq 0$)
- C = **1** (retenue : le résultat dépasse 255)
- N = 0 (bit 7 du résultat = 0)
- V = 0 (pas de dépassement en signé, les deux opérandes sont < 128)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.3.3 Le pointeur de pile (SP) et le compteur de programme (PC)

Table 3.3 – Les registres spéciaux du processeur AVR.

Registre	Taille	Rôle
PC (Program Counter)	14 bits	Contient l'adresse de la **prochaine instruction** à exécuter. Incrémenté automatiquement après chaque instruction. Modifié par les sauts (`RJMP`, `RCALL`).
SP (Stack Pointer)	16 bits	Pointe vers le **sommet de la pile** en SRAM. Utilisé automatiquement par `PUSH`/`POP`, `RCALL`/`RET` et les interruptions. Doit être initialisé en début de programme.
SREG	8 bits	Registre de statut contenant les 8 drapeaux (I, T, H, S, V, N, Z, C). Mis à jour automatiquement par l'ALU.

3.4 Instructions de transfert : LDI, MOV, LDS, STS

Les instructions de transfert sont les plus fondamentales : elles permettent de **déplacer des données** entre les registres et la mémoire. Sans elles, aucun calcul n'est possible — il faut d'abord **charger** les valeurs avant de pouvoir les traiter.

Les instructions de transfert, c'est comme déplacer des ingrédients

Avant de cuisiner (= calculer), il faut :

- **LDI** = prendre un ingrédient du placard et le poser sur le plan de travail (charger une valeur directe dans un registre)
- **MOV** = déplacer un ingrédient d'un récipient à un autre (copier un registre)
- **LDS** = aller chercher un ingrédient dans le frigo (charger depuis la SRAM)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- **STS** = ranger un ingrédient au frigo pour plus tard (stocker en SRAM)

3.4.1 `LDI` — Load Immediate (charger une valeur directe)

Instruction LDI

`LDI Rd, K` — Charge la valeur immédiate K dans le registre Rd.

- **Rd** : registre destination (**R16 à R31 uniquement**)
- **K** : valeur constante de 0 à 255 (8 bits)
- **Cycles** : 1
- **SREG** : non modifié

</> Exemples d'utilisation de LDI

```
ldi r16, 42          ; R16 = 42 (decimal)
ldi r17, 0xFF        ; R17 = 255 (hexadecimal)
ldi r18, 0b10101010; R18 = 170 (binaire)
ldi r19, 'A'         ; R19 = 65 (code ASCII de 'A')
```

Erreurs courantes :

```
ldi r5, 42           ; ERREUR : R5 < R16, LDI impossible
ldi r16, 256         ; ERREUR : 256 > 255, depasse 8 bits
ldi r16, -1          ; OK si assemble comme 0xFF (255 non signe)
```

3.4.2 `MOV` — Move (copier un registre)

Instruction MOV

`MOV Rd, Rr` — Copie le contenu du registre Rr dans le registre Rd.

- **Rd** : registre destination (R0 à R31)
- **Rr** : registre source (R0 à R31)
- **Cycles** : 1
- **SREG** : non modifié

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

- **Note** : Rr n'est **pas modifié**, c'est une **copie**

</> MOV pour contourner la limitation de LDI

Comment charger la valeur 42 dans R5 (qui n'accepte pas LDI) ?

```
ldi r16, 42          ; D'abord charger dans R16 (OK car R16 >= R16)
mov r5, r16          ; Puis copier R16 dans R5 (MOV fonctionne sur
   tous)
; Maintenant R5 = 42 et R16 = 42 (les deux contiennent la valeur)
```

Attention : MOV ne « déplace » pas — il **copie**. Après `MOV R5, R16`, les deux registres contiennent 42.

3.4.3 LDS — Load from Data Space (charger depuis la SRAM)

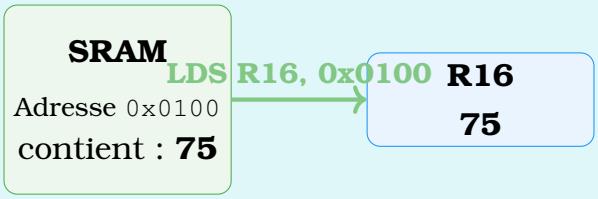
Instruction LDS

`LDS Rd, k` — Charge dans Rd la valeur stockée à l'adresse k en SRAM.

- **Rd** : registre destination (R0 à R31)
- **k** : adresse en SRAM (16 bits, de 0x0000 à 0xFFFF)
- **Cycles** : 2 (accès mémoire = plus lent)
- **SREG** : non modifié
- **Taille** : 32 bits (2 mots en Flash — double de la taille normale !)

</> Lire une valeur en SRAM

```
; Supposons que l'adresse SRAM 0x0100 contient la valeur 75
lds r16, 0x0100     ; R16 = contenu de l'adresse 0x0100 = 75
```



 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.4.4 `STS` — Store to Data Space (stocker en SRAM)

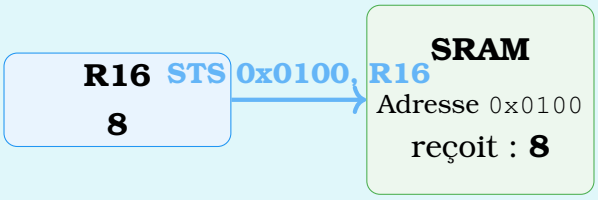
Instruction STS

`STS k, Rr` — Stocke le contenu du registre Rr à l'adresse k en SRAM.

- **k** : adresse en SRAM (16 bits)
- **Rr** : registre source (R0 à R31)
- **Cycles** : 2
- **SREG** : non modifié
- **Note** : c'est l'opération **inverse** de LDS

</> Stocker un résultat en SRAM

```
ldi r16, 5          ; R16 = 5
ldi r17, 3          ; R17 = 3
add r16, r17        ; R16 = 8
sts 0x0100, r16     ; Stocker 8 a l'adresse SRAM 0x0100
```



 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.4.5 Tableau récapitulatif des instructions de transfert

Table 3.4 – Les 4 instructions de transfert fondamentales en assembleur AVR.

	Instruction	Cycles	Taille	Registres	Description
→	`LDI Rd, K`	1	16 bits	R16–R31	Charge une valeur constante (0–255) dans un registre
⇄	`MOV Rd, Rr`	1	16 bits	R0–R31	Copie un registre dans un autre
↓	`LDS Rd, k`	2	32 bits	R0–R31	Charge depuis une adresse SRAM dans un registre
↑	`STS k, Rr`	2	32 bits	R0–R31	Stocke un registre à une adresse SRAM

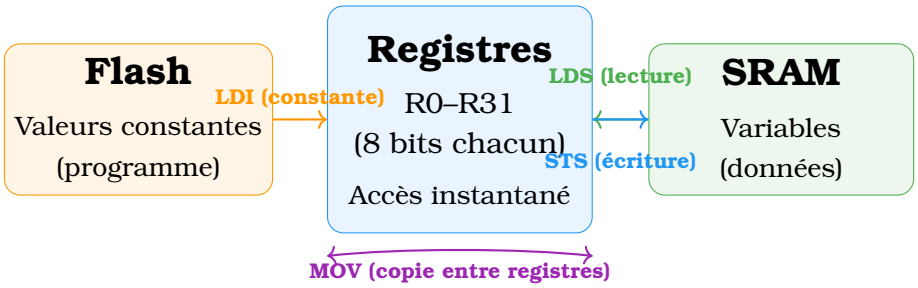


Figure 3.4 – Les flux de données entre registres, Flash et SRAM via les instructions de transfert.

3.5 Premier programme : analyse ligne par ligne

Reprenons le programme du listing 3.1 et analysons-le **instruction par instruction** en traçant l'état complet de la machine.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.5.1 Trace d'exécution détaillée

Trace d'exécution pas à pas

PC	#	Instruction	R16	R17	SREG	Explication
0x0000	0	RJMP main	?	?	0x00	Le µC démarre. PC saute vers `main`. Registres non initialisés.
0x0001	1	LDI R16, 5	0x05	?	0x00	Charge 5 dans R16. LDI ne modifie pas SREG.
0x0002	2	LDI R17, 3	0x05	0x03	0x00	Charge 3 dans R17. R16 inchangé.
0x0003	3	ADD R17, R16	0x05	0x08	Z=0 C=0 N=0	R17 = 3+5 = **8**. Pas de retenue, pas de zéro, résultat positif.
0x0004	4	RJMP endless	0x05	0x08	inchangé	Boucle infinie. Résultat (8) conservé dans R17.

3.5.2 Vérification en binaire

```
  00000101   (5 = R16)
+ 00000011   (3 = R17)
  --------
  00001000   (= 8)
```

Vérification : $5 + 3 = 8$ ✓Pas de retenue ✓Résultat $\neq 0$ ✓

3.5.3 Exercice guidé : modifier le programme

Essayez par vous-même !

Modifiez le programme pour calculer $200 + 100$:

```
ldi r16, 200        ; Charger 200 dans R16
ldi r17, 100        ; Charger 100 dans R17
add r17, r16        ; R17 = 100 + 200 = ???
```

Questions :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

1. Quelle valeur contient R17 après l'addition ?
2. Quel drapeau SREG est activé ?
3. Pourquoi le résultat n'est-il pas 300 ?

Réponse : $200 + 100 = 300$, mais sur 8 bits le maximum est 255. Donc $300 - 256 = 44$. R17 = 44, et le drapeau C (Carry) = 1.

3.6 Comparaison avec l'assembleur x64 (ouverture)

Pour élargir votre perspective, voici la même addition en assembleur x64 (architecture des PC modernes) :

Table 3.5 – La même addition dans les deux architectures.

Étape	**AVR (8 bits)**	**x64 (64 bits)**
Charger 5	`LDI R16, 5`	`mov rax, 5`
Charger 3	`LDI R17, 3`	`mov rbx, 3`
Additionner	`ADD R16, R17`	`add rax, rbx`
Résultat	R16 = 8 (max 255)	RAX = 8 (max $2^{64} - 1$)
Fin du programme	`rjmp endless`	`call ExitProcess`

Remarque

Les concepts sont **identiques** : charger des valeurs dans des registres, effectuer une opération, vérifier les drapeaux. Seule la syntaxe et la taille des registres changent. Ce que vous apprenez en assembleur AVR est directement transposable à n'importe quelle autre architecture.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.7 Conclusion du chapitre

Ce que vous avez appris

1. **La traduction :** chaque mnémonique assembleur correspond à un opcode binaire précis. La correspondance est 1 :1.
2. **L'encodage :** une instruction AVR fait 16 bits, découpée en champs (opcode, registre, valeur).
3. **La structure d'un programme :** directives (`.include`, `.def`, `.equ`, `.org`), étiquettes (`main:`), instructions, commentaires (`;`).
4. **Les registres :** R0–R15 (restreints), R16–R31 (complets), R26–R31 (pointeurs X, Y, Z).
5. **Le SREG :** 8 drapeaux automatiques. Les 4 principaux : Z (zéro), C (carry), N (négatif), V (overflow).
6. **Les transferts :** `LDI` (constante → registre), `MOV` (registre → registre), `LDS` (SRAM → registre), `STS` (registre → SRAM).
7. **L'analyse :** vous savez tracer l'exécution d'un programme instruction par instruction, registre par registre.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Fiche de révision express

Concept	Ce qu'il faut retenir
Mnémonique	Nom lisible d'une instruction machine (LDI, MOV, ADD...)
Directive	Instruction pour l'assembleur, pas pour le CPU (`.include`, `.def`, `.equ`)
Étiquette	Nom symbolique d'une adresse (`main:`, `loop:`)
LDI Rd, K	Charge une constante (0–255) dans R16–R31 uniquement
MOV Rd, Rr	Copie Rr dans Rd (tous registres, SREG inchangé)
LDS Rd, k	Charge depuis l'adresse SRAM k dans Rd (2 cycles)
STS k, Rr	Stocke Rr à l'adresse SRAM k (2 cycles)
SREG	Z=zéro, C=carry, N=négatif, V=overflow, H=half-carry
Boucle infinie	`rjmp endless` — obligatoire sur µC (pas d'OS)
Commentaire	`; texte` — ignoré par l'assembleur, vital pour l'humain

Fin du Chapitre 3 — Fondements du langage assembleur

Vous savez maintenant lire, écrire et analyser un programme assembleur AVR simple.

Chapitre suivant : ***Calculs et contrôle du flux***

Instructions arithmétiques (`ADD`, `SUB`, `MUL`), logiques (`AND`, `OR`, `EOR`), comparaisons (`CP`, `CPI`), branchements (`BREQ`, `BRNE`), boucles et conditions.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.8 Travaux dirigés du chapitre 2

Travaux dirigés : Du code source à l'exécution

5 exercices progressifs ancrés dans des situations réelles — traduction, structure, registres, transferts, analyse de programmes

⇄ TD 1 — Traduction et mnémoniques

Contexte : Vous êtes développeur firmware junior. Votre responsable vous demande de vérifier que vous comprenez le lien entre le code assembleur et le code machine binaire exécuté par le processeur.

Situation 1 — Reconnaître les mnémoniques

Associez chaque mnémonique à sa signification en français, puis donnez le sens de chacun de ses opérandes.

Mnémonique	Signification (anglais)	Ce qu'elle fait (français)
`LDI R16, 42`	_____	_____
`MOV R5, R16`	_____	_____
`ADD R16, R17`	_____	_____
`SUB R16, R17`	_____	_____
`LDS R18, 0x0100`	_____	_____
`STS 0x0100, R18`	_____	_____
`RJMP main`	_____	_____
`NOP`	_____	_____

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Situation 2 — Décodage d'instructions binaires

Voici 4 mots de 16 bits extraits de la Flash d'un ATmega328P. Pour chacun, identifiez (si possible) l'instruction assembleur correspondante.

#	Binaire (16 bits)	Instruction assembleur
1	`1110 0000 0000 0101`	_____
2	`1110 1111 1111 1111`	_____
3	`0000 1100 0000 0001`	_____
4	`1100 1111 1111 1111`	_____

Indice : L'opcode de LDI commence par `1110`, celui de ADD par `0000 11`, celui de RJMP par `1100`. LDI avec K=255 s'écrit `1110 1111 Rd 1111`.

Situation 3 — Encodage d'une instruction

L'instruction `LDI Rd, K` est encodée ainsi sur 16 bits :

$$\underbrace{1110}_{\text{opcode}}\ \underbrace{K_7K_6K_5K_4}_{\text{bits 7-4 de K}}\ \underbrace{d_3d_2d_1d_0}_{\text{Rd - 16}}\ \underbrace{K_3K_2K_1K_0}_{\text{bits 3-0 de K}}$$

a) Encodez l'instruction `LDI R20, 100` sur 16 bits. Montrez chaque étape.

b) Encodez l'instruction `LDI R31, 0xFF`. Quelle est la valeur hexadécimale de ce mot de 16 bits ?

c) Pourquoi `LDI R15, 42` est-elle impossible à encoder avec ce format ?

d) Si le champ `Rd` ne faisait que 3 bits au lieu de 4, quels registres LDI pourrait-il cibler ?

TD 2 — Structure d'un programme assembleur AVR

Contexte : Vous rejoignez une équipe qui développe le firmware d'un thermostat. Votre première tâche est de comprendre et corriger un programme

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

assembleur existant.

Situation 1 — Identifier les éléments d'un programme

Examinez le programme suivant et identifiez chaque élément en annotant les lignes numérotées.

```
1  .include "m328Pdef.inc"
2  .def temp = r16
3  .equ SEUIL = 25
4  .org 0x0000
5      rjmp debut
6  debut:
7      ldi temp, SEUIL
8      sts 0x0200, temp
9  boucle:
10     nop
11     rjmp boucle
```

Pour chaque ligne, indiquez si c'est : une **directive**, une **étiquette**, une **instruction** ou un **commentaire** manquant.

Ligne	Type	Justification	Commentaire à ajouter
1	_____	_____	_____
2	_____	_____	_____
3	_____	_____	_____
4	_____	_____	_____
5	_____	_____	_____
6	_____	_____	_____
7	_____	_____	_____
8	_____	_____	_____

Situation 2 — Corriger les erreurs de structure

Le programme suivant contient 5 erreurs de structure. Identifiez-les et

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

proposez la correction.

```
.equ F_CPU = 16000000
.def temperature = r5
.org 0x0000
ldi temperature, 20
boucle
    rjmp boucle
```

Erreur 1 : _____

Erreur 2 : _____

Erreur 3 : _____

Erreur 4 : _____

Erreur 5 : _____

Situation 3 — Écrire un programme structuré

Écrivez un programme assembleur AVR **complet et commenté** pour ATmega328P qui :

- Définit un alias `compteur` pour R20
- Définit une constante `LIMITE` valant 10
- Charge la valeur 0 dans `compteur`
- Charge la valeur `LIMITE` dans R21
- Stocke R21 à l'adresse SRAM `0x0100`
- Termine par une boucle infinie avec l'étiquette `fin`

Votre programme doit respecter la structure vue en cours (`.include`, `.def`, `.equ`, `.org`, `rjmp main`, commentaires).

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

TD 3 — Registres et registre de statut SREG

Contexte : Vous déboguez un programme embarqué et devez comprendre précisément l'état des registres et des drapeaux après chaque opération.

Situation 1 — Classification des registres AVR

a) Classifiez les registres suivants en trois catégories (restreints R0–R15, usage complet R16–R25, pointeurs R26–R31) : R3, R16, R27, R0, R25, R30, R12, R31, R20, R8.

b) Pour chaque instruction ci-dessous, indiquez si elle est **valide** ou **invalide**. Si invalide, expliquez pourquoi et proposez une correction.

Instruction	Valide ?	Raison/Correction
`LDI R16, 100`	_____	_____
`LDI R8, 50`	_____	_____
`LDI R17, 256`	_____	_____
`MOV R0, R16`	_____	_____
`LDI R31, 0xFF`	_____	_____
`LDI R16, -1`	_____	_____

c) R0 ne supporte pas LDI. Écrivez une séquence de 2 instructions pour placer la valeur 200 dans R0.

Situation 2 — Analyse du registre SREG

Pour chaque opération AVR ci-dessous, calculez le résultat et indiquez l'état des drapeaux Z, C, N et V du SREG. Justifiez en montrant le calcul binaire.

a) `LDI R16, 5` puis `LDI R17, 3` puis `ADD R16, R17`

R16 après ADD = _____ Z = __ C = __ N = __ V = __

b) `LDI R16, 200` puis `LDI R17, 100` puis `ADD R16, R17`

R16 après ADD = _____ Z = __ C = __ N = __ V = __

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

c) `LDI R16, 5` puis `SUB R16, R16`
R16 après SUB = _____ Z = __ C = __ N = __ V = __

d) `LDI R16, 1` puis `LDI R17, 2` puis `SUB R16, R17`
R16 après SUB = _____ Z = __ C = __ N = __ V = __

Situation 3 — Rôle du PC et du SP

a) Le compteur de programme (PC) vaut `0x0003` avant l'exécution de `LDI R16, 42` (instruction sur 1 mot = 2 octets). Quelle est la valeur du PC après l'exécution ?

b) L'instruction `LDS R16, 0x0100` occupe **2 mots** (4 octets) en Flash. Si le PC vaut `0x0010` avant son exécution, quelle est sa valeur après ?

c) À quoi sert le pointeur de pile (SP) ? Donnez un exemple concret de son utilisation.

d) Le SP de l'ATmega328P est initialisé à la valeur `RAMEND` (= `0x08FF`) au démarrage. Si on exécute `PUSH R16` (empiler 1 octet), quelle sera la nouvelle valeur du SP ? *Rappel : PUSH décrémente SP de 1.*

→ TD 4 — Instructions de transfert : LDI, MOV, LDS, STS

Contexte : Vous programmez un capteur de température embarqué. Les données transitent entre les registres et la SRAM. Vous devez écrire et analyser des séquences de transfert.

Situation 1 — Utilisation de LDI et MOV

a) Tracez l'état des registres après chaque instruction :

```
1 ldi r16, 100      ; Apres : R16 = ___
2 ldi r17, 0x0A     ; Apres : R17 = ___
3 mov r18, r16      ; Apres : R18 = ___ , R16 = ___
4 mov r5, r17       ; Apres : R5  = ___ , R17 = ___
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

```
ldi r19, 0b11110000 ; Apres : R19 = ___
mov r20, r19        ; Apres : R20 = ___
```

b) Vous devez charger la valeur 200 dans R5 (qui n'accepte pas LDI). Écrivez la séquence de 2 instructions.

c) Quelle est la différence entre :

- `LDI R16, 0` et `MOV R16, R16` ?
- Les deux mettent-ils R16 à zéro ?
- Laquelle recommandez-vous pour initialiser un registre à zéro ? Pourquoi ?

Situation 2 — LDS et STS : échanges avec la SRAM

La SRAM de l'ATmega328P commence à l'adresse `0x0100`. Supposons que les adresses SRAM suivantes contiennent ces valeurs :

Adresse SRAM	Valeur
0x0100	42
0x0101	17
0x0102	255
0x0103	0

a) Tracez l'état des registres et de la SRAM après chaque instruction de la séquence suivante :

```
lds r16, 0x0100   ; R16 = ___
lds r17, 0x0101   ; R17 = ___
add r16, r17      ; R16 = ___
sts 0x0104, r16   ; SRAM[0x0104] = ___
```

b) Écrivez une séquence qui :

a) Lit la valeur à `0x0102`

b) L'additionne avec la valeur à `0x0103`

c) Stocke le résultat à `0x0105`

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

c) LDS et STS prennent 2 cycles chacune, alors que LDI et MOV n'en prennent qu'un seul. Expliquez pourquoi.

Situation 3 — Comparaison AVR et x64

a) Associez chaque instruction AVR à son équivalent x64 :

AVR	x64 équivalent
`LDI R16, 42`	`add rax, rbx`
`MOV R17, R16`	`mov rax, 42`
`ADD R16, R17`	`mov [0x0100], rax`
`LDS R16, 0x0100`	`mov rbx, rax`
`STS 0x0100, R16`	`mov rax, [0x0100]`

b) Quelle est la principale différence entre `LDI R16, 42` (AVR) et `mov rax, 42` (x64) en termes de :

- Taille de la valeur maximale chargeable
- Nombre de registres pouvant être cibles
- Taille de l'instruction en mémoire

TD 5 — Analyse complète d'un programme assembleur AVR

Contexte : Vous recevez un programme assembleur AVR sans commentaires. Votre mission est de l'analyser complètement, de le commenter, et d'en comprendre l'objectif.

Programme à analyser

```
1 .include "m328Pdef.inc"
2 .def a   = r16
3 .def b   = r17
```

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

```
.def res = r18

.org 0x0000
    rjmp main

main:
    ldi a, 12
    ldi b, 8
    mov res, a
    add res, b
    sts 0x0200, res
    sub res, b
    sts 0x0201, res
    mov r0, a
    add r0, a
    sts 0x0202, r0
fin:
    rjmp fin
```

Situation 1 — Trace d'exécution

Complétez le tableau de trace d'exécution ligne par ligne :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Ligne	Instruction	a	b	res	r0	SREG (Z,C)	SRAM
9	`ldi a, 12`	12	?	?	?	–	–
10	`ldi b, 8`	12	8	?	?	–	–
11	`mov res, a`	12	8	____	?	–	–
12	`add res, b`	12	8	____	?	_____	–
13	`sts 0x0200, res`	12	8	____	?	–	_____
14	`sub res, b`	12	8	____	?	_____	–
15	`sts 0x0201, res`	12	8	____	?	–	_____
16	`mov r0, a`	12	8	____	____	–	–
17	`add r0, a`	12	8	____	____	_____	–
18	`sts 0x0202, r0`	12	8	____	____	–	_____

Situation 2 — Compréhension et commentaires

a) Que contiennent les trois adresses SRAM après l'exécution ?

- `0x0200` = _____ (ce que ce calcul représente : _____)
- `0x0201` = _____ (ce que ce calcul représente : _____)
- `0x0202` = _____ (ce que ce calcul représente : _____)

b) En une phrase, quel est l'objectif de ce programme ?

c) Ajoutez des commentaires pertinents sur chaque ligne d'instruction du programme.

d) La ligne 16 `mov r0, a` est-elle nécessaire ? Pourquoi ne peut-on pas écrire directement `add a, a` pour obtenir $2 \times 12 = 24$? *Indice : `add` modifie le registre destination.*

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Situation 3 — Extension du programme

a) Modifiez le programme pour calculer également la **différence** $a - b$ et la stocker à l'adresse `0x0203`.

b) Ajoutez une séquence qui charge les valeurs stockées à `0x0200` et `0x0201` dans deux registres, puis les additionne et stocke le résultat à `0x0204`.

c) Que faudrait-il ajouter pour vérifier si le résultat de `add res, b` a provoqué un dépassement (Carry = 1) ? Quel drapeau SREG surveiller ?

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

3.9 Quiz du chapitre 2 : testez vos connaissances

Vrai/Faux QCM Pratique

Répondez sans regarder le cours. Temps estimé : 15 minutes.

1 QCM

Quelle est la différence fondamentale entre une mnémonique et un opcode ?

A. La mnémonique est binaire, l'opcode est textuel

B. La mnémonique est un texte lisible, l'opcode est le code binaire correspondant dans le processeur

C. Ce sont deux noms différents pour la même chose

D. L'opcode est écrit par le programmeur, la mnémonique par le processeur

2 VRAI/FAUX

L'instruction `MOV R17, R16` copie le contenu de R17 dans R16 (R16 reçoit la valeur de R17).

☐ **Vrai** ☐ **Faux**

Indice : `MOV Rd, Rr` — qui est la destination (d) et qui est la source (r) ?

3 QCM

Après l'exécution de `LDI R16, 200` suivie de `LDI R17, 100` et `ADD R16, R17`, que vaut R16 ?

A. 300

B. 44 (et le drapeau Carry = 1)

C. 200 (R16 n'est pas modifié)

D. 255 (valeur maximale sur 8 bits)

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

4 VRAI/FAUX

Une directive assembleur (comme `.def` ou `.equ`) est traduite en instruction machine et occupe de la place en Flash.

☐ **Vrai** ☐ **Faux**

Indice : une directive est une instruction pour l'assembleur (l'outil), pas pour le processeur.

5 QCM

Quelle instruction utilise-t-on pour lire une valeur stockée en SRAM et la placer dans un registre ?

A. `LDI Rd, k`

B. `MOV Rd, Rr`

C. `LDS Rd, k`

D. `STS k, Rr`

6 VRAI/FAUX

Le drapeau Z (Zero) du SREG est activé (Z=1) quand le résultat d'une opération est exactement 0.

☐ **Vrai** ☐ **Faux**

Indice : testez avec `LDI R16, 5` puis `SUB R16, R16`.

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

7 PRATIQUE

Identifiez l'erreur dans chaque ligne et proposez la correction :

a) `ldi r5, 42`
Erreur : _____
Correction : _____

b) `ldi r16, 256`
Erreur : _____
Correction : _____

c) `sts r16, 0x0100`
Erreur : _____
Correction : _____

d) `.def compteur = r5 = r6`
Erreur : _____
Correction : _____

8 QCM

Pourquoi un programme assembleur AVR se termine-t-il toujours par une boucle infinie (`rjmp endless`) ?

A. Pour économiser de l'énergie en mode veille

B. Le µC n'a pas de système d'exploitation pour reprendre la main ; sans boucle infinie, le PC exécuterait des données aléatoires

C. C'est une convention mais pas obligatoire

D. L'instruction `rjmp` est plus rapide que `nop`

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

9 PRATIQUE

Tracez l'état des registres et du SREG après chaque instruction :

```
ldi r16, 10
ldi r17, 10
sub r16, r17
```

Après `ldi r16, 10` : R16 = _____ SREG = inchangé

Après `ldi r17, 10` : R17 = _____ SREG = inchangé

Après `sub r16, r17` : R16 = _____

Z = ____ C = ____ N = ____

Que signifie Z = 1 dans ce contexte ? _____

10 PRATIQUE

Défi programme : écrivez un programme AVR complet, structuré et commenté qui :

- Charge la valeur 15 dans R16
- Charge la valeur stockée à l'adresse SRAM `0x0100` dans R17 (supposée valoir 5)
- Calcule R16 + R17 et stocke le résultat à `0x0101`
- Calcule R16 − R17 et stocke le résultat à `0x0102`
- Se termine par une boucle infinie

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

10

Écrivez votre programme ici :

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Récapitulatif : Ce que vous devez maîtriser

Traduction et mnémoniques

- Mnémonique = nom lisible d'une instruction
- Opcode = code binaire correspondant
- Correspondance 1 :1 (pas d'optimisation)
- LDI encodé sur 16 bits : opcode + Rd + K
- LDI ne fonctionne que sur R16–R31

Structure d'un programme

- Directives (`.include`, `.def`, `.equ`, `.org`)
- Étiquettes (`main:`, `boucle:`)
- Instructions assembleur
- Commentaires (`; texte`)
- Boucle infinie obligatoire sur µC

Registres et SREG

- R0–R15 : usage restreint (pas de LDI)
- R16–R31 : toutes les instructions
- R26–R31 : pointeurs X, Y, Z
- SREG : Z (zéro), C (carry), N (négatif), V (overflow)
- LDI et MOV ne modifient pas SREG

Instructions de transfert

- `LDI Rd, K` : constante → registre
- `MOV Rd, Rr` : registre → registre
- `LDS Rd, k` : SRAM → registre
- `STS k, Rr` : registre → SRAM
- LDS/STS : 2 cycles (accès mémoire)

Compétences clés pour la suite :

Lire du code assembleur AVR sans erreur • Tracer l'état des registres instruction par instruction • Identifier les drapeaux SREG Utiliser LDI, MOV, LDS, STS correctement • Commenter tout le code assembleur • Structurer un programme AVR complet

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Fin des exercices — Chapitre 2

Vous avez pratiqué les fondements du langage assembleur :

Mnémoniques • Structure • Registres • SREG • Transferts • Analyse

Chapitre suivant : ***Calculs et contrôle du flux***

Instructions arithmétiques et logiques, comparaisons, branchements, boucles et conditions

 © 2025–2026 Dr AFENYIVEH Dzo M. Serge

Table des matières détaillée